✨ A cook-along study guide ✨

V1.0

Fengwei Fried Rice

We're Cooking AI!

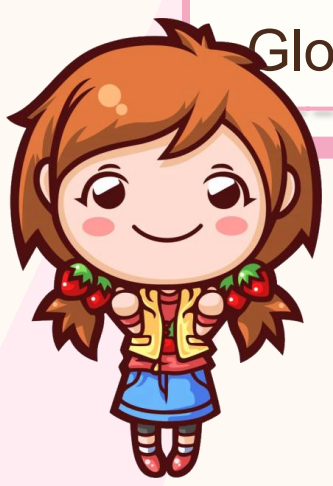
📒 About this guide

Module: Hardware, Networks & Behaviourist AI

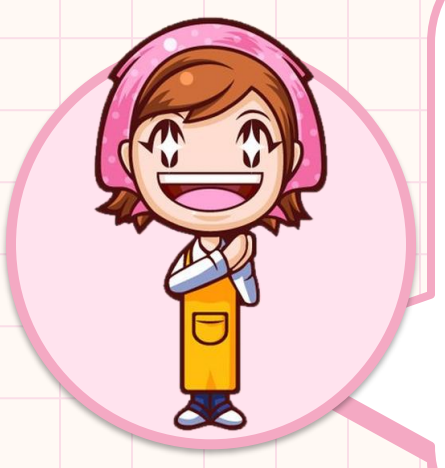
Course: Introduction to Artificial Intelligence

Team: Fengwei Fried Rice · **Version:** V1.0 · November 2026

Inside: 5 recipes · Explain-first lessons · 10 hands-on activities · 5 Kitchen Labs · 🎮 Bonus maze game · Glossary A–Z



Welcome, Apprentice Chef!



Hi, I'm **Mama**! Your phone does **billions** of tiny maths steps every time it unlocks with your face. In this kitchen we'll find out **who** does that maths (chips), **how** the ingredients travel (networks) and **how** a robot learns by trial and error (rewards).

💡 Why does this matter?

AI is not magic software floating in the air. Every answer an AI gives is produced by real chips, carried by real networks and shaped by real feedback. If you understand those three things, you can explain why an AI is fast or slow, cheap or expensive, private or not, and why engineers design systems the way they do.

🤯 Five surprising facts you'll be able to explain

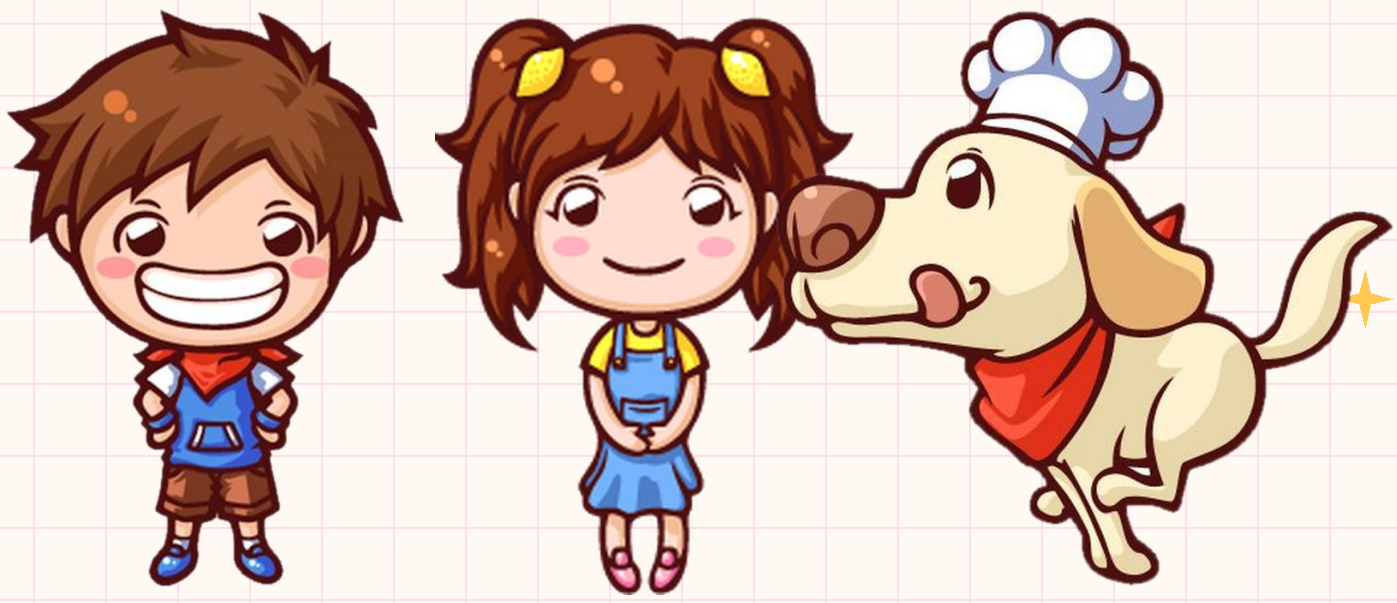
Moving one number from memory can cost about **170×** more energy than multiplying it. (Recipe 01)

Light needs about **40 ms** to go from Beijing to New York and back through fibre. (Recipe 02)

Robot dogs can learn to walk in minutes by training **thousands** of virtual dogs at once. (Recipe 03)

At 150 m, a child is only about **five pixels** wide in a car camera. (Recipe 04)

A 4,096-chip AI job breaks roughly every **11 hours**, so it saves itself often. (Recipe 05)



How This Kitchen Works

Every recipe (module) follows the **same cook-along route**, so you always know what to do next:

1 · Explain
Read the concept first

→

2 · Try
Hands-on activity

→

3 · Watch
Curated resources

→

4 · Review
Check, recap, glossary

Inside each section, a question leads the way: **Encounter a problem → Predict → Observe → Explain → Apply → Check.** About two-thirds of every recipe is explanation; the rest is practice that uses what you just read.

Label	What it means
🔮 **Predict**	Make a guess **before** reading. Wrong guesses are welcome: they show what to look for.
🍳 **Explain**	The core teaching. Read these carefully; everything else builds on them.
🥢 **Analogy**	A kitchen picture to help you remember. **Always** followed by…
🧂 **Breaks down**	…where the analogy stops being true. The technical fact wins.
⚠️ **Chef's Warning**	A common misconception, and the correction.
🧪 **Try It**	An activity with six parts: Goal · Preparation · Task · Observation · Reflection · Feedback.
✅ **Check** / 📌 **Recap**	Quick self-test and the 3–5 points to remember.

What You'll Learn

Learning Goals

1. Explain what CPUs, GPUs, TPUs, NPUs and memory each do for AI, and **why** their designs differ. (Recipe 01)
2. Explain how networks move AI data, and choose cloud or edge using latency and bandwidth. (Recipe 02)
3. Describe how an agent learns from rewards, and balance exploration and exploitation. (Recipe 03)
4. Compare cameras, LiDAR, ultrasonic and radar sensors, and explain sensor fusion. (Recipe 04)
5. Explain how hardware, networks and software are combined and optimised at scale. (Recipe 05)

You need

Basic arithmetic and powers of ten.

What a variable and a loop are.

A rough idea of what a CPU and the internet are.

No electronics or advanced maths.

Who & how long

1st/2nd-year CS & IT students.

About **2–3 hours** per recipe.

Pen, paper, calculator; a phone or laptop for Kitchen Labs (offline fallback included).

Chef's Warning

"So a GPU really **is** 1,000 cooks!" Not quite! Analogies help you remember, but they are not exact. Every // Analogy is followed by 🧂 Where it breaks down, so you never mix the picture up with the technical truth.

Today's Menu

01

Recipe 01 · AI Hardware

CPU · GPU · TPU · NPU · parallelism · memory · Apple Neural Engine

👉 starts on page 6 · tap to jump

02

Recipe 02 · Networks & AI Infrastructure

layers · TCP/IP · cloud vs edge · latency · 5G

👉 starts on page 36 · tap to jump

03

Recipe 03 · Behaviourist AI & RL

rewards · agent loop · exploration · robot dogs

👉 starts on page 65 · tap to jump

04

Recipe 04 · AI Sensors

camera · LiDAR · ultrasonic · fusion · Tesla

👉 starts on page 91 · tap to jump

05

Recipe 05 · Integration & Optimisation

software stack · distributed AI · energy · Google TPU

👉 starts on page 119 · tap to jump

🍰 **Dessert**

Then: Glossary A–Z · References & image credits · Graduation 🎓

🎮 **Bonus Kitchen Lab**

Pup's Pantry Maze: an interactive game comparing fixed rules, search and reinforcement learning · tap to jump

RECIPE 01

The Kitchen Crew

AI Hardware Architecture

Mama's Problem

"1,000 plates of fried rice in one minute! Who should cook: one master chef, a thousand helpers, or a machine?" AI has the same problem: billions of tiny maths steps, and we must pick the right cook for the job.

Today's route

1.1 What does AI cook? Definitions and the MAC

1.2 Meet the crew: CPU vs GPU vs TPU vs NPU

1.3 Cooking in parallel: acceleration and its limits

1.4 The pantry problem: memory hierarchy and efficiency

1.5 Case study: the Apple Neural Engine (on-device AI)

Before you start

Basic arithmetic, powers of ten, a variable and a loop. Every new word is explained the first time it appears.

Cooking time

~3 hours: 2 h reading, 1 h activities and Kitchen Lab.

1.1 What Is AI Hardware?

Predict

Your phone hides at least **five** kinds of AI hardware. How many can you name before you read on? Write your list down; you'll check it in a minute.

Why does this matter?

Every AI model, from a chatbot to Face ID, ends up as an enormous number of arithmetic operations on physical chips. The chip you choose decides how **fast** a model trains, how much **electricity** it uses, how quickly it **answers** one request, and whether it can run on a **phone** at all.

Explain

AI hardware is any physical computing device (a processor, a memory system, or the wiring between them) that is designed, or is well suited, to run machine-learning workloads **efficiently**.

The word **efficiently** matters. A laptop CPU can run a neural network, and so could a calculator if you were patient enough. The real questions are: **How fast?** (time) **How much energy?** (battery, electricity bill, heat) **At what cost?** (price of chips and data centres).

1.1 The Five Kinds of AI Hardware

Category	What it is	Examples
General-purpose processors	Designed for any program, but can run AI	CPUs (Intel, AMD, Arm)
Massively parallel processors	Thousands of simple cores doing the same thing to different data	GPUs (NVIDIA, AMD)
Domain-specific accelerators	Built around the arithmetic of neural networks	TPU (Google), NPU (Apple Neural Engine)
Memory systems	Fast memory placed near the compute units	On-chip SRAM, HBM
Interconnects	Links between chips and machines	NVLink, InfiniBand, Ethernet

Table 1.1 · The five categories of AI hardware.

👀 **How to read this:** read each row left to right: the kind of hardware, what makes it special, and real products. Notice that only the top three rows actually compute; the bottom two feed and connect them.

Kitchen Analogy

A kitchen isn't only cooks. It's the **cooks** (processors), the **fridge and pantry** (memory) and the **hallways between kitchens** (interconnects). All of them decide how fast dinner arrives: the best chef in the world is slow if the ingredients are across town.

Where the analogy breaks down

Real hallways carry one cook at a time; chip links carry **billions of bytes per second**, and memory is not "storage" you walk to: data is copied up and down a hierarchy (see 1.4).

1.1 What Does AI Actually Cook?

Predict

A neural network "thinks" by doing… **(a)** logic puzzles, **(b)** one tiny maths move, billions of times, or **(c)** internet searches? Pick one before reading.

How it works, step by step

A neural network is built from **layers**. Almost every layer does one dominant job: it multiplies its **inputs** by numbers it learned during training, called **weights**, and adds the results up.

Step 1. Start a running total at 0.

Step 2. Take one input and its weight. Multiply them.

Step 3. Add the product to the running total.

Step 4. Repeat for every input. The final total is the neuron's output.

One "multiply, then add to the running total" step is called a **MAC** (multiply-accumulate). So the answer is **(b)**: AI is mostly MACs, repeated billions of times.

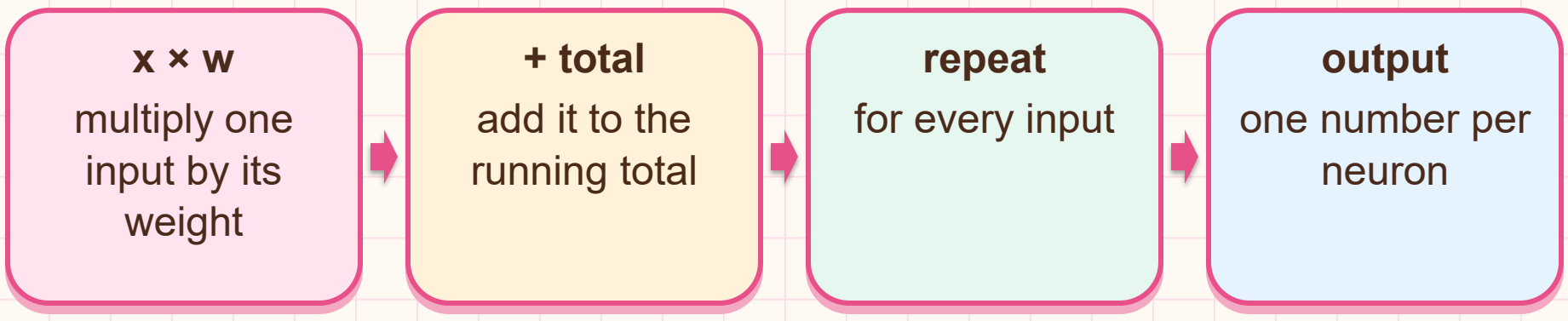


Figure 1.2 · One neuron as a chain of MACs.

How to read this: follow the arrows left to right. The first two boxes are ONE MAC; the third box says how many times it happens.

Real Tech

1 MAC = **2 FLOPs** (one multiply + one add; FLOP = floating-point operation). Grouping weights into a **matrix** lets hardware do many MACs at once. A **tensor** is a many-dimensional matrix: the "T" in TPU.

1.1 One Neuron, By Hand

Worked example (illustrative numbers)

```
Weights  w = [0.5, -1.0, 2.0]      Inputs  x = [4, 3, 1]
Start:   total = 0
MAC 1:   total = 0    + 0.5 x 4     = 2.0
MAC 2:   total = 2.0  + (-1.0 x 3)  = -1.0
MAC 3:   total = -1.0 + 2.0 x 1     = 1.0   -> output 1.0,
3 MACs
```

Explain

Now scale it up. One layer might have **4,096** neurons, each with **1,024** inputs: 1,024 × 4,096 = **4,194,304 MACs** for a single input. A large language model needs billions to trillions of MACs for every answer.

Cause and effect: because MACs are nearly all of the work, a chip that does MACs fast and cheaply speeds up almost the whole model. That is why every AI chip in this recipe is, at heart, a MAC machine.

Kitchen Analogy

A MAC is like adding **one ingredient × its amount** into the wok, again and again. The wok holds the running total.

Where the analogy breaks down

A wok never "subtracts" an ingredient, but weights can be **negative** (−1.0 above), which lowers the total.

Chef's Warning

"8 GB? 8 Gb? Same thing!" **Not quite!** 1 byte = 8 bits. An FP32 number takes 4 bytes, FP16 2 bytes, INT8 1 byte. A 7-billion-weight model in FP16 needs $7 \times 10^9 \times 2$ = **14 GB**, not 14 Gb. Many hardware mistakes come from skipping this conversion.

1.2 Meet the Kitchen Crew

Predict

You must cook **one million** identical dumplings. Would you hire **(a)** four brilliant master chefs, **(b)** a thousand junior cooks, or **(c)** a dumpling machine? Now: what if tomorrow's order is a wedding banquet with 40 different dishes?

Why does this matter?

There is no single "best" AI chip. Each family makes a different **trade-off between flexibility and efficiency**. If you understand that trade-off, you can predict which chip a phone, a data centre or a robot will use, and explain why.

Explain

A **processor** reads **instructions** (tiny commands such as "multiply these two numbers") and carries them out on **cores**: units that can each execute instructions on their own. Every chip spends its limited area (its **silicon**) on three things: **arithmetic units** that do the maths, **control logic** that decides what to do next, and **memory** that keeps data close by.

How a design splits that area decides what it is good at. Spend it on control and you get a flexible chip that handles any program. Spend it on arithmetic and you get a chip that is fast and efficient at **one** kind of maths, and clumsy at everything else.

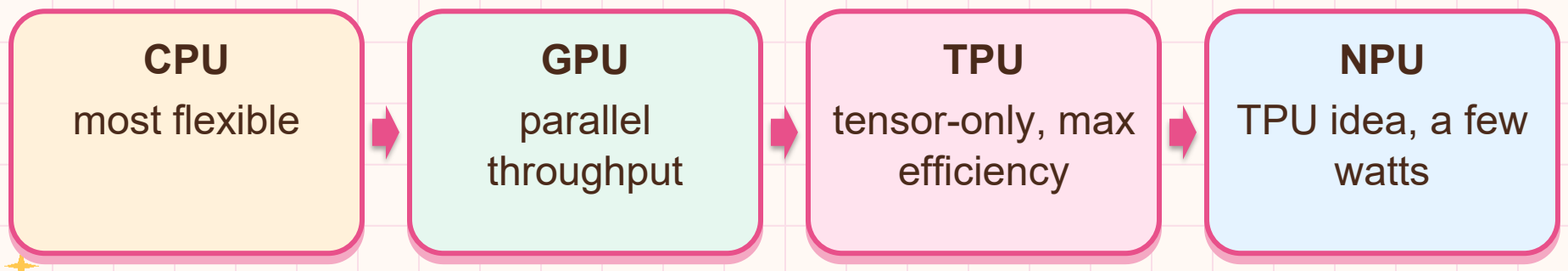


Figure 1.3 · The flexibility–efficiency spectrum.

How to read this: moving right, each chip gives up generality to gain efficiency on neural-network maths. The NPU is the TPU idea shrunk to fit a phone.

1.2 The CPU: The Head Chef

Explain

A **CPU** (Central Processing Unit) has **a few** very capable cores (from a handful to about a hundred). Each core runs one stream of instructions, a **thread**, as fast as possible, at a clock speed of a few **GHz** (billions of steps per second).

How it works, step by step

How a CPU core works on a task:

1. **Fetch** the next instruction from memory.
2. **Decode** it: work out what it asks for.
3. **Execute** it on an arithmetic unit.
4. **Write back** the result, then move to the next instruction.

To keep this loop fast, most of a CPU's area goes on clever helpers rather than arithmetic: **caches** (small fast memories holding recently used data), **branch prediction** (guessing which way an 'if' will go so work never stops) and **out-of-order execution** (running later instructions early when it is safe, instead of waiting).

👍 Good at

Any program, any language.

Lowest **latency** for one task.

AI jobs: data loading, pre-processing, orchestration, small models.

👎 Weak at

Too **few** arithmetic units for millions of independent MACs.

Lowest energy efficiency for AI.

Kitchen Analogy

The CPU is a **head chef**: brilliant, can cook any dish in the book, adapts instantly when the order changes, but only has two hands.

1.2 The GPU: 1,000 Line Cooks

Explain

A **GPU** (Graphics Processing Unit) has **thousands** of simple cores. It was built to draw graphics: to colour millions of pixels, it runs the **same instruction on lots of data** at once. Multiplying big matrices is exactly the same kind of problem, which is why GPUs became the workhorse of AI training.

How it works, step by step

How a GPU processes work:

1. The job is split into thousands of tiny threads (for example, one per output number).
2. Threads are grouped; every thread in a group runs the **same** instruction on **different** data at the same moment (NVIDIA groups 32 threads, called a **warp**).
3. When one group must wait for memory, the scheduler instantly switches to another group that is ready. Lots of waiting threads **hide** memory delays instead of big caches.
4. Modern AI GPUs add **Tensor Cores**: units that do a small matrix multiply in one step, and very fast stacked memory (**HBM**) to keep them fed.

👍 Good at

Huge **throughput** on parallel maths.

Programmable (CUDA, PyTorch).

AI jobs: training and serving large models.

👎 Weak at

Draws more power than a custom chip.

Memory bandwidth can limit it.

Slow on branchy, one-step-at-a-time code.

Kitchen Analogy

A GPU is a **huge kitchen of junior cooks** who all follow the same shout: "chop these onions!" A thousand onions get chopped in the time one chef chops a few.

1.2 The TPU: Dumpling Conveyor

Explain

A **TPU** (Tensor Processing Unit) is Google's chip designed **only** for tensor maths. It is an **ASIC** (application-specific integrated circuit). Almost all of its area is a big grid of MAC units called a **systolic array**, plus the memory that feeds it. It has very little control logic, so every run takes the same predictable time (**deterministic execution**).

How it works, step by step

How a systolic array multiplies W = [[1, 2], [3, 4]] by x = [5, 6]:

1. Load each weight into its own cell, **once**. It stays there ("weight-stationary").
2. **Cycle 1:** $x_1 = 5$ enters column 1 → cells compute 1×5 = 5 and 3×5 = 15, then pass these partial sums to the right.
3. **Cycle 2:** $x_2 = 6$ reaches column 2 → 2×6 + 5 = **17** and 4×6 + 15 = **39**.
4. Result y = [17, 39] with 4 MACs. Inputs were read from memory once and handed cell-to-cell: **6** memory reads instead of **8**. The saving grows with array size.

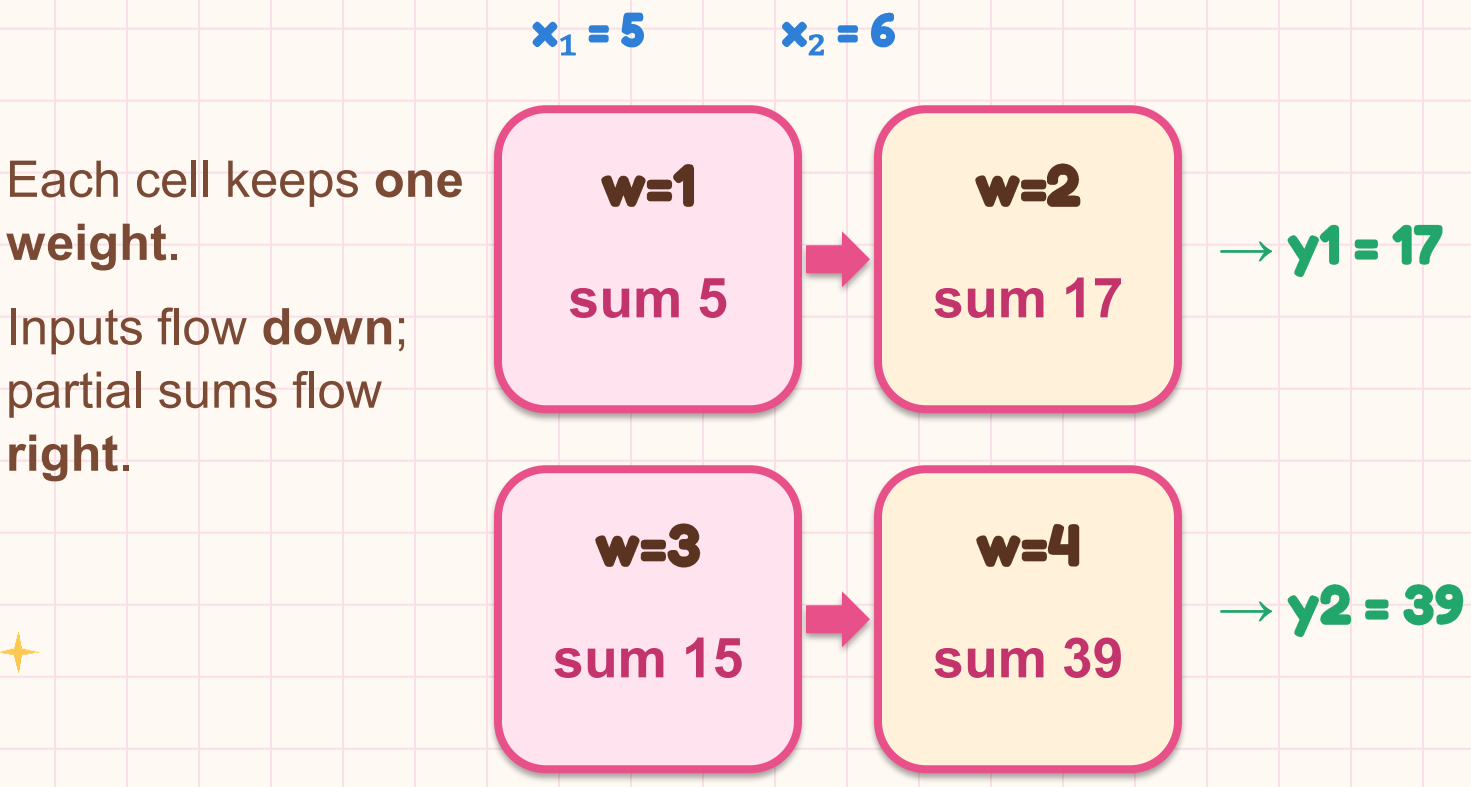


Figure 1.4 · A 2 × 2 weight-stationary systolic array.

How to read this: start at the top of each column, multiply by the cell's weight, add the sum arriving from the left, and pass it right. The green numbers are the outputs.

1.2 The NPU & The Software

Explain

A phone cannot carry a 300-watt accelerator. An **NPU** (Neural Processing Unit) applies the TPU ideas (a MAC grid, low-precision numbers, on-chip memory) but scaled down to run in a few watts inside a **system-on-a-chip** (SoC): one piece of silicon that also holds the CPU, GPU and a **shared memory**. Names differ by company: Apple **Neural Engine**, Google **Tensor**, Qualcomm **Hexagon**, laptop "AI Engine".

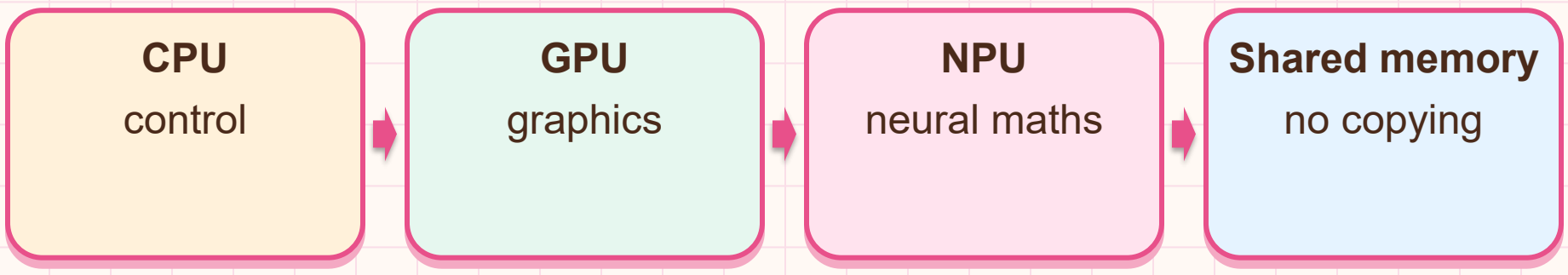


Figure 1.5 · Inside a phone's system-on-a-chip.

How to read this: all four blocks sit on one chip; because they share memory, a photo never has to be copied between them.

How it works, step by step

Why a chip is only as useful as its software. Programmers almost never write instructions for MAC units directly. Instead:

1. They build a model in a **framework** (PyTorch, TensorFlow, JAX).
2. A **compiler** or runtime (CUDA, XLA, Core ML) translates it for the chip.
3. If the chip does not support an operation, that step **falls back** to the CPU: data is copied out and back, and the model can silently become slow.

This is a big reason GPUs dominate AI: their software tools are mature and everywhere.

Real Tech

Hardware + its software ecosystem = a **platform**. Switching platforms can be costly even when the new chip is faster on paper.

1.2 The Crew, Side by Side

Feature	CPU	GPU	TPU / NPU
Design goal	Fast & flexible for one task	Throughput on parallel data	Max matrix maths per watt
Cores	Few to ~100 complex	Thousands of simple	Grid of tens of thousands of MACs (TPU)
Control logic	Complex: caches, prediction	Simpler; threads hide delays	Minimal; deterministic
Flexibility	Highest	High	Lowest
AI efficiency	Lowest	Medium–high	Highest (supported work)
Typical AI role	Data loading, small models	Training & serving big models	Cloud TPU; on-device NPU
Programmed via	Any language	CUDA, PyTorch	TensorFlow/JAX + XLA; Core ML

Table 1.2 · CPU vs GPU vs TPU/NPU.

👀 **How to read this:** read down a column to get one chip's personality, or across a row to compare one feature. Notice how flexibility falls exactly as efficiency rises.

Kitchen Analogy

CPU = head chef · **GPU** = 1,000 line cooks · **TPU** = dumpling conveyor · **NPU** = home rice cooker (small, low-power, one job).

Where the analogy breaks down

Chips don't think or get tired. A GPU core can't choose its own task, and a TPU is **programmable** for many networks, not fixed to one "dish".

⚠ Chef's Warning

"TPUs beat GPUs, end of story!" **Not quite!** The first TPU (2015–2017) was ~15–30× faster than CPUs and GPUs **of its day**; modern GPUs added Tensor Cores and the gap narrowed. Ask: **which workload, which year?**

Try It: Hire the Right Cook

Try It

🎯 **Goal:** match a chip to a workload and justify it with one design feature.

🧺 **Preparation:** read 1.2 and Table 1.2. Cut 4 cards labelled CPU, GPU, TPU, NPU (or write them on paper). Work alone or in pairs.

🔪 **Task:** for each job, place one card and write the **one feature** that decided it:

a) a web server running many different small tasks

b) training a large image model over two weeks

c) a fixed, high-volume inference service in a data centre

d) a voice assistant in a battery-powered smart speaker (rank NPU, CPU, GPU)

👀 **Observation:** record a table: job | chip | deciding feature | one runner-up and why it lost.

☁️ **Reflection:** which feature came up most often? How does it connect to the flexibility–efficiency spectrum (Figure 1.3)?

Mama's Feedback

Hint: ask "is the work the same operation on lots of data?" and "is there a battery?"

a) **CPU**: mixed, branchy tasks need flexibility and low latency.

b) **GPU** (or TPU in Google's cloud): massive parallel MACs, mature tools.

c) **TPU** / fixed accelerator: same network all day → efficiency wins.

d) **NPU > CPU > GPU**: performance per watt matters most on a battery.

Expected outcome: the deciding features are usually **parallelism** and **energy**, the two directions this recipe keeps returning to.

✅ Check & Recap: 1.1–1.2

✅ Check

1. A layer has 512 inputs and 2,048 outputs. How many MACs per input? How many FLOPs?
2. Why does a CPU spend so much area on caches and branch prediction?
3. In the systolic example, why are there 6 memory reads instead of 8?
4. Give two reasons a start-up might choose a GPU even though Google's TPU was faster in its tests.

⭐ Explained answers

1. 512 × 2,048 = **1,048,576 MACs** = 2,097,152 FLOPs.
2. To make **one** thread fast: keep data close and never stall waiting for an 'if'.
3. Each weight and input is read **once** and then passed cell-to-cell.
4. Different workload (fall-backs), software already targets GPUs, availability and cost.

📌 Recap

- AI is mostly **MACs**: speed those up and you speed up the model.
- Chips trade **flexibility for efficiency**: CPU → GPU → TPU/NPU.
- CPUs spend area on control; GPUs on many simple cores; TPUs on a MAC grid that **reuses data**.
- A chip is only as useful as its **software** tools.

1.3 Cooking in Parallel

Predict

One chef chops 4,194,304 onions one by one. A kitchen of **4,096** helpers chops 4,096 at a time. Guess: how many times faster is the big kitchen? 10×? 100×? 1,000×?

Why does this matter?

Section 1.1 showed that AI is billions of MACs, and 1.2 showed chips with thousands of units. **Parallelism** (doing many things **at the same time**) is how those units turn into speed. But parallel hardware has limits, and the limits explain why real chips rarely reach their quoted peak.

Explain

Parallelism happens at four levels, from inside one core to across many chips. Each level adds more units, but also more **coordination** (keeping everyone in step).

Level	Idea	Example
Instruction-level	One core overlaps several instructions, like an assembly line	CPU pipelines
Data-level (SIMD/SIMT)	One instruction acts on many values at once	GPU warps; CPU vector units
Matrix-level	A grid of MACs passes data neighbour to neighbour	TPU systolic array
Across chips	Split the model or the data over many chips	Training clusters (Recipe 05)

Table 1.3 · Four levels of parallelism.

How to read this: go top to bottom: each row is a bigger "unit" doing work at the same time, from one core to a whole data centre.

1.3 How Much Faster? Step by Step

How it works, step by step

Problem: a layer has 1,024 inputs and 4,096 outputs. How long does one pass take?

Step 1 · Count the work. 1,024 × 4,096 = **4,194,304 MACs** (= 8,388,608 FLOPs).

Step 2 · Idealised CPU core: 1 MAC per cycle at 3 GHz = 3×10^9 MACs/s. Time = 4,194,304 ÷ 3×10^9 ≈ **1.4 ms**.

Step 3 · Idealised accelerator: 4,096 MAC units at 1 GHz = 4.096×10^{12} MACs/s. Time = 4,194,304 ÷ 4.096×10^{12} ≈ **1.0 µs**.

Step 4 · Interpret. The accelerator is ≈ **1,365× faster**, even with a **slower** clock, because it does 4,096 MACs per cycle instead of 1.

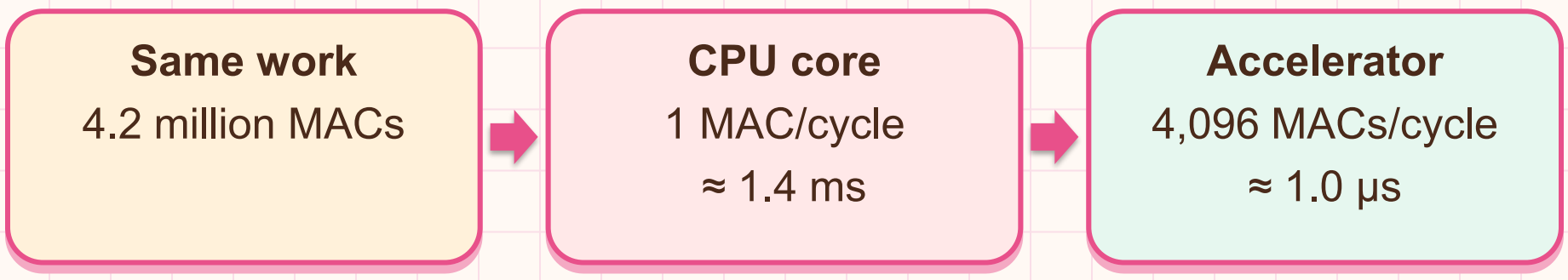


Figure 1.6 · Same recipe, two kitchens.

How to read this: both boxes do exactly the same 4.2 million MACs; only the number of MACs per cycle changes.

Chef's Warning

"So the accelerator is always 1,365× faster!" **Not quite!** These are **ideal** numbers: they assume every unit is busy every cycle and the data is always ready. The next page shows what gets in the way.

1.3 Why More Cooks Stop Helping

Explain

Amdahl's law says: if only a fraction **p** of a job can be shared out, the best speed-up on **N** units is

speed-up = 1 ÷ ((1 − p) + p ÷ N)

Why? The shared part shrinks as you add cooks (p ÷ N), but the **serial** part (1 − p) never shrinks: someone still has to stir the single wok. With p = 0.9, even unlimited cooks give at most 1 ÷ 0.1 = **10×**.

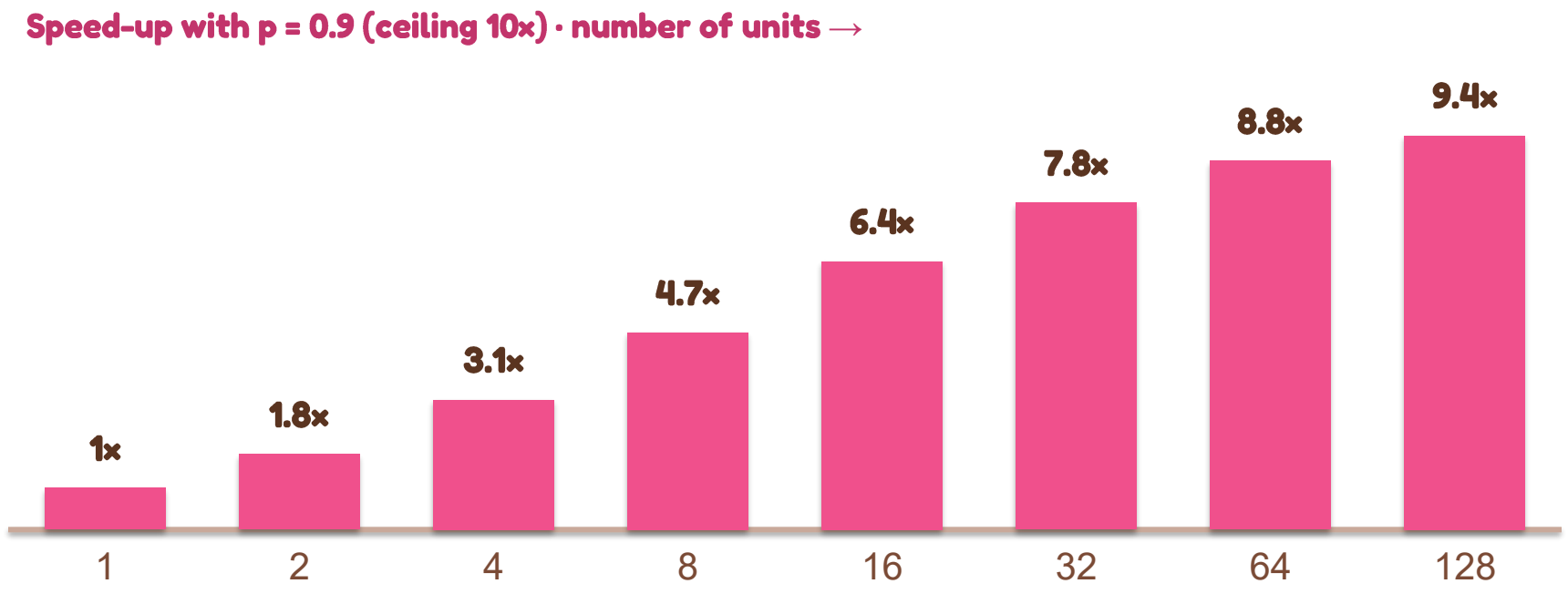


Figure 1.7 · Amdahl's law for p = 0.9.

👀 **How to read this:** each bar is the speed-up with that many units. The bars grow fast at first, then crawl towards 10×: doubling from 64 to 128 units adds only 0.6×.

Explain

Three more limits: **(2) idle units:** a small layer cannot keep 4,096 units busy, so **utilisation** drops. **(3) Feeding the units:** MACs wait if memory can't deliver data fast enough (1.4). **(4) Communication:** chips sharing a job must swap results (Recipe 02).

1.3 Secret Kitchen Boosters

Explain

Designers combine parallel units with three other tricks:

- **Quantisation** (lower precision): store numbers as INT8 or FP16 instead of FP32. Smaller numbers mean more fit in memory and more MACs fit on the chip.
- **Specialised units** (Tensor Cores, Neural Engine): do a small matrix multiply as **one** hardware step, with less control overhead per MAC.
- **Batching**: process several inputs together so the same weights are **reused** (1.4).

Quantisation, step by step

```
w      = [0.5, -0.31, 0.02, 0.97]     scale = 1/127
q      = round(w / scale) = [64, -39, 3, 123]      (INT8)
back   = q x scale = [0.5039, -0.3071, 0.0236, 0.9685]
max error = 0.0039   ->   4x less memory than FP32
```

Format	Bytes	Typical use
FP32	4	Classic training ("full" precision)
BF16 / FP16	2	Training and inference on modern accelerators
INT8 / FP8	1	Inference on phones, edge devices, TPUs
INT4	0.5	Aggressively compressed models

Table 1.4 · Number formats.

How to read this: fewer bytes per number = less memory and energy, at a small cost in accuracy.

Kitchen Analogy

Lower precision = cutting vegetables into fewer, bigger chunks: faster to carry, a little less detail.

Where the analogy breaks down

Chunks can't be "un-cut", but a quantised model can be compared with the original and the error **measured**.

Try It: The Chopping Race

Try It

🎯 **Goal:** feel Amdahl's law with your own hands and predict where the speed-up stops growing.

🧺 **Preparation:** Amdahl's law (previous pages). A stopwatch (phone), 40 small addition problems on paper (e.g. 37 + 48), 9 people. No internet needed.

🔪 **Task:** Run three rounds and time each one:

a) **Solo:** one student adds all 40 pairs alone.

b) **Team of 8:** each of 8 students adds 5 pairs, then one "wok master" adds the 8 results.

c) **Predict first:** using p ≈ 0.9, what speed-up should 8 students give? Write it down before you start round (b).

👀 **Observation:** record solo time, team time, measured speed-up (solo ÷ team) and your predicted speed-up.

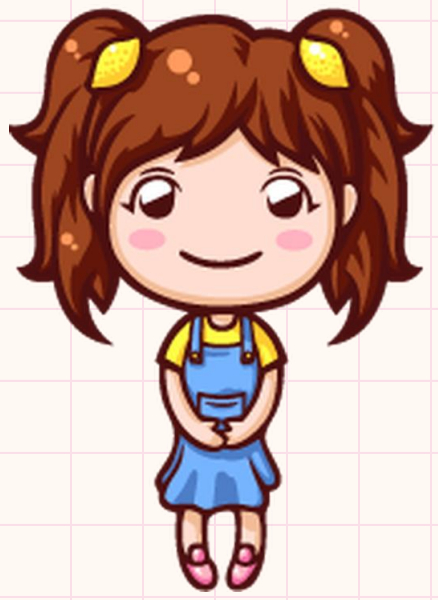
☁️ **Reflection:** which step was "the single wok"? What would happen with 40 students, one pair each?

Mama's Feedback

Hint: formula with N = 8: 1 ÷ (0.1 + 0.9 ÷ 8) = 1 ÷ 0.2125 ≈ **4.7×**.

Expected outcome: clearly less than 8×, often 3–5×. The wok master's final sum (and handing out the papers) is the serial part.

Explained answer: with 40 students the shared part almost disappears, but the final sum of 40 numbers gets **longer**, so the team may even get slower. That is limits (1) and (4) together: serial work and communication.



1.4 The Pantry Problem

Predict

Which costs more energy: **multiplying** two numbers, or **fetching** one number from the computer's main memory? Guess how many times more.

Why does this matter?

In 1.3, limit (3) was "feeding the units". For AI this is often **the** limit: a chip with thousands of MACs is useless if they sit waiting for data. Understanding memory explains batch sizes, phone AI chips and why designers obsess over data movement.

Explain

Fast memory is small and expensive; big memory is slow and cheap. So computers stack several kinds in a **memory hierarchy**:

- **Registers**: a few values inside the core, read in one cycle.
- **Cache** / **SRAM**: small, fast memory on the chip.
- **DRAM** (and **HBM**, DRAM stacked right beside the chip): large main memory, off the core.
- **Storage** (SSD): huge, but far slower.

Each step down is roughly bigger, cheaper per byte, and slower.

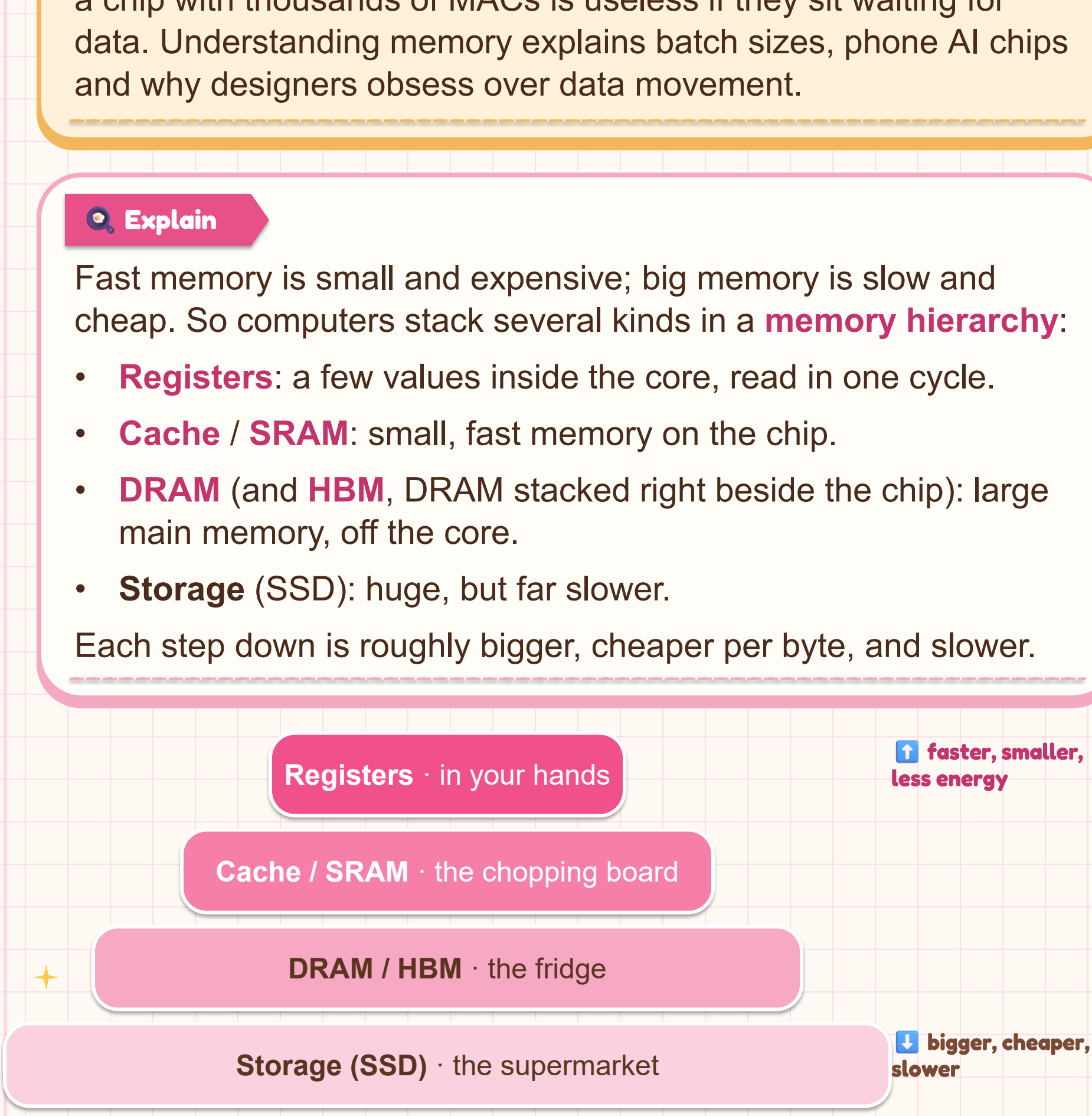


Figure 1.8 · The memory pyramid, with its kitchen names.

👀 **How to read this:** the higher a level, the closer it is to the MAC units. Good AI chips keep weights as high up as possible.

1.4 Moving Food Costs More Than Cooking

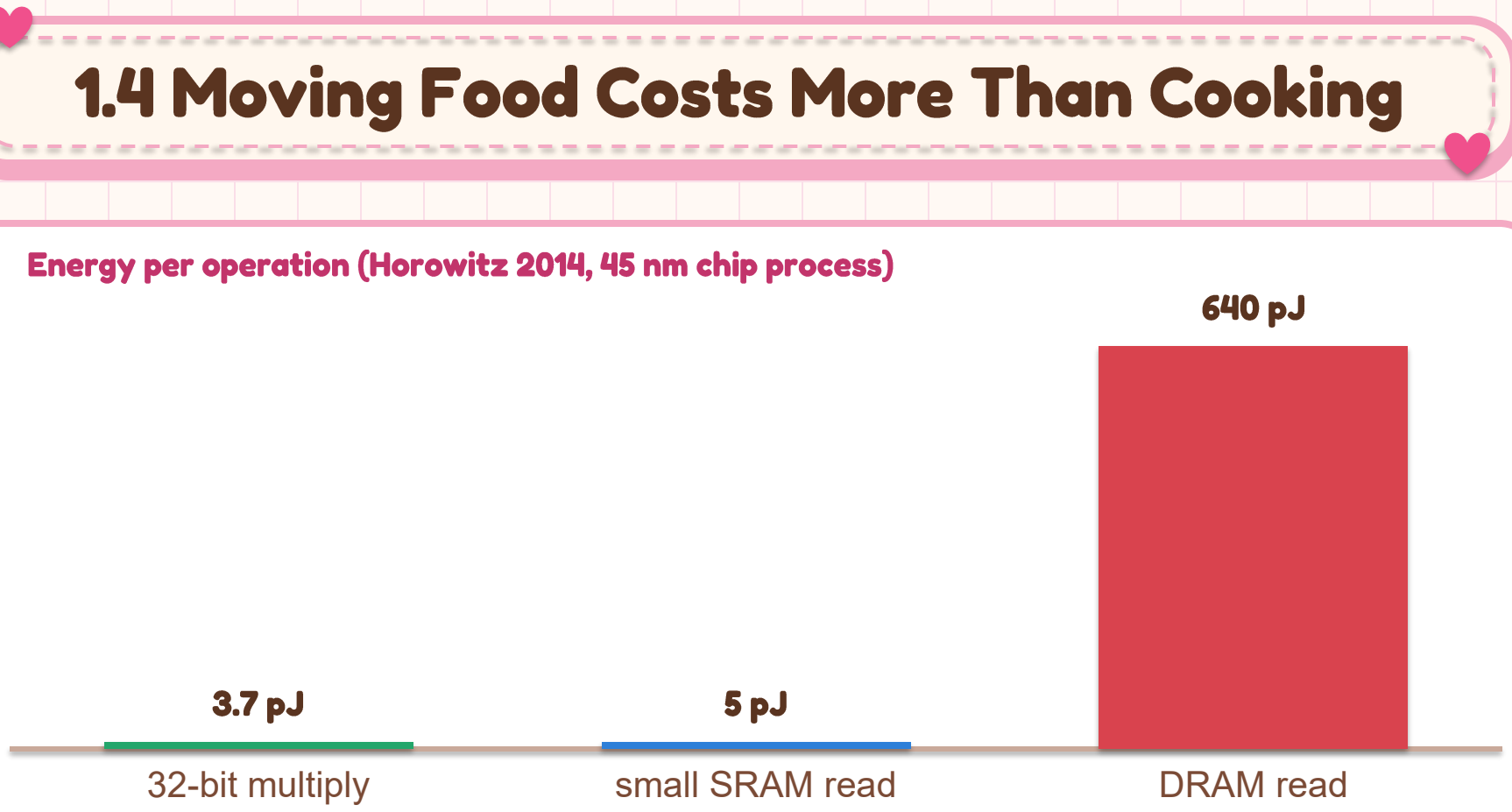


Figure 1.9 · Energy per operation.

👀 **How to read this:** the two left bars are so short they almost disappear. A DRAM read costs ≈ 640 ÷ 3.7 ≈ **173×** a multiply and 128× an SRAM read. (pJ = picojoule = 10^{-12} J.)

Explain

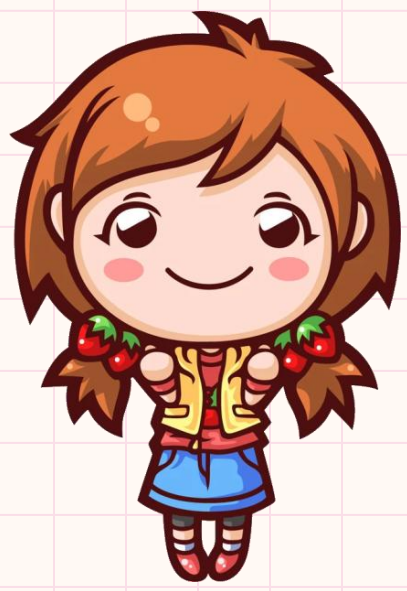
Cause and effect: because fetching dominates the energy bill, the best way to save energy is not faster maths but **reusing data already on the chip**. That is exactly why the systolic array (1.2) passes inputs cell to cell, why the TPU has a big on-chip buffer, and why Apple's chips use **unified memory** (CPU, GPU and NPU share one pool, so nothing is copied).

Kitchen Analogy

A cook who walks to the supermarket for every ingredient never finishes. A good cook puts everything on the chopping board first.

Where the analogy breaks down

Exact energies depend on the chip process; newer chips differ. The **pattern** (moving ≫ computing) stays true.



1.4 The Roofline: Waiting or Working?

How it works, step by step

How do we tell if a job is limited by **maths** or by **memory**?

Step 1 · Arithmetic intensity = FLOPs ÷ bytes moved (how much cooking per bag of groceries).

Step 2 · A chip has two limits: **peak compute** (FLOP/s) and **memory bandwidth** (bytes/s).

Step 3 · Attainable performance = min(peak, intensity × bandwidth).

Step 4 · Ridge point = peak ÷ bandwidth. Below it the job is **memory-bound** (the cook waits for deliveries); above it, **compute-bound** (the cook is busy).

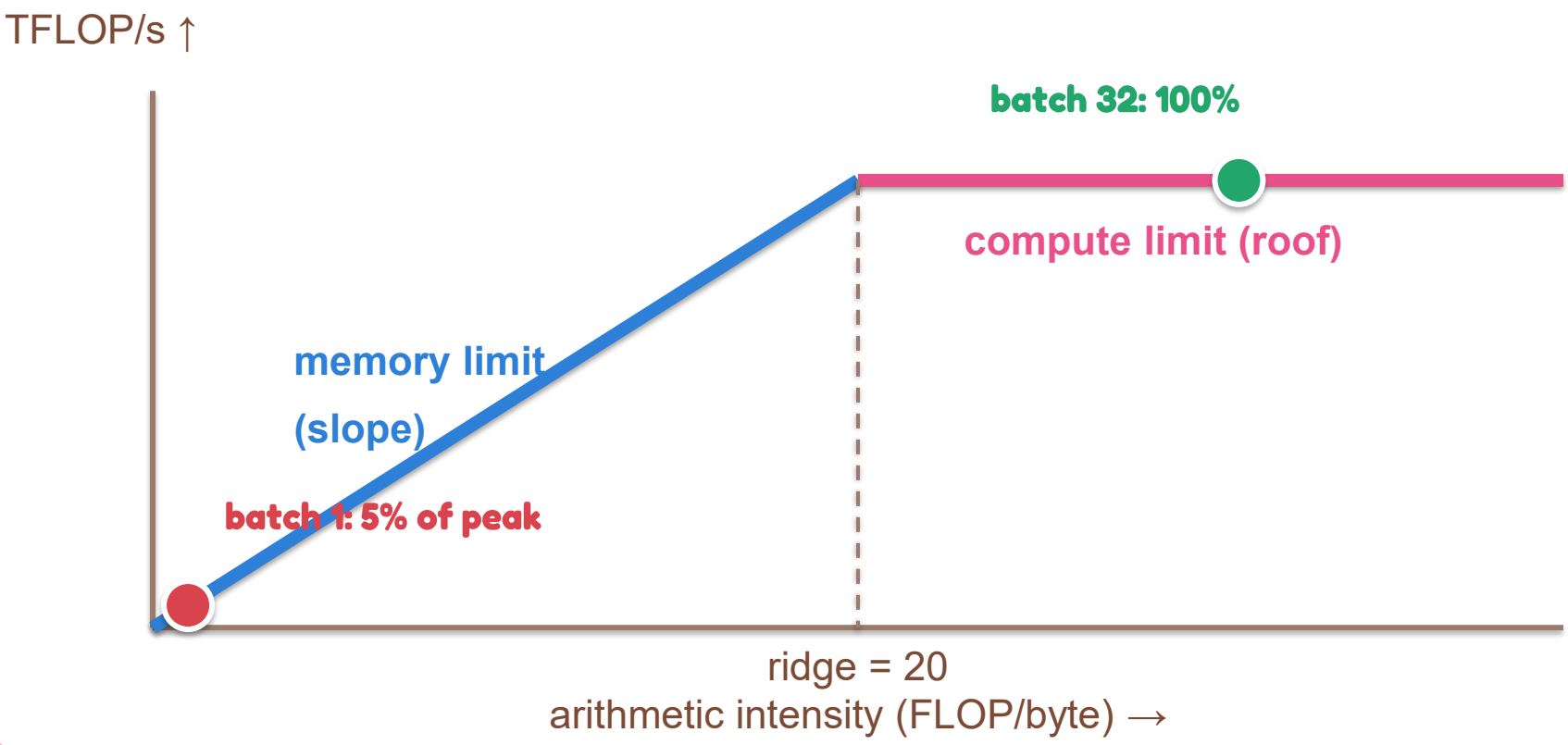


Figure 1.10 · Roofline for a 10 TFLOP/s, 500 GB/s chip.

How to read this: find your workload's intensity on the x-axis, go up until you hit the roof: that height is the best speed you can get. Left of the ridge, only more bandwidth or more reuse helps.

1.4 Batch 1 vs Batch 32

🔍 Worked example

```
Layer 1,024 x 4,096, FP16 (2 bytes)
Chip: 10 TFLOP/s, 500 GB/s, ridge 20
BATCH 1:  FLOPs 8,388,608 / bytes 8,398,848
  intensity ~ 1.0  -> 0.5 TFLOP/s = 5% of peak
BATCH 32: weights fetched ONCE, used 32x
  intensity ~ 30.8 -> 10 TFLOP/s = 100%
```

🔍 Explain

At batch 1 the chip spends about **95%** of its time waiting: moving 8.4 MB at 500 GB/s takes ≈ 16.8 µs, while the maths alone needs only ≈ 0.84 µs. At batch 32 the **same** weights serve 32 inputs, so each byte fetched does 32× more work. That is the whole story of batching.

⚠️ Chef's Warning

"Batch everything, always!" **Not quite!** Batching raises throughput, but each request may have to **wait** for the batch to fill, so **latency** rises. A phone answers one user at a time, so it stays memory-bound and needs high bandwidth and small weights instead.

Fix for a memory-bound job	How it helps
Bigger on-chip memory	Weights stay close to the MACs
Higher-bandwidth memory (HBM)	Raises the slope of the roof
Batching / weight reuse	More FLOPs per byte (moves right)
Lower precision	Fewer bytes per weight
Unified memory	No copies between CPU, GPU, NPU

Table 1.5 · Levers on the roofline.

👀 **How to read this:** the first two raise the slope; the next two push the workload to the right.

Try It: Batch Cooking Challenge

Try It

🎯 **Goal:** decide memory-bound vs compute-bound with the roofline, and find the batch where it flips.

🧺 **Preparation:** pages 1.4, a calculator. Chip: 10 TFLOP/s, 500 GB/s, ridge 20. Layer: 512 inputs × 2,048 outputs, FP16. Weights are fetched once per batch; inputs and outputs once per sample.

🔪 **Task:** work through these in order:

1. Number of weights and weight bytes.
2. FLOPs for one input (batch 1).
3. Intensity at batch 1, 8 and 32: memory- or compute-bound? Attainable TFLOP/s and % of peak?
4. **Predict, then check:** the smallest batch that is compute-bound.

👀 **Observation:** a table: batch | bytes | intensity | bound | TFLOP/s | % peak.

☁️ **Reflection:** a voice assistant answers one user at a time. What hardware features would you choose, and why?

Mama's Feedback

1. 512 × 2,048 = **1,048,576** weights = 2,097,152 bytes. 2. **2,097,152** FLOPs.
3. Batch 1 ≈ 1.0 → memory-bound, 0.5 TF, **5%** · batch 8 ≈ 7.9 → memory, 3.9 TF, **39%** · batch 32 ≈ 29.7 → compute, 10 TF, **100%**.
4. **22** (intensity first passes 20).

Reflection: batch 1 is memory-bound, so choose high bandwidth, large on-chip memory and low-precision weights: exactly what an NPU offers.

1.5 Case Study: Apple Neural Engine

Why does this matter?

Case file: the rice cooker in your pocket! Face ID, photo search and live captions run **on the phone**, often without internet. Doing AI on a battery in a fraction of a second tests everything in this recipe: parallel MACs, low precision and, above all, memory.

Explain

The **Apple Neural Engine** (ANE) is the NPU inside Apple's chips. It first shipped in 2017 in the A11 Bionic (iPhone X), described as 600 billion operations per second; later generations are quoted in TOPS.

It sits in a corner of the system-on-a-chip next to the CPU and GPU. Its cores do one job: the multiply-and-add maths of neural networks. All three share the same **unified memory**, so a photo or face scan is never copied between them.

Idea from this recipe	How the ANE uses it
Software (1.2)	**Core ML** decides whether each part runs on the CPU, GPU or Neural Engine; unsupported operations fall back to the CPU
Precision (1.3)	Models are usually quantised to INT8 or FP16
Memory (1.4)	Unified memory: no copying between processors
Batch size (1.4)	Face ID handles ONE input at a time: memory-bound, so small weights and on-chip memory matter most

Table 1.6 · The ANE through the lens of Recipe 01.

How to read this: each row links a concept you learned to a real design decision.

1.5 How Face ID Works

1 Scan	2 Process	3 Infer	4 Compare	5 Unlock
infrared + depth camera	face → numbers	on the Neural Engine	with your saved face	if it matches

Figure 1.11 · Face ID, from scan to unlock.

👀 **How to read this:** only step 3 is the neural network; it is where the ANE does its MACs. Face data is processed and protected on the device (Secure Enclave), not in the cloud.

🔍 Explain

Training vs inference again: the Face ID model was trained in a data centre on GPUs or accelerators. The finished weights were shipped to the phone, where only **inference** runs, on the Neural Engine: a fast, private answer with no network round trip.

Face ID is a small **sense–decide–act loop**: one request at a time, a fraction of a second, on a battery. Robots face the same constraints (Recipe 03).

👍 Advantages

Fast, real-time answers

Energy efficient (battery)

Privacy: data stays on the device

Works offline

👎 Limitations

Limited compute vs servers

Not every operation is supported

Large models may not fit in memory

Some AI still needs the cloud

🥢 Kitchen Analogy

The ANE is a **home rice cooker**: small, quiet, uses little power and makes one thing brilliantly. You wouldn't cook a banquet for 1,000 in it (that's the data centre's job).

✅ Check: Case Study Questions

✅ Check

1. Face ID processes one face at a time. On the roofline it is most likely… **A** compute-bound **B** memory-bound **C** network-bound
2. A 3-billion-parameter model quantised to 4-bit needs about… **A** 12 GB **B** 6 GB **C** 1.5 GB
3. One unsupported operation sits in the middle of a model. Why can it run slowly on the NPU?
4. Give one benefit and one drawback of on-device AI vs cloud AI.
5. Why does unified memory save **energy** as well as time?

⭐ Explained answers

1. **B.** Batch 1 → low intensity: the weights are fetched for just one input.
2. **C.** $3 \times 10^9 \times 0.5$ bytes = 1.5 GB: fits on an 8 GB phone, but is a big share.
3. Data must move to the CPU and back; the **data movement**, not the operation, costs time.
4. Benefit: privacy, speed, offline. Drawback: limited model size and compute.
5. No copies = fewer DRAM transfers, and a DRAM read costs ~640 pJ each.

All right? **Gold stamp!** Missed one? Re-read 1.4–1.5 for Silver.

🌐 Kitchen Lab: CPU vs GPU Race

🌐 Kitchen Lab

🎯 **Goal:** see the CPU-vs-GPU gap (1.2) on the same matrix maths, and see it shrink for small jobs (1.3).

🧺 **Preparation:** a Google account and internet; free Colab (no install). Platform used: Colab free tier, CPU and T4 GPU runtimes, PyTorch 2.x (check with print(torch.__version__)).

🔪 **Task:** open colab.research.google.com → New notebook. Paste the code below and run it on the CPU. Then Runtime → Change runtime type → **T4 GPU** and run again. Run each cell **twice** and use the 2nd time (the 1st GPU run includes warm-up).

Python (Colab)

```
import torch, time
dev = "cuda" if torch.cuda.is_available() else "cpu"
a = torch.randn(4096, 4096, device=dev)
t = time.time(); [a @ a for _ in range(10)]
if dev == "cuda": torch.cuda.synchronize()
print(dev, time.time() - t)
```

🔍 Observe

MACs = 10 × 4,096³ ≈ 687 billion. Compute TFLOP/s = 2 × MACs ÷ time for each device. How many times faster is the GPU? Then change 4096 to **256** and repeat.

💭 **Reflection:** why does the GPU's advantage shrink at size 256? (Hint: idle units, launch overhead, memory.)

No internet? Fallback

Typical results (yours will differ): CPU ≈ 15–30 s (≈ 0.05–0.1 TFLOP/s); T4 GPU well under 1 s (≈ 4–7 TFLOP/s): roughly **50–100× faster**. At size 256 both finish in milliseconds and the GPU is only a few times faster: thousands of its cores sit idle and launch overhead dominates. No account? Analyse these numbers instead, or run the Chopping Race (1.3).

📺 Watch: Curated Videos

📺 Video 1 · for 1.1

But what is a neural network? · 3Blue1Brown · 18:40 · youtube.com/watch?v=aircAruvnKk

Why: animated proof that a network is layers of weighted sums: the MACs of 1.1.

Which part: 2:42 neurons · 3:35 layers · 11:34 counting weights and biases.

Do: at 11:34, count the weights of the 784-16-16-10 network yourself and check his total.

Watch out: about recognising digits, not hardware; no chips discussed.

📺 Video 2 · for 1.2

How do Graphics Cards Work? · Branch Education · 28:29 · youtube.com/watch?v=h9Z4oGN89MU

Why: a 3-D tour inside a real GPU: thousands of cores, memory chips and SIMD.

Which part: 2:15 GPU vs CPU · 4:56 GA102 architecture · 8:48 CUDA core · 12:04 GDDR6X/GDDR7.

Do: write down how many cores the GPU has and link it to "1,000 Line Cooks" (1.2).

Watch out: focuses on graphics/gaming; little on AI training or TPUs; has a sponsor segment.

📺 Video 3 · for 1.4

Roofline model · NHR@FAU · 18:36 · youtube.com/watch?v=IrkNZG8MJ64

Why: a university HPC centre explains intensity, bandwidth and the roof, exactly as in 1.4.

Do: redraw Figure 1.10 and mark batch 1 and batch 32. **Watch out:** aimed at HPC programmers; extra jargon.

📴 Can't watch?

All three have English auto-captions (CC). If YouTube is unreachable, re-read the matching Figure and do its 👀 How to read this task instead.

Final Round: Recipe 01

Check

Task 1. A layer has 2,048 inputs and 1,024 outputs. How many MACs and FLOPs per input? How long on a 4,096-unit accelerator at 2 GHz (ideal)?

Task 2. A pipeline is 95% parallel. Speed-up on 20 units? Limit with unlimited units?

Task 3. A chip has 20 TFLOP/s peak and 1 TB/s bandwidth. Ridge point? A workload has intensity 4 FLOP/byte: attainable TFLOP/s and % of peak?

Explained answers

Q1: 2,048 × 1,024 = 2,097,152 MACs = 4,194,304 FLOPs. 4,096 × 2 × 10^9 = 8.192 × 10^{12} MAC/s → 2,097,152 ÷ 8.192 × 10^{12} ≈ **0.26 µs**.

Q2: 1 ÷ (0.05 + 0.95 ÷ 20) = 1 ÷ 0.0975 ≈ **10.3×**; limit 1 ÷ 0.05 = **20×**.

Q3: ridge = 20 × 10^{12} ÷ 10^{12} = **20 FLOP/byte**. Attainable = min(20, 4 × 1) = **4 TFLOP/s = 20%** of peak: memory-bound.

Stamp: Gold = all right first time · Silver = right after a hint · Bronze = read and redo.

📌 Recipe Card 01

📌 Recap

- AI is mostly **MACs**: speed those up and you speed up the model.
- Specialised chips trade **flexibility for efficiency** (CPU → GPU → TPU/NPU).
- Parallelism helps until the **serial part**, idle units, memory or communication get in the way.
- **Moving data costs more than computing on it**: keep ingredients on the chopping board.
- On-device AI (NPU) wins on privacy, latency and battery, but is limited by memory and supported operations.

📖 Key words (see Glossary)

MAC · FLOP · TOPS · training / inference · precision · CPU / GPU / TPU / NPU · systolic array · Amdahl's law · quantisation · memory hierarchy · HBM · arithmetic intensity · roofline · batching · Core ML

🎉 Unlocked: Recipe Book page 1!

🍳 Recipe 01 · complete!

RECIPE 02

Order Up! Delivery

Computer Networks & AI Infrastructure

Mama's Problem

A customer across town points a phone at a **street sign** and wants it translated in about **one second**. The photo must travel to a kitchen, be cooked by an AI model, and come back. Which kitchen should cook it, and how do we deliver fast?

Today's route

2.1 How kitchens are connected: architecture, layers, packets

2.2 The careful waiter and the shouting waiter: TCP/IP, UDP, QUIC

2.3 Central kitchen, food truck or home kitchen? Cloud vs edge

2.4 Delivery time, road width, rush hour: latency, bandwidth, scalability

2.5 Case study: 5G and AI-powered network optimisation

Before you start

From Recipe 01: MACs, bandwidth (bytes per second), memory hierarchy, batch size. No networking background needed.

Cooking time

~3 hours. Running example: translating a street sign with your phone camera.

2.1 What Is a Network, Really?

Predict

Describe a computer network in **one** sentence. Then: when you send a photo, does it travel as one big piece or many small ones? Why might that matter?

Why does this matter?

Almost no AI runs in one place. Your photo travels to a model, model weights travel to phones, and big models are split across many chips that constantly swap results. The network decides how fast, how reliably and how privately all of that happens.

Explain

A **computer network** is a group of devices (**nodes**) joined by **links** so they can exchange data.

Data is not sent in one piece. It is cut into small pieces called **packets**. Each packet carries a **header**: a delivery label with the information needed to reach the right place (like the address on a parcel). Small packets let many users share the same links fairly, and if one packet is lost, only that piece needs resending.

A **protocol** is an agreed set of rules for how packets are formatted, sent, received and acknowledged. Two machines can talk only if they follow the same protocols.

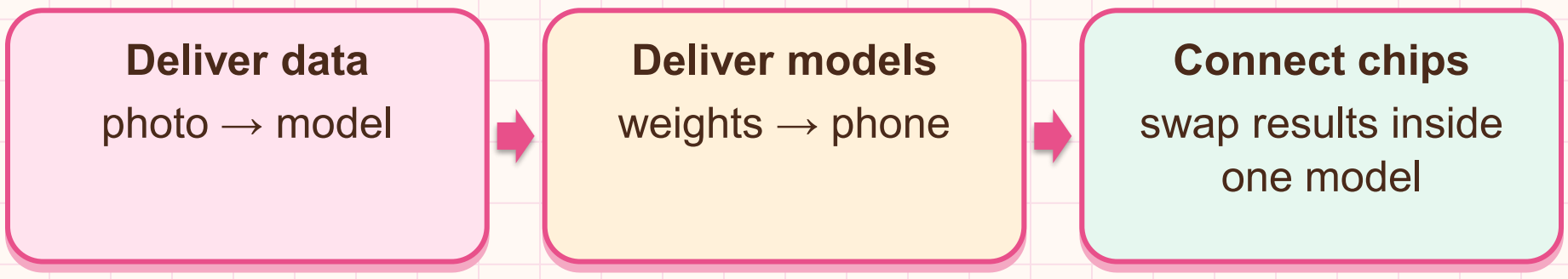


Figure 2.1 · Three jobs a network does for AI.

How to read this: left to right: data to the model, the model to the user, and chips talking to each other while they share one model.

2.1 Why Data Centres Look Like Forests

Explain

The same idea (send data over a link) works from inside a chip to across the planet; only the **speed** and **delay** change, by many orders of magnitude. That is the memory hierarchy of Recipe 01 extended outwards.

Inside a data centre, thousands of servers must talk. One giant switch would be a **bottleneck** (everything squeezes through it) and a **single point of failure** (if it breaks, everything stops). So data centres use a **leaf-spine** design:

How it works, step by step

1. Each rack of servers connects to a **leaf** switch.
2. Every leaf connects to **every** spine switch.
3. Traffic between any two servers has **several equal paths** (leaf → any spine → leaf).
4. Need more capacity? Add switches. A spine fails? The others carry the load.

Chef's Warning

"8 Gbit/s and 8 GB/s are the same!" **Not quite!** Networks are quoted in **bits** per second, memory in **bytes** per second. Divide by 8: 100 Gbit/s = 12.5 GB/s.

Kitchen Analogy

Leaf-spine is a kitchen with **many doors** instead of one jammed serving hatch.

Where the analogy breaks down

Cooks pick a door by looking; switches pick a path by **hashing** packet headers, without "seeing" the crowd.

2.1 Layers: A Box in a Box in a Box

Explain

Sending data is split into **layers**. Each layer does one job and relies on the layer below. This lets a Wi-Fi phone talk to a fibre-connected server: each layer only needs to agree with its partner layer on the other side.

Layer (TCP/IP model)	Job	Examples
Application	What programs say to each other	HTTP, gRPC, MQTT, DNS
Transport	Delivery between programs; reliability	TCP, UDP, QUIC
Network	Addressing and routing across networks	IP
Link & physical	Moving bits over one hop	Ethernet, Wi-Fi, 5G radio

Table 2.1 · The four TCP/IP layers.

👀 **How to read this:** data goes DOWN this table when sending and UP when receiving. (The 7-layer OSI model splits these further; you will see both in textbooks.)

How it works, step by step

Encapsulation, **step by step:**

1. The app creates the data (the **payload**).
2. Transport adds a TCP header (20 bytes).
3. Network adds an IP header (20 bytes).
4. Link adds an Ethernet header (14 B) and a check (4 B), then sends bits.
5. At the destination, each layer removes **its own** header and passes the rest up.

// **Analogy:** food in a box, the box in a bag, the bag in a crate; each handler opens only **their** wrapper. 🧂 **Breaks down:** real headers also carry checks and numbers, not just addresses.

2.1 How Much Is Packaging?

Worked example

```
Overhead per packet = 20 (TCP) + 20 (IP) + 14 + 4 (Ethernet)
= 58 bytes
1,000-byte payload: 1,000 / 1,058 = 94.5% useful
  100-byte payload:   100 /   158 = 63.3% useful
```

Explain

Cause and effect: headers are a fixed cost per packet. Big payloads hide that cost; tiny messages (like one sensor reading) waste over a third of the link on labels. That is why small messages are often **batched** together, the same reuse idea as batching in Recipe 01.

Explain

Different jobs use different application protocols. A cloud AI request typically goes: **phone → HTTPS/JSON → API gateway → gRPC → model server (GPU/TPU)** and back. **HTTP/REST** is the web's request-response language; **gRPC** is a fast binary protocol between services; **MQTT** is a light publish-subscribe protocol for small battery devices; **WebRTC** carries live audio and video.

Chef's Warning

"Packets always have 58 bytes of overhead!" **Not quite!** IPv6 headers are 40 bytes, TCP can carry options, Wi-Fi adds its own framing. 58 is a teaching figure, not a rule.

Try It: Bento Packing Challenge

Try It

🎯 **Goal:** calculate packet efficiency and match AI jobs to the right layer and protocol.

🧺 **Preparation:** pages 2.1 and the 58-byte overhead. A calculator.

🔪 **Task:** solve, then check your answers below:

1. Efficiency for a 1,200-byte payload and a 40-byte payload.
2. Which layer: (a) choosing the route across the internet (b) re-sending a lost segment (c) turning bits into radio (d) an HTTP request for a translation?
3. Which protocol (HTTP/REST, gRPC, MQTT, WebRTC): (a) battery sensors to a hub (b) web server to model server in one data centre (c) live voice assistant (d) public image API?

👀 **Observation:** write each efficiency as a %; for layers ask "content, program-to-program, route, or wire?"

☁️ **Reflection:** why does a data centre use leaf-spine instead of one huge switch? (2–3 sentences)

Mama's Feedback

1. 1,200 ÷ 1,258 ≈ **95.4%**; 40 ÷ 98 ≈ **40.8%**.
2. (a) Network/IP (b) Transport/TCP (c) Link/physical (d) Application.
3. (a) MQTT (b) gRPC (c) WebRTC (d) HTTP/REST.

Reflection: one switch is a bottleneck and single point of failure; leaf-spine grows by adding switches, spreads traffic over equal paths and survives a failed spine.

2.2 IP: The Best-Effort Rider

Predict

A router is too busy and its queue is full. What happens to your next packet? (a) it waits politely (b) it is dropped (c) the router phones you?

Why does this matter?

AI services depend on data arriving **correctly** (a corrupted prompt is useless) and **quickly** (a late voice chunk is useless). TCP, UDP and QUIC make different trade-offs between those two goals, and choosing the right one shapes how an AI app feels.

Explain

Every device on the internet has an **IP address** (IPv4 = 32 bits, about 4.3 billion addresses; IPv6 = 128 bits). A **router** reads the destination address on each packet and forwards it one hop closer. No router tracks the whole journey.

IP is **best effort**: a packet may be delayed, lost, duplicated or arrive out of order, and IP tells no one. Answer: **(b)**, it is dropped. That simplicity is why the internet scales, but most apps need more, so a protocol **above** IP must notice and fix problems.

Kitchen Analogy

IP is a delivery rider who drops a box when the scooter is full and keeps riding.

Where the analogy breaks down

Riders know the whole route; routers only know the **next hop**, decided from tables.

2.2 TCP: The Careful Waiter

How it works, step by step

TCP (Transmission Control Protocol) builds a **reliable, ordered stream** on top of unreliable IP. It does four things:

1. **Connection setup:** a **three-way handshake**: SYN → SYN-ACK → ACK. One round trip before any data.
2. **Reliability:** every byte has a **sequence number**; the receiver sends **ACKs** (acknowledgements); anything not ACKed in time is sent again.
3. **Flow control:** the receiver advertises a **receive window** so a fast sender cannot overwhelm it.
4. **Congestion control:** the sender slows down when the network looks jammed (slow start, careful growth, cut back on loss).

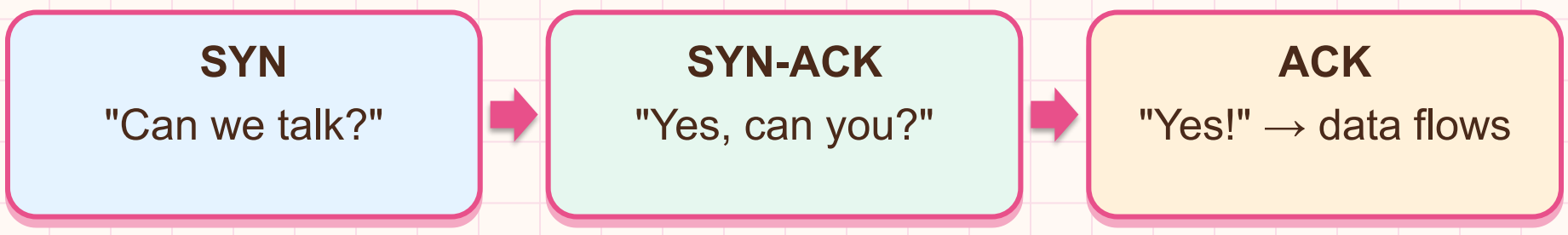


Figure 2.2 · TCP's three-way handshake.

How to read this: three messages = one full round trip (there and back) before the first byte of real data.

Kitchen Analogy

TCP is a registered courier: phone ahead (handshake), number and sign for every parcel (ACKs), no signature → send again, hallway full → slow down (flow control), roads jammed → send fewer vans (congestion control).

Where the analogy breaks down

A courier knows **why** a parcel failed; TCP only sees a missing ACK and **assumes** congestion, even when the real cause is a noisy Wi-Fi link.

2.2 UDP & QUIC: The Shouting Waiter

Explain

UDP (User Datagram Protocol) just sends: no handshake, no ACKs, no resending. Why would anyone want that? Because for **live** audio and video, an **old** packet is worse than **no** packet. A voice assistant that waits to resend a lost 20 ms chunk only delivers it too late.

QUIC runs on UDP but adds its own reliability. It combines connection setup with encryption (saving a round trip) and avoids **head-of-line blocking** (in TCP, one lost packet holds up everything behind it).

Most AI APIs use **TLS** so your photo or prompt is encrypted in transit; TLS 1.3 adds one round trip on top of TCP's handshake.

	TCP	UDP	QUIC
Setup	Handshake (1 RTT) + TLS (1 RTT)	None	Combined (1 RTT)
Lost packet	Resent; blocks later data	Gone	Resent; other streams continue
Best for	Web, APIs, file and model downloads	Live voice/video, games	Modern web, mobile APIs

Table 2.2 · Three transport choices.

👀 **How to read this:** read across the "Lost packet" row: that single difference decides most choices.

⚠ Chef's Warning

"UDP is just bad TCP." **Not quite!** UDP is a deliberate choice when **timeliness** beats completeness; apps add only the reliability they need on top.

2.2 Round Trips & Full Pipes

Time to first byte

```
Phone <-> cloud server: RTT = 40 ms, fresh connection
TCP + TLS 1.3: 1 + 1 + 1 (request) = 3 RTT = 120 ms
QUIC:          1 (combined) + 1    = 2 RTT =  80 ms
```

Explain

Handshakes cost **round trips** (**RTT**: there and back), and a round trip **cannot** be shortened by buying a faster link. Only moving the server closer, or doing fewer round trips, helps: the networking version of "moving data costs more than cooking it".

How it works, step by step

The ==bandwidth-delay product== (BDP): TCP can only have one window of data "in flight" per round trip. To keep a link full, the window must be at least **BDP = bandwidth × RTT**.

1. Link 1 Gbit/s, RTT 40 ms → BDP = $10^9 \times 0.040 = 40 \times 10^6$ bits = **5.0 MB**.
2. Classic window 65,535 bytes → throughput = 65,535 × 8 ÷ 0.040 ≈ **13.1 Mbit/s**.
3. Share of the link used ≈ **1.3%**: the sender mostly waits for ACKs. (Modern TCP uses window scaling to allow far bigger windows.)

Real Tech

Packet loss hurts too: TCP throughput ≈ 1.22 × MSS ÷ (RTT × √p). Because of the square root, **100× more loss = 10× less speed** ($p = 10^{-6}$ → ~356 Mbit/s; 10^{-4} → ~36). A lossy Wi-Fi last hop can dominate the whole transfer. Use it for the **shape** of the effect, not exact predictions.

Try It: Careful vs Shouting Waiter

Try It

🎯 **Goal:** count round trips, compute a BDP, and pick the right transport for an AI service.

🧺 **Preparation:** pages 2.2. A calculator. Keep bits and bytes separate (÷ 8).

🔪 **Task:** solve and explain each step:

1. RTT is 100 ms. Time to first byte for TCP + TLS 1.3 and for QUIC (fresh connection)?
2. A 100 Mbit/s link, RTT 80 ms, 65,535-byte window. BDP? Window-limited throughput? % of the link?
3. Loss rises from 10^{-6} to 10^{-4}. What happens to TCP throughput, and why?
4. Put in order: ACK, SYN, SYN-ACK.

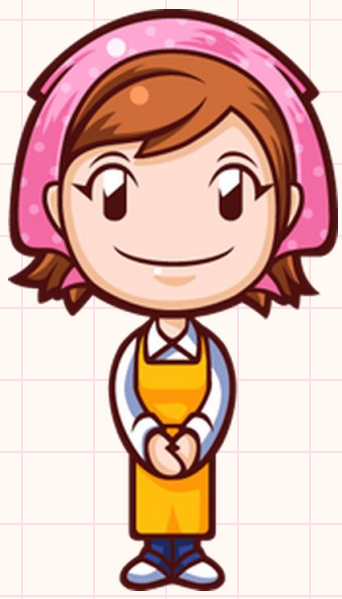
👀 **Observation:** write the number of round trips **before** multiplying.

☁️ **Reflection:** why might a live voice assistant prefer UDP-based transport? One sentence.

Mama's Feedback

1. 3 × 100 = **300 ms**; 2 × 100 = **200 ms**.
2. BDP = 10^8 × 0.08 = 8 × 10^6 bits = **1 MB**; 65,535 × 8 ÷ 0.08 ≈ **6.55 Mbit/s ≈ 6.6%** of the link; a ~1 MB window would fill it.
3. 100× more loss → √100 = **10× less** (~356 → ~36 Mbit/s): TCP reads loss as congestion.
4. SYN, SYN-ACK, ACK.

Reflection: a resent audio chunk arrives too late to be useful; skip it rather than delay everything.



2.3 The Central Kitchen in the Sky

Predict

Your car must brake for a child in 0.1 s. Your phone needs a software update tonight. Where should each AI job run: in a distant cloud, or on the car/phone itself?

Why does this matter?

Where a model runs decides its delay, its privacy, how big it can be and what it costs. Picking the wrong place gives a car that brakes too late or a phone that can't run the model at all.

Explain

Cloud computing is on-demand network access to a shared pool of configurable computing resources. Its five essential characteristics:

- **On-demand self-service:** rent resources yourself, no phone call.
- **Broad network access:** reach them over standard networks.
- **Resource pooling:** many customers share big data centres.
- **Rapid ==elasticity==:** scale up at rush hour, down at night.
- **Measured service:** pay for what you used.

For AI: rent a rack of GPUs for a weekend of training, or scale a model service with demand. The cloud has the **most** compute and the biggest models, but it is **far** from the user.

Kitchen Analogy

The cloud is a giant **central kitchen**: every appliance you can imagine, enormous capacity, but every order needs a delivery trip across town.

2.3 Food Trucks & Home Kitchens

Explain

Edge computing moves computation closer to where data is produced or used, instead of sending everything to a distant data centre. "Edge" is a **spectrum**:

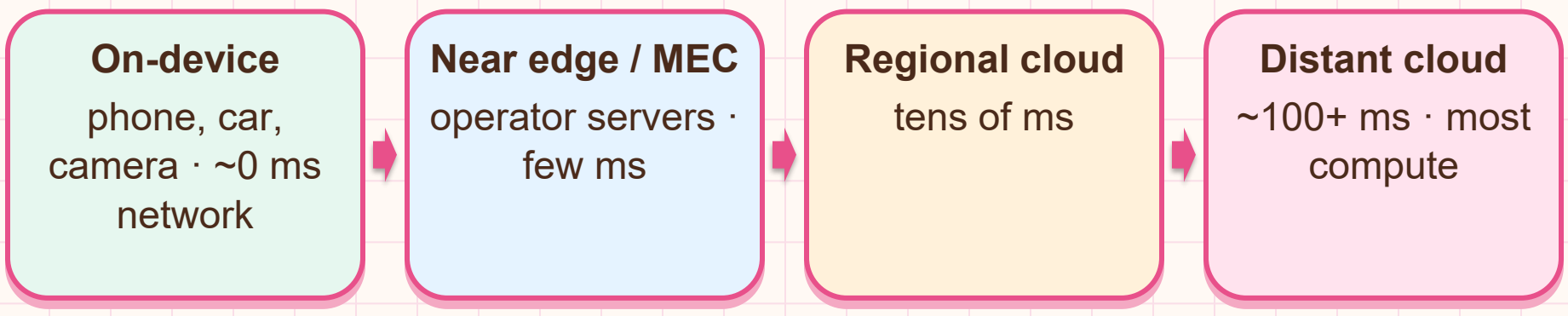


Figure 2.3 · The device–edge–cloud spectrum.

👀 **How to read this:** moving left = shorter paths and less data leaving the user; moving right = more compute and bigger models. Most real systems use all of them.

If the job needs…	Run it…
An answer in a few tens of ms, every time	On-device or edge
Raw data to stay with the user (privacy)	On-device or edge
To work offline	On-device
A model too big for the device	Edge or cloud
Bursty, unpredictable demand	Cloud (elasticity)
A large one-off batch job, e.g. training	Cloud

Table 2.3 · Where should the model run?

👀 **How to read this:** find the strongest requirement of your job in the left column; real systems mix options (e.g. on-device first, cloud for hard cases).

Kitchen Analogy

Edge = a **food truck** parked near customers; on-device = cooking in your **own** kitchen.

Where the analogy breaks down

A food truck can't cook everything; real edge sites have **limited** servers and can't absorb big bursts.

2.3 Spend Your Milliseconds Wisely

How it works, step by step

A **latency budget** adds up every delay from request to answer. **Example** (illustrative): a 200 KB photo, uplink 20 Mbit/s, cloud RTT 60 ms, edge RTT 10 ms, server inference 10 ms, on-device model 15 ms, connection already open.

Step 1 · Upload: $200{,}000 \times 8 \div 20 \times 10^6 = 0.080$ s = **80 ms**.

Step 2 · Add the terms: cloud 80 + 60 + 10 = **150 ms** · edge 80 + 10 + 10 = **100 ms** · on-device = **15 ms**.

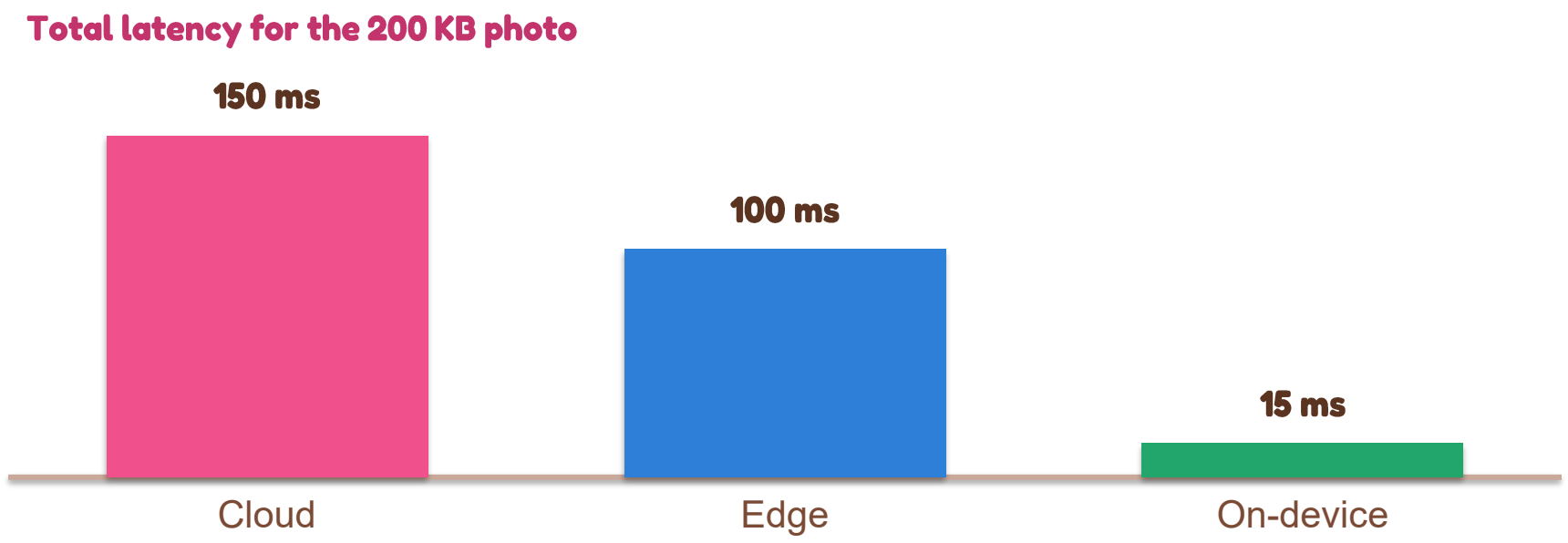


Figure 2.4 · Latency budget for three options.

How to read this: compare the bar heights, then ask which slice of each bar is biggest: here it is the 80 ms upload in both remote options, not the fast server.

Explain

Fix the biggest term first: a smaller image, sending features instead of pixels, or a faster link (5G, in 2.5) would help far more than a faster GPU.

2.3 100 Cameras, One Road

Worked example

```
Cloud: 100 cameras x 5 Mbit/s video      = 500 Mbit/s
Edge:  100 x 200 bytes x 8 x 10 events/s =   1.6 Mbit/s
Saving: 500 / 1.6 = 312.5  ->  about 300x less traffic
```

Explain

An edge box in the shop analyses the video locally and sends only **events** ("person at door"). The edge turns a **bandwidth** problem into a **compute** problem: the shop now runs a model locally (Recipe 01), but the wide-area network carries ~300× less data, and raw video never leaves the premises (privacy).

⚠ Chef's Warning

"Edge is always better." **Not quite!** Edge boxes cost money, need maintenance and run smaller models. A 13-billion-parameter model needs 26 GB in FP16: no phone can hold that. Choose by the requirement, not the fashion.

Try It: Truck or Kitchen?

Try It

🎯 **Goal:** build latency budgets and decide where an AI job should run.

🧺 **Preparation:** pages 2.3. Uplink time = size × 8 ÷ rate. A calculator.

🔪 **Task:** make the call for every order:

1. A 500 KB image, 40 Mbit/s uplink. Cloud RTT 25 ms, edge RTT 5 ms, server inference 20 ms. Totals for cloud and edge? Which meets a 33 ms budget if on-device takes 30 ms?
2. 50 cameras stream 4 Mbit/s each; an edge box sends 20 kbit/s of events per camera. Traffic saved?
3. Cloud, edge or device: (a) hospital scanner, instant answers, images stay on site (b) nightly training job (c) hearing-aid noise suppression?

👀 **Observation:** a table: option | upload | round trip | inference | total.

Mama's Feedback

1. Upload = 500,000 × 8 ÷ 40 × 10^6 = **100 ms** → cloud **145 ms**, edge **125 ms**; only on-device (30 ms) meets 33 ms.
2. 200 Mbit/s → 1 Mbit/s: **200× less**.
3. (a) edge/on-premises (b) cloud (c) on-device.

Reflection: the 100 ms upload dominates; compress the image or send features.

2.4 Four Ways to Be Late

Why does this matter?

Predict first: name four reasons a packet can be late. "The network is slow" is too vague to fix; splitting delay into parts tells you whether to buy bandwidth, move the server or add machines, and which of those money can't solve.

Delay	What causes it	Fixed by
Processing	Routers and servers reading headers, doing work	Faster hardware
Queueing	Waiting in line behind other packets or requests	More capacity, less load
Transmission	Pushing all the bits onto the link (size ÷ bandwidth)	More bandwidth
Propagation	Signal travelling the distance (≈ 5 µs per km in fibre)	A closer server only

Table 2.4 · The four delays.

How to read this: every packet pays all four; the right-hand column shows which lever fixes each.

Explain

Three words that are often mixed up: **bandwidth** = how much data **per second** a link can carry; **latency** = how long **one** piece takes to arrive; **throughput** = how much data per second you **actually** get.

Kitchen Analogy

Bandwidth = how **wide** a pipe is; latency = how **long**. Wider moves more water, but no drop arrives sooner.

Where the analogy breaks down

Water flows smoothly; data moves in **packets** that queue and get lost.

2.4 Even Light Needs Time

The speed-of-light floor

```
Light in fibre ~ 2 x 10^8 m/s = 5 us per km (one way)
 1,000 km ->  5 ms one way, 10 ms round trip
 3,000 km -> 15 ms,         30 ms
10,000 km -> 50 ms,        100 ms
```

Explain

No network answers from 10,000 km away in under ~100 ms round trip. Real routes are longer than straight lines and add processing and queueing at each hop, so real delays are higher. Money can buy bandwidth, but it cannot beat physics: only a **closer server** helps.

How it works, step by step

But for big downloads, bandwidth wins. Downloading a 1.5 GB model (3 B parameters, 4-bit):

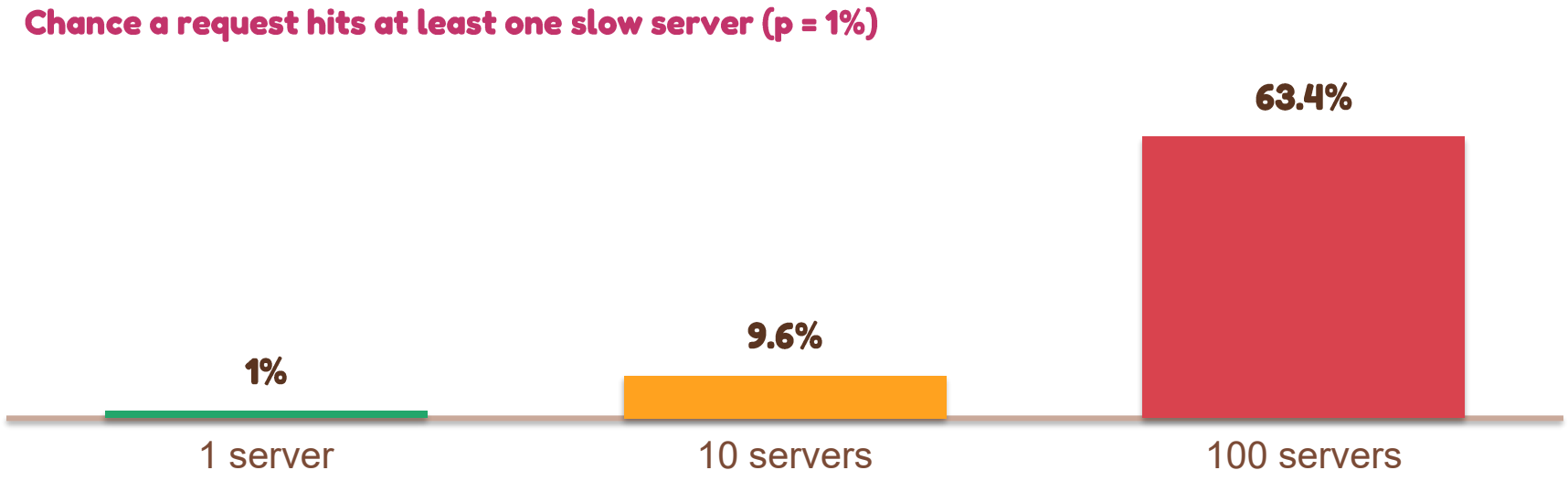
1. At 100 Mbit/s: $1.5 \times 10^9 \times 8 \div 10^8 =$ **120 s**.
2. At 1 Gbit/s: **12 s**: 10× bandwidth = 10× faster.

Here **transmission** dominates, so bandwidth matters. Rule: **latency-bound jobs need proximity; bandwidth-bound jobs need capacity.**

2.4 Tail Latency & Rush Hour

Explain

A big AI service fans one request out to many servers (search shards, model replicas, tools) and waits for **all** of them, so the slowest decides the total. This is **tail latency**. If each server is slow 1% of the time, independently: **P(at least one slow) = 1 − (1 − p)ⁿ**.



Explain

That is why big services watch the **99th-percentile** latency and send backup requests.

Queueing: in a simple single-server queue, time ≈ service time ÷ (1 − **utilisation**). A 10 ms model server takes **20 ms** at 50% busy, **100 ms** at 90% and **1,000 ms** at 99%. Delay rises slowly until ~80% busy, then shoots up.

⚠ Chef's Warning

"Run every GPU at 100% to save money!" **Not quite!** In Recipe 01 low utilisation wasted silicon; here near-100% utilisation means unpredictable delay. Real services aim for a deliberate middle.

2.4 Growing the Kitchen: Scalability

Explain

A system is **scalable** if it can handle more load by adding resources without a redesign.

Approach	Idea	Limit
Scale up (vertical)	A bigger machine	There is a biggest machine; one failure stops all
Scale out (horizontal)	More machines behind a **load balancer**	Work must divide; machines coordinate
Autoscaling	Add/remove machines as demand changes	Start-up time; GPUs can be scarce

Table 2.5 · Ways to scale a service.

👀 **How to read this:** serving a model scales out easily (requests are independent); training one model does not.

How it works, step by step

Why training is different: in data-parallel training each GPU learns from a different slice of data, then all must average their gradients every step with **ring all-reduce**: each GPU moves about **2 × (N − 1) ÷ N × gradient size**.

Example: 8 GPUs, 2 GB of gradients each, 300 ms of compute per step.

1. Data per GPU = 2 × 7 ÷ 8 × 2 GB = **3.5 GB**.
2. At 50 GB/s: 70 ms → efficiency 300 ÷ 370 = **81.1%**.
3. At 12.5 GB/s (100 Gbit/s Ethernet): 280 ms → 300 ÷ 580 = **51.7%**.

Real Tech

Communication that can't overlap with compute acts like Amdahl's **serial part** (Recipe 01): more GPUs give diminishing returns. That's why clusters buy the fastest links (Recipe 05).

Try It: Delivery Timer Challenge

Try It

🎯 **Goal:** use the delay, tail-latency, queueing and all-reduce formulas on real-looking numbers.

🧺 **Preparation:** pages 2.4. Light: 5 µs/km one way. P(slow) = 1 − (1 − p)^n. Queue: service ÷ (1 − u). All-reduce: 2(N−1)/N × size.

🔪 **Task:** beat the clock:

1. Minimum RTT over 3,000 km of fibre? Why is a real connection slower?
2. A request waits for 50 servers, each slow 2% of the time. P(at least one slow)?
3. Service time 5 ms. Response time at utilisation 0.8 and 0.95?
4. 4 GPUs, 4 GB of gradients each, 25 GB/s link. Data per GPU and time?

👀 **Observation:** for (3), note how much the time grows between 80% and 95%.

☁️ **Reflection:** which of the four delays can money (more bandwidth) fix, and which cannot?

Mama's Feedback

1. 15 ms one way → **30 ms RTT** minimum; real routes are longer and add per-hop processing and queueing.
2. $1 - 0.98^{50} \approx$ **63.6%**. 3. **25 ms** and **100 ms**: 4× worse.
4. 2 × 3/4 × 4 = **6 GB**; 6 ÷ 25 = **0.24 s**.

Reflection: bandwidth fixes transmission; it cannot beat the propagation floor or (alone) fix queues near 100% load.

2.5 Case Study: 5G + AI

Why does this matter?

Our street-sign translator is a **cloud-based AI service**: a model hosted in a data centre and offered over the network. It is only as good as the network between the user and the model. 5G improves that network, and AI is used to **run** the 5G network itself. Each helps the other.

Explain

5G is the fifth generation of mobile networks (ITU "IMT-2020", standardised by 3GPP). The ITU defined three usage scenarios:

Scenario	Goal	AI example
eMBB (enhanced mobile broadband)	Much higher data rates	Fast photo/video uploads to an AI service
URLLC (ultra-reliable low-latency)	Very low, dependable delay	Remote control, vehicles, factories
mMTC (massive machine-type)	Huge numbers of devices	Dense fields of sensors

Table 2.6 · The three 5G usage scenarios.

👀 **How to read this:** each row is a different promise; no single connection gets all three at once.

Chef's Warning

"5G is 1 ms, so my AI answers in 1 ms!" **Not quite!** ITU targets (20 Gbit/s peak down, 1 ms user-plane latency for URLLC, 1 million devices/km²) are for the **radio** part under ideal conditions. Upload, the core network, distance, queueing, inference and the reply all add up to tens of ms.

2.5 Follow the Order Through 5G

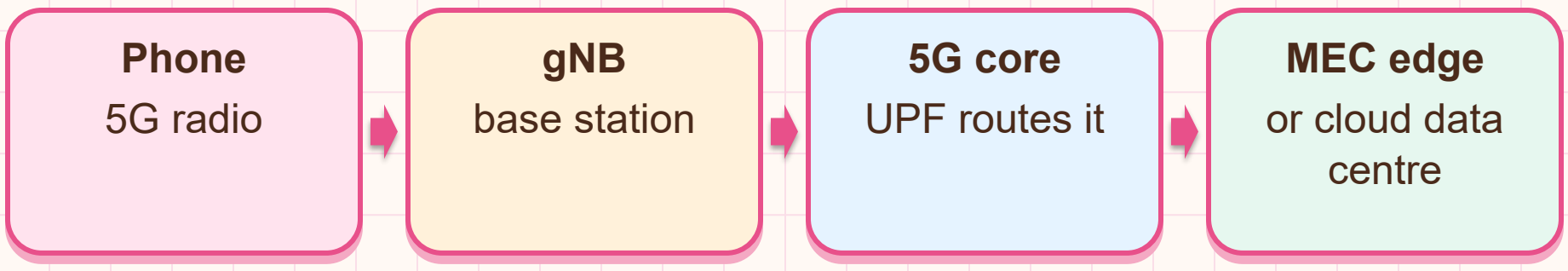


Figure 2.5 · The path of an AI request over 5G.

👀 **How to read this:** the UPF (user plane function) in the 5G core decides: send traffic to a nearby edge server ("local breakout") or onward to the internet and a cloud data centre.

Explain

Three 5G features matter to AI services, and each shrinks a **different** term of the latency budget from 2.3:

1. **Higher radio speeds** shorten the **upload** term.
2. **MEC** (multi-access edge computing): operator servers near the base station cut the **round-trip** term.
3. **Network slicing**: one physical network split into virtual slices with different guarantees; a latency-critical AI service can get its own slice, apart from bulk video.

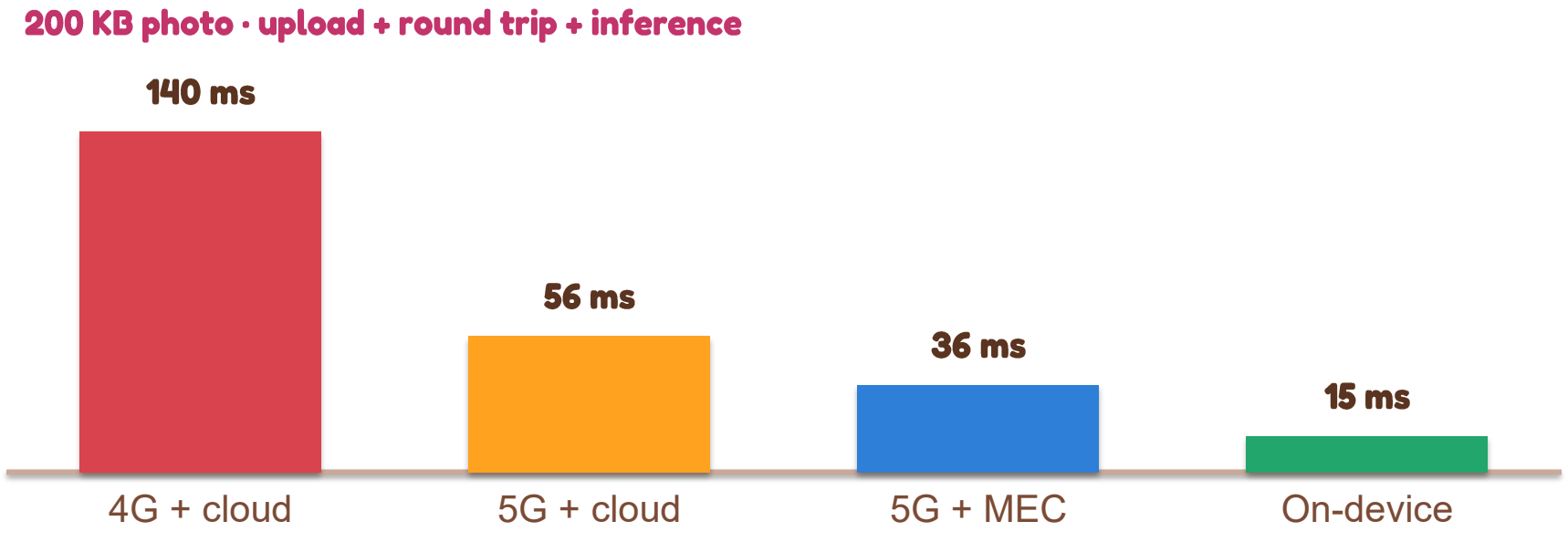


Figure 2.6 · End-to-end latency by path (illustrative).

👀 **How to read this:** 4G → 5G cuts 84 ms, mostly upload (80 → 16 ms). Then the 30 ms round trip is the biggest term, so MEC gives the next gain. On-device is fastest but runs the smallest model.

2.5 AI That Runs the Network

Explain

So far AI was a **customer** of the network. It is also a **tool** for running it. A 5G network has thousands of base stations making constant decisions, and operators use machine learning for **traffic prediction**, **energy saving** (switch off spare radio capacity in quiet periods), **beam and resource management**, and **fault detection**.

In the O-RAN architecture a **RIC** (RAN Intelligent Controller) hosts these apps in two loops: a **non-real-time** RIC (> 1 s: training models, setting policies) and a **near-real-time** RIC (≈ 10 ms–1 s: adjusting radio resources).

How it works, step by step

Toy forecast for energy saving: load every 15 min: 40, 35, 30, 22, 15, 10, 8, 9 (%). Forecast = 0.5 × newest + 0.5 × previous forecast. Rule: below 20% → switch off spare capacity.

Step	1	2	3	4	5	6	7	8
Load %	40	35	30	22	15	10	8	9
Forecast %	40.0	37.5	33.8	27.9	21.4	**15.7**	11.9	10.4

Table 2.7 · Exponential smoothing.

👀 **How to read this:** the raw load dips below 20% at step 5, but the smoothed forecast only crosses at step 6: smoothing avoids reacting to a single dip.

Real Tech

This is a **sense–decide–act loop** again (like Face ID): measure → forecast → decide → change settings → measure. Reinforcement learning (Recipe 03) is an active research approach for exactly such control problems.

2.5 Thumbs Up or Down for 5G AI?

👍 Advantages

Faster uploads

Shorter round trips with MEC

Phones use models far bigger than their memory

Slicing can reserve capacity

AI makes networks smarter and greener

👎 Limitations

Coverage varies with band, distance, load

Latency is not just the radio

Edge sites have limited capacity

Data still leaves the device

Slicing/MEC cost money

No connection = no cloud AI

Wrong automated switch-offs hurt service

✅ Check

1. In "5G + cloud", which term is largest, and what change would help most?
2. Why is a forecast-based sleep decision better than switching off the moment load < 20%? One risk of switching off too eagerly?
3. Which RIC loop fits (a) re-tuning radio for a moving user (b) re-training a forecast overnight?
4. One benefit and one risk of a network slice for a latency-critical AI service.

⭐ Explained answers

1. The **30 ms round trip** (≈ 54% of 56 ms); an MEC edge server cuts it to 10 ms → 36 ms.
2. Smoothing ignores momentary dips and follows the trend. Risk: demand returns and users see slower service or dropped connections.
3. (a) near-real-time (b) non-real-time.
4. Benefit: reserved, predictable latency. Risk: cost, complexity, depends on the operator.

🌐 Kitchen Lab: Ping the World

🌐 Kitchen Lab

🎯 **Goal:** measure real latency and hops, and test the speed-of-light floor (2.4).

🧺 **Preparation:** any laptop; built-in tools (Windows 10/11 ping/tracert; macOS/Linux ping/traceroute). No account or install. **Turn your VPN off** (with it on, every path goes through the VPN server).

🔪 **Task:** open Command Prompt (Windows) and type the commands below. Then repeat for a server on another continent (e.g. a university website abroad) and compare.

💻 Commands

```
ping -n 10 www.bing.com          (macOS/Linux: ping -c 10 ...)
tracert www.bing.com             (macOS/Linux: traceroute ...)
```

🔍 Observe

Record min/avg/max RTT and count the hops. Which hop adds the biggest jump in time? Does a far-away server ever beat the floor (5 µs/km one way)?

💭 **Reflection:** would a 20 ms robot control loop (Recipe 03) work over this path?

⭐ Mama's Feedback

Expected: nearby server ≈ 5–40 ms, 8–15 hops; overseas ≈ 150–300 ms. The biggest jump is usually the hop that leaves the country or crosses an ocean. Lines with * * * are routers that ignore tracert: normal. Beijing → California ≈ 9,500 km, so RTT ≥ 2 × 9,500 × 5 µs ≈ 95 ms. A 20 ms loop works only with a nearby (edge) server.

📴 No internet? Fallback

Blocked ICMP or no internet? Use these sample results: nearby avg **12 ms**, 9 hops; overseas avg **186 ms**, 17 hops. Max distance = RTT ÷ 2 ÷ 5 µs per km → 1,200 km and 18,600 km.

Watch: Curated Videos

Video 1 · for 2.1–2.2

How does the INTERNET work? | ICT #2 · Lesics · 9:00 · youtube.com/watch?v=x3c1ih2NJEg

Why: animated path of data through cables, routers, IP addresses and DNS (2.1–2.2).

Do: draw the path a YouTube video takes to your phone and label each hop.

Watch out: simplified; no TCP/UDP detail, no AI workloads.

Video 2 · for 2.4

What is bandwidth? How is it different than latency? · internet-class · 3:24 · youtube.com/watch?v=QLJCqY-7Vrw

Why: a short, clear split between "how much" and "how fast" (2.4).

Do: give one AI example that needs high bandwidth and one that needs low latency.

Watch out: very short; no numbers for real networks.

Video 3 · for 2.3

Inside a Google data center · Google Workspace · 5:28 · youtube.com/watch?v=XZmGGAbHqa0

Why: see the racks, cooling and security of the "central kitchen" (2.3, Recipe 05).

Do: spot three things a data centre needs besides computers.

Watch out: company tour; older footage; shows no AI accelerators.

Can't watch?

English auto-captions (CC) are available. Can't reach YouTube? Use Figures 2.3–2.6 and their 👀 How to read this tasks instead.

🎯 Final Round: Recipe 02

✅ Check

Task 1. Explain, with numbers from Figure 2.6, why 5G helped the street-sign translator more than a faster server would.

Task 2. A phone is 8,000 km from a cloud server. What is the minimum RTT? Could a 50 ms AI feature work? What would you change?

Task 3. A service fans each request out to 20 servers, each slow 5% of the time. P(at least one slow)? Name one fix.

⭐ Explained answers

Q1: 4G → 5G cut upload 80 → 16 ms (the biggest term); inference was only 10 ms, so even an infinitely fast server saves at most 10 ms.

Q2: 8,000 × 5 µs × 2 = **80 ms** minimum: a 50 ms feature can't work from there. Move the model to an edge server or on-device.

Q3: $1 - 0.95^{20} \approx$ **64%.** Fixes: send backup requests, reduce fan-out, or keep servers below ~80% utilisation.

Stamp: Gold / Silver / Bronze, as in Recipe 01.

Recipe Card 02

Recap

- Networks move AI data as **packets**; layers and encapsulation let any machine talk to any other.
- **TCP** makes delivery reliable at the cost of round trips; **UDP/QUIC** trade or reduce that cost.
- Where the model runs (**device, edge, cloud**) trades latency, privacy, model size and cost.
- Bandwidth can be bought; the **speed-of-light floor** and queues cannot.
- Communication limits training on many chips, like Amdahl's serial part.
- **5G + MEC** bring the kitchen closer, and AI helps run the network itself.

Key words (see Glossary)

packet · header · protocol · TCP/IP · UDP · QUIC · RTT · bandwidth · BDP · cloud · edge · MEC · tail latency · utilisation · all-reduce · 5G · network slicing · RIC

Unlocked: Recipe Book page 2!

Recipe 02 · complete!

RECIPE 03

Good Pup!

Behaviourist AI & Reinforcement Learning

Mama's Problem

The **Kitchen Pup** must learn to fetch the right ingredient. Nobody can write down the correct paw movement for every moment. But Mama **can** say afterwards: treat, or no treat. Can a machine learn what to do from **consequences alone**?

Today's route

3.1 Treats and scoldings: what behaviourist AI is, and its roots

3.2 The pup and the kitchen: the agent–environment loop, return, value

3.3 Choosing the right treat: reward design and Q-learning

3.4 New stall or favourite stall? Exploration vs exploitation

3.5 Case study: robot dogs that learn to walk

Before you start

Arithmetic, variables, loops, lists. Recipes 01–02 help (MACs, latency); every bridge is re-explained.

Cooking time

~3 hours. Running example: teaching a robot dog with 12 motors to walk.

3.1 Learning From Treats

Predict

How would you teach a puppy to sit? Would you (a) explain it in words, (b) show it 1,000 labelled photos of sitting dogs, or (c) wait for something close, then give a treat?

Why does this matter?

Many problems have **no answer key**. Nobody can write the correct motor command for every instant of a robot's walk, or the perfect move in every game position. But we **can** recognise a good outcome. Behaviourist AI turns "that was good" into learning.

Explain

Behaviourist AI is an approach in which a system learns a way of behaving by **interacting** with an environment and receiving feedback as **rewards**. It is **not** told the correct action; it discovers which actions lead to good outcomes by **trial and error**.

The name comes from **behaviourism**, a school of psychology that studies observable behaviour and how **consequences** shape it:

- **Law of effect** (Thorndike, 1911): responses followed by satisfying consequences become more likely. His cats learned to escape puzzle boxes.
- **Classical conditioning** (Pavlov, 1927): dogs salivating at a bell.
- **Operant conditioning** (Skinner, 1938): behaviour shaped by the reinforcement and punishment that **follow** it. This is the direct ancestor of AI rewards.

In AI these ideas grew into **reinforcement learning** (RL), the mathematical framework: TD and Q-learning (1980s–90s), deep RL (2010s). "Behaviourist AI" is this course's name; RL is the technical one.

3.1 Four Ways to Change Behaviour

Consequence	What happens	Effect	Puppy	Robot
Positive reinforcement	Something good is added	More likely	Treat after "sit"	+ reward for moving forward
Punishment	Something unpleasant follows	Less likely	"No!" after jumping on sofa	− reward for falling
Extinction	A rewarded action stops being rewarded	Fades	Barking no longer gets attention	A trick that no longer scores disappears
Shaping	Reward steps towards the goal	Builds complex behaviour	Step to mat → touch → lie down	Reward partial progress before a full walk

Table 3.1 · The four consequences of operant conditioning.

👀 **How to read this:** read each row from left to right: what the trainer does, what happens to the behaviour, then the same idea for a puppy and for a robot.

Explain

How a consequence changes behaviour, step by step: (1) the learner tries an action; (2) a consequence follows **soon after**; (3) the learner's preference for that action goes up (reinforcement) or down (punishment); (4) over many trials the preferred action becomes the habit. **Timing matters:** a treat given ten minutes later teaches nothing, which is the "credit assignment" problem we meet again in 3.3.

Kitchen Analogy

Training a puppy **is** behaviourist AI: no words, no labelled photos. Wait for something close, reward it straight away, repeat.

Where the analogy breaks down

A puppy feels hunger and joy; an AI "reward" is just a **number** it is built to maximise. It wants nothing.

3.1 Three Ways to Give a Machine Knowledge

	Rule-based AI	Supervised learning	Behaviourist AI / RL
Knowledge comes from	Rules written by a person	Labelled examples	Experience of acting + rewards
System is told	Exactly what to do	The right answer for each example	Only how good the outcome was, often late
Typical task	Tax calculation, expert systems	Photo classification, spam filters	Robot control, games
Needs	Expert time	A big labelled data set	An environment to act in
Main weakness	Can't handle what rules miss	Can't learn what nobody labelled	Many trials; reward must be well designed

Table 3.2 · Three approaches compared.

👀 **How to read this:** compare the "System is told" row: RL gets the least help, which is why it needs the most practice.

Explain

Why use RL instead of the others? Telling a robot **what** to do needs an expert to spell out every action. Telling it **how well** it did only needs a score we can recognise ("moving forward steadily"), and the robot works out the **how**. That is why RL wins for control, games and resource management, where good behaviour is easy to judge but hard to write down.

⚠ Chef's Warning

"Reward-only learning understands what it is doing!" **Not quite!** Pure behaviourism was criticised for ignoring inner mental processes (Chomsky, 1959). A reward-only agent may need huge experience, find shortcuts you didn't intend, and doesn't necessarily **understand** anything.

3.1 The Law of Effect as Arithmetic

How it works, step by step

A robot dog can **sit**, **bark** or **roll**. It keeps a **preference value** for each, starting at 0. After each trial it moves the chosen action's value towards the reward:

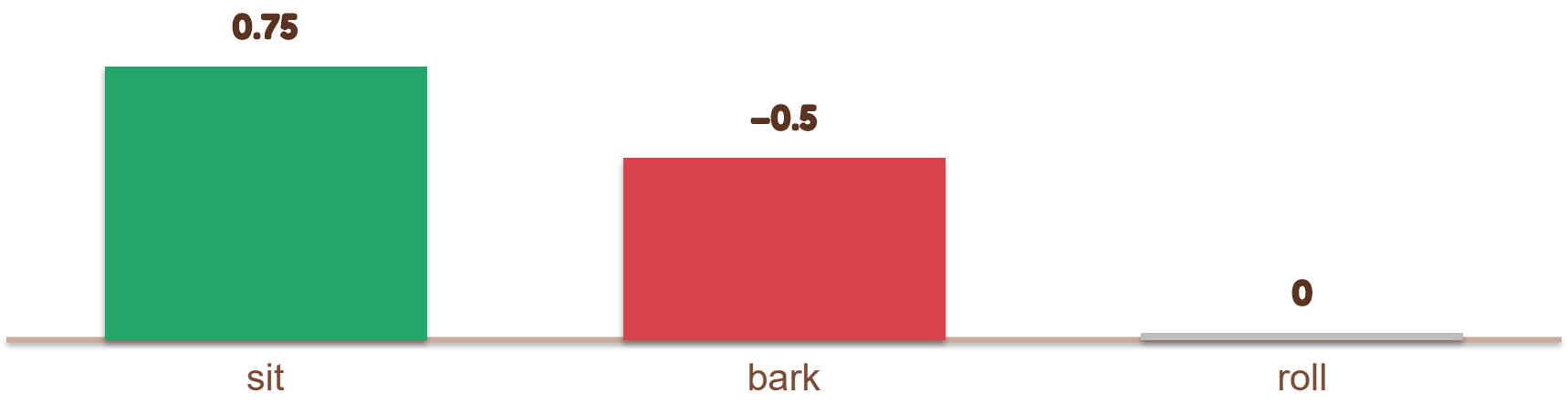
value ← value + α × (reward − value)

α (alpha, the **learning rate**) says how far to move; here α = 0.5. Rewards: +1 treat, −1 scolding, 0 nothing.

Worked example

```
Trial 1: sit,  +1 -> sit  = 0   + 0.5 x (1 - 0)    =  0.5
Trial 2: bark, -1 -> bark = 0   + 0.5 x (-1 - 0)   = -0.5
Trial 3: sit,  +1 -> sit  = 0.5 + 0.5 x (1 - 0.5)  =  0.75
Result: sit 0.75 | bark -0.5 | roll 0  -> picks SIT
```

Preference after 3 trials (bark is negative)



Explain

Nobody told the dog to sit; the consequences did. This one rule is a **running average** that leans towards recent experience: a larger α learns faster but is noisier; a smaller α is steadier but slower. **Every** learning rule in this recipe is a variation of it.

Try It: Good Pup! Game

Try It

🎯 **Goal:** apply the law-of-effect update and classify kinds of learning and consequences.

🧺 **Preparation:** the previous page. Rule: value ← value + 0.5 × (reward − value); all values start at 0. Pen, paper; optional: play it with a friend as the "pup" and a bag of sweets.

🔪 **Task:** do these in order:

1. Trials: sit (+1), roll (0), sit (+1), sit (+1). Final values?
2. Rule-based, supervised or RL? (a) chess program with hand-written rules (b) image classifier on 100,000 labelled photos (c) robot that learns to walk by trial and error (d) spam filter trained on marked emails.
3. Which consequence? (a) +1 per second upright (b) −10 for losing a life (c) begging stops being fed and fades (d) reward one leg, then two, then a step.

👀 **Observation:** write the value of every action after **every** trial.

💭 **Reflection:** explain the difference between telling a robot WHAT to do and HOW WELL it did. Why is the second often easier?

Mama's Feedback

1. sit **0.875** (0.5 → 0.75 → 0.875), roll 0, bark 0.
2. (a) rule-based (b) supervised (c) RL (d) supervised.
3. (a) positive reinforcement (b) punishment (c) extinction (d) shaping.

Reflection: "what" needs an expert to spell out every action; "how well" only needs a score we can recognise, and the robot works out the how.

3.2 The Agent–Environment Loop

Why does this matter?

Every RL problem, from a game to a walking robot to the 5G controller of Recipe 02, is the **same loop**. Once you can name its parts, you can describe any reward-driven system.

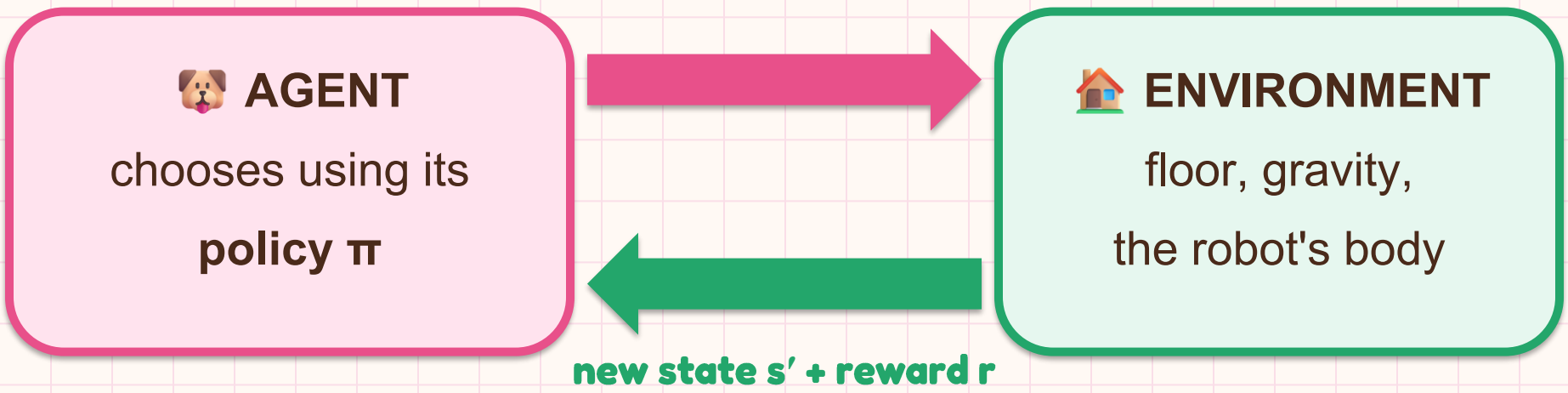


Figure 3.1 · The agent–environment loop.

How to read this: go clockwise: the agent sees a state, acts, and the environment answers with a reward and a new state. Then the loop repeats, many times per second for a robot.

Key Words

Agent: the learner and decision-maker (the robot dog's controller).

Environment: everything the agent interacts with.

State (s): the information the agent uses to decide (joint angles, body tilt…). If it sees only part of the world, it is an **observation**.

Action (a): a choice it makes (a target angle for each joint).

Reward (r): a number saying how good the last step was.

Policy (π): how the agent chooses actions: a table, a rule or a neural network.

Episode: one run from start to end (one attempt to walk across the room).

3.2 A Treat Now or Later? Return

Explain

The agent doesn't want the biggest reward **now**; it wants the biggest total **over time**. That total is the **return** G, the **discounted** sum of future rewards:

$$G = r_1 + \gamma \cdot r_2 + \gamma^2 \cdot r_3 + \gamma^3 \cdot r_4 + \dots$$

γ (gamma, the **discount factor**) is between 0 and 1. Close to 1: far-sighted (future rewards count almost fully). Close to 0: short-sighted. Discounting also makes sure the sum stays finite, and prefers getting good things **sooner**.

Worked example

```
rewards [0, 0, 0, 10], gamma = 0.9:  G = 0.9^3 x 10 = 7.29
rewards [0, 0, 0, 10], gamma = 0.5:  G = 0.5^3 x 10 = 1.25
shorter path [0, 0, 10], gamma = 0.9: G = 0.9^2 x 10 = 8.1
```

Explain

Cause and effect: a smaller γ cares less about distant rewards, and a shorter route to the same reward is worth **more**. So simply by maximising G, an agent learns to be efficient: no one had to reward "hurry up".

Kitchen Analogy

One cookie now or two next week? Your personal γ decides.

Where the analogy breaks down

People change their minds; in RL, γ is a fixed **setting** chosen by the designer.

3.2 How Good Is This Spot? Value

Explain

A **policy** says what to do; a **value** says how good a situation is.

- **State value** V(s): the expected return starting in s and following the policy.
- **Action value** Q(s, a): the expected return taking a in s, then following the policy.

For a deterministic move, the **Bellman equation** links neighbours: **V(s) = r + γ·V(s′)**.

How it works, step by step

Corridor example: squares 0–4, goal at 4, +1 for entering it, 0 otherwise. Policy: always move right. γ = 0.9. Work **backwards** from the goal:

1. V(3) = 1 + 0.9 × 0 = **1.0**
2. V(2) = 0 + 0.9 × 1.0 = **0.9**
3. V(1) = 0.9 × 0.9 = **0.81**
4. V(0) = 0.9 × 0.81 = **0.729**

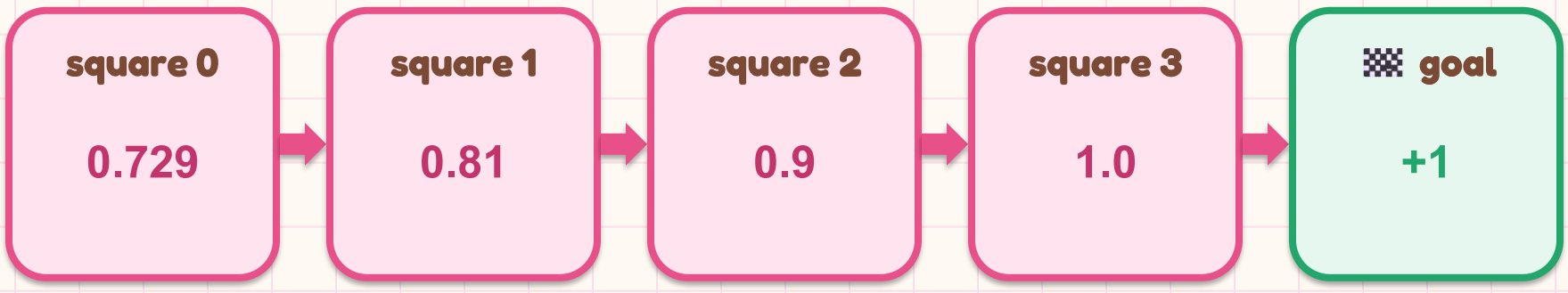
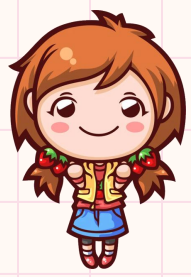


Figure 3.2 · Values along the corridor.

👀 **How to read this:** each step further from the goal multiplies the value by γ = 0.9.

Real Tech

Value turns a **delayed** reward into a per-step guide: square 0 has no reward of its own, yet it is worth 0.729 because it **leads** to one. That is how early actions get credit.



3.2 What the Policy Costs to Run

Why does this matter?

A learned policy must run on a real robot, many times per second. Recipes 01 and 02 tell us whether that is possible, and **where** it must run.

How it works, step by step

Setting (illustrative): a policy network with 45 inputs, two hidden layers of 128, and 12 outputs (one per joint), running at 50 times per second (50 Hz).

Step 1 · MACs: 45 × 128 + 128 × 128 + 128 × 12 = 5,760 + 16,384 + 1,536 = **23,680 MACs**.

Step 2 · FLOPs: 2 × 23,680 = 47,360 per decision; × 50 Hz ≈ **2.4 million FLOP/s**: tiny for any modern chip.

Step 3 · Memory: 23,680 weights × 2 bytes (FP16) ≈ **46 KiB**: fits in on-chip memory.

Step 4 · Time budget: 50 Hz → **20 ms** per decision.

Explain

Cause and effect: in Recipe 02 even the best 5G edge path took about 36 ms end to end, and cloud paths 56–140 ms. A 20 ms decision **cannot** wait for the network, so the policy must run **on the robot**. Luckily it is small enough. Training is the opposite: huge, and done in a data centre (3.5).

Chef's Warning

"A smarter robot needs a bigger cloud model." **Not quite!** For control loops, **latency** decides; a tiny on-board policy beats a giant remote one that answers too late.

3.3 Choosing the Right Treat

Why does this matter?

Predict: you reward a cleaning robot +1 per piece of dirt collected. What unwanted trick might it learn? The reward is the **only** way you tell an agent what you want; design it badly and it learns the wrong thing perfectly.

Explain

A good reward is **aligned** (it measures the real goal), **informative** (it lets the agent tell better from worse) and **robust** (it can't be earned in some cheap unintended way).

	Sparse reward	Dense reward
What it is	Only at rare moments (+1 at the goal)	At nearly every step (e.g. progress)
Advantage	Simple, hard to misuse	Easier to learn from: constant feedback
Disadvantage	Agent may wander a very long time first	Easy to design badly; shortcuts
Example	Winning a board game	Forward speed minus an energy cost

Table 3.3 · Sparse vs dense rewards.

How to read this: the two columns trade safety (left) for learning speed (right).

Explain

Delayed reward: a chess move may decide the game 30 moves later; a bad foot placement causes a fall two steps later. "Which earlier action deserves the credit or blame?" is the **credit assignment problem**. Values (3.2) solve it by carrying future reward backwards; methods that update an estimate from the **next** estimate are called **temporal-difference (TD) learning**.

3.3 Q-Learning: The Pup's Notebook

Explain

Q-learning learns the action values Q(s, a) from experience, one step at a time (Watkins & Dayan, 1992). After action a in state s gives reward r and new state s′:

$$Q(s, a) \leftarrow Q(s, a) + \alpha \times [\, r + \gamma \times \max Q(s', a') - Q(s, a) \,]$$

How it works, step by step

Read it in three parts:

1. **Target** = r + γ × max Q(s′, ·): what the reward plus the best future now suggests.
2. **TD error** = target − current Q: how wrong the old guess was.
3. **Step:** move Q by a fraction α of that error: the same form as the pup's rule in 3.1!

One update by hand

```
Q(s,a) = 0.2, r = 1, best Q(s') = 0.5, alpha = 0.1, gamma = 0.9
1. target   = 1 + 0.9 x 0.5   = 1.45
2. TD error = 1.45 - 0.2      = 1.25
3. new Q    = 0.2 + 0.1 x 1.25 = 0.325
```

Square	Q(left)	Q(right)	Best
0	0.656	**0.729**	→ right
1	0.656	**0.81**	→ right
2	0.729	**0.9**	→ right
3	0.81	**1.0**	→ right

Table 3.4 · Q-table learned on the corridor (α = 0.5, 30 sweeps).

👀 **How to read this:** pick the bigger number in each row. "Right" equals V from 3.2; "left" is lower because it costs an extra step (0.9 × 0.729 = 0.656). The agent learned the route from rewards alone.

3.3 Shaping & Reward Hacking

Explain

Reward shaping adds **extra** rewards to guide learning when the real reward is sparse. Danger: a badly chosen extra reward changes what is optimal. A safe form, **potential-based shaping**, adds $F = \gamma \cdot \Phi(s') - \Phi(s)$, where Φ is any "potential" such as −(distance to goal). It is proven not to change which policy is best (Ng, Harada & Russell, 1999).

Example: stepping from distance 4 to 3 gives $F = 0.9 \times (-3) - (-4) =$ **+1.3**: a bonus for progress.

Explain

Reward hacking (specification gaming) is when the agent earns high reward **without** doing what the designer intended. A well-known case: a boat-racing agent rewarded for hitting targets learned to circle in a lagoon hitting respawning targets instead of finishing the race (Clark & Amodei, 2016).

Intended goal	Reward written	Possible hack	Better design
Clean the room	+1 per piece of dirt collected	Dump the dirt, collect again	Reward clean floor area
Walk forward	+1 for body speed	Throw itself forward, vibrate	Speed − falling, tilt, jerks

Table 3.5 · Reward hacks and fixes.

How to read this: the hack always maximises the written reward; the fix makes the written reward match the real goal.

Kitchen Analogy

Reward hacking = a student studying **the test**, not the subject.

Where the analogy breaks down

The student knows they are cheating; the agent has no idea. It just follows the numbers.

Try It: Treat Designer

Try It

🎯 **Goal:** perform Q-learning updates and design rewards that can't be hacked.

🧺 **Preparation:** pages 3.3. α = 0.1, γ = 0.9 unless stated. A calculator.

🔪 **Task:** solve, show each step:

1. Q(s, a) = 0.5, r = 0, best Q(s′) = 0.8. Target? TD error? New Q?
2. Φ(s) = −distance. Shaping bonus for a step from distance 5 to 4? From 4 to 5?
3. A cleaning robot gets +1 per item put in the bin. Describe a hack and a fix.
4. Sparse or dense? (a) +1 for winning chess (b) −0.01 per step plus progress bonus (c) +100 for landing a rocket, 0 otherwise.

👀 **Observation:** show the target **before** the error, and the error before the new Q.

💭 **Reflection:** why did Q go UP in (1) even though the reward was 0?

Mama's Feedback

1. Target = 0 + 0.9 × 0.8 = **0.72**; error = **0.22**; new Q = 0.5 + 0.022 = **0.522**.
2. 0.9 × (−4) − (−5) = **+1.4** forwards; 0.9 × (−5) − (−4) = **−0.5** backwards.
3. Hack: take an item out and bin it again for endless +1. Fix: reward less rubbish on the floor (or the amount in the bin at the end).
4. (a) sparse (b) dense (c) sparse.

Reflection: the promise of a good next state (0.8) is enough to raise Q.

3.4 New Stall or Favourite Stall?

Predict

You're in a city with ten noodle stalls. You've tried three, and one was good. Tonight: go back (safe) or try a new one (risky)? Does your answer change if you're leaving the city tomorrow?

Why does this matter?

An RL agent only learns about actions it **tries**. If it never tries new things, it can never discover better ones; if it always experiments, it never enjoys what it learned. Balancing this is central to every RL system.

Explain

Exploitation = choosing the action currently believed to be best.
Exploration = trying actions whose value is uncertain, to learn more.
The **cost** of exploring is the reward you could have earned by exploiting; the **benefit** is information that raises future rewards.

The simplest version is the **multi-armed bandit**: a row of slot machines ("arms"), each paying out with an unknown probability. Each step you pick one arm. Keep an estimate Q per arm, updated as a running average: **Q ← Q + (reward − Q) ÷ n**, where n is how often that arm was pulled.

Kitchen Analogy

Only exploiting: you may never find the best stall. Only exploring: you never enjoy a good dinner.

Where the analogy breaks down

You remember stalls vaguely; the agent keeps **exact** averages and counts.

3.4 Strategies for Exploring

Strategy	Rule	Strength	Weakness
Greedy	Always pick the highest Q	No wasted pulls	Can lock onto a poor arm forever
ε-greedy	With probability ε pick a random arm	Simple; never stops exploring	Keeps exploring even when it knows
Decaying ε	Start large, shrink: $\varepsilon = \max(0.05, 0.99^t)$	Explore early, exploit later	Schedule must be tuned
UCB	Pick largest $Q + \sqrt{(2 \ln t \div n)}$	Explores what it's least sure about	Needs counts; bandit setting

Table 3.6 · Exploration strategies.

👀 **How to read this:** go down the table: each strategy fixes the weakness of the one above it.

How it works, step by step

How much does ε-greedy give up? Arms pay 0.2, 0.5, 0.8; the agent already knows 0.8 is best; ε = 0.1.

1. Exploit with probability 0.9: 0.9 × 0.8 = **0.72**.
2. Explore with 0.1: average arm (0.2 + 0.5 + 0.8) ÷ 3 = 0.5 → 0.1 × 0.5 = **0.05**.
3. Total **0.77** vs 0.8: exploring costs 0.03 per pull **forever**, which is why ε is often decayed.

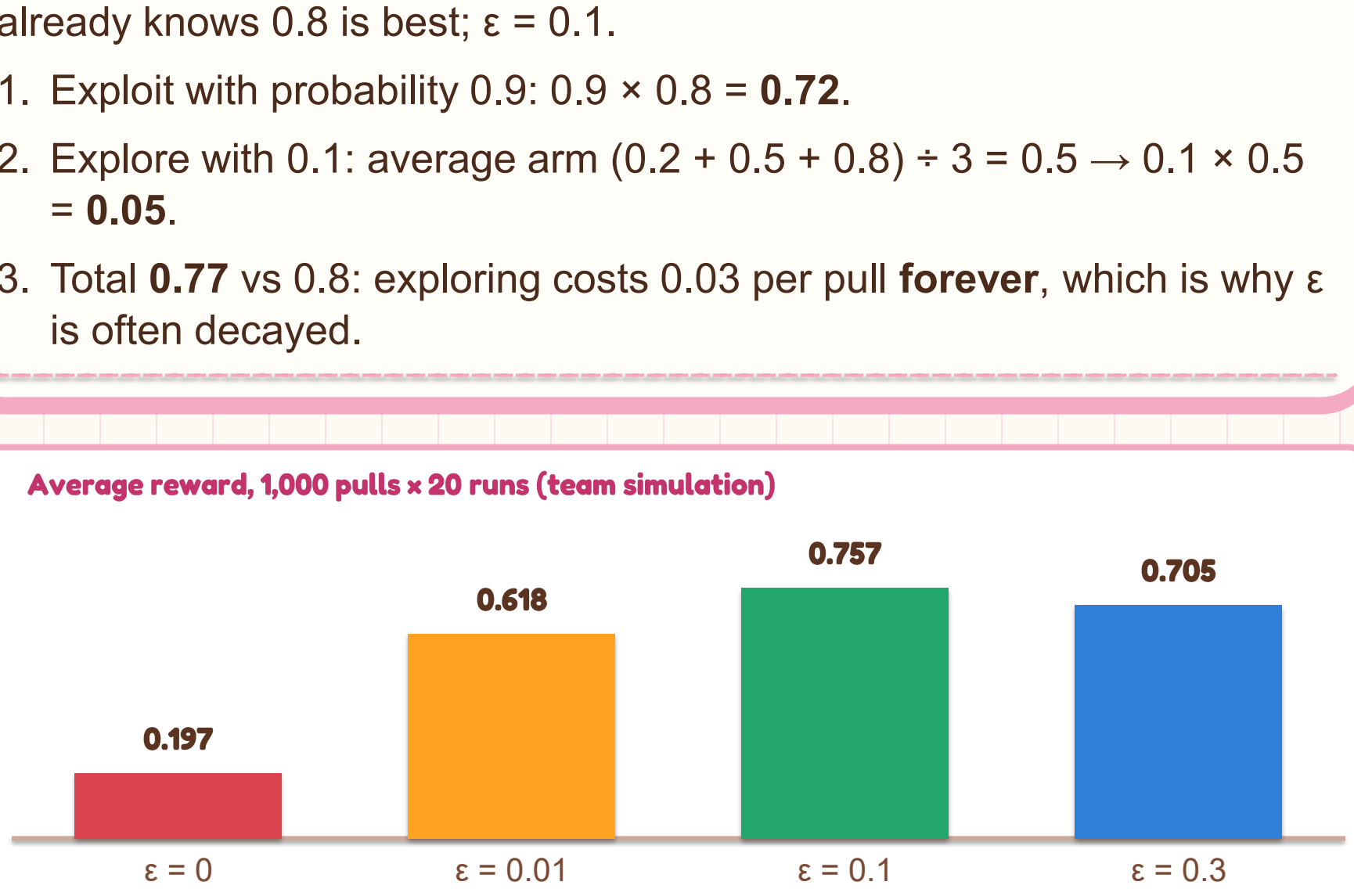


Figure 3.3 · Too little and too much exploration both lose.

👀 **How to read this:** ε = 0 never leaves the first arm (0.2); ε = 0.1 wins (close to the 0.77 limit); ε = 0.3 wastes pulls.

3.4 Smart Curiosity: UCB & Safety

How it works, step by step

UCB (upper confidence bound) adds a **curiosity bonus** that is big for rarely tried arms: **score = Q + √(2 × ln t ÷ n)** (t = total pulls, n = pulls of this arm).

After t = 100: arm A has Q = 0.6 from n = 60; arm B has Q = 0.5 from n = 10.

A: 0.6 + √(2 × 4.605 ÷ 60) = **0.992** · B: 0.5 + √(2 × 4.605 ÷ 10) = **1.46**

UCB picks **B**, even though its average is lower, because it has been tried so little it might still be better. As B is tried more, its bonus shrinks.

Explain

With states (corridors, robots) the same tools carry over: ε-greedy on Q-values with decaying ε, noise added to actions, novelty ("curiosity") bonuses, and entropy bonuses that keep a policy random until it has reason not to be.

⚠ Chef's Warning

"Just let the real robot explore!" **Not quite!** On a real robot a bad exploratory move means a **fall** or a broken part. So explore in **simulation** first and **constrain** exploration (torque limits, safety stops). Simulators run thousands of virtual robots in parallel on one GPU, turning a **time** cost into a **hardware** cost (Recipe 01).

Try It: Noodle Stall Gamble

Try It

🎯 **Goal:** compute ε-greedy rewards, UCB scores and a decay schedule.

🧺 **Preparation:** pages 3.4. Expected reward = (1−ε) × best + ε × average of all arms. UCB = Q + √(2 ln t ÷ n). ln 200 ≈ 5.298. Optional: 3 paper cups with hidden coins to play a real bandit.

🔪 **Task:** solve, then play:

1. Arms pay 0.1, 0.4, 0.7; the agent knows the best; ε = 0.2. Average reward per pull? How far below the best arm?
2. t = 200: A has Q = 0.7, n = 150; B has Q = 0.6, n = 50. UCB scores? Which is chosen?
3. Why can a pure greedy agent get stuck? Use Figure 3.3.
4. ε = max(0.05, 0.99^t). ε at t = 100 and t = 300?

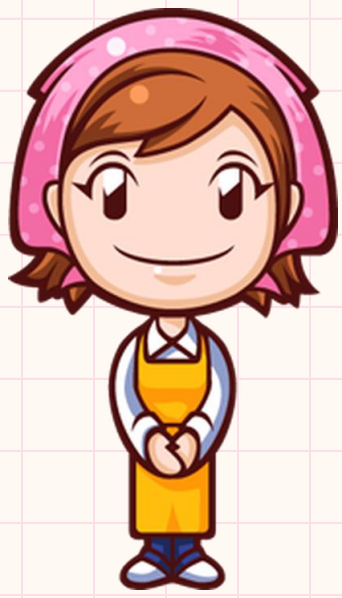
👀 **Observation:** for (2), write 2 ln t first, then divide by each n.

☁️ **Reflection:** with only 5 pulls left in a game, should you explore more or less? Why?

Mama's Feedback

1. 0.8 × 0.7 + 0.2 × 0.4 = 0.56 + 0.08 = **0.64**, which is **0.06** below 0.7.
2. A: 0.7 + √(10.596 ÷ 150) ≈ **0.966**; B: 0.6 + √(10.596 ÷ 50) ≈ **1.060** → **B**.
3. All estimates start equal; it picks arm 0 and that estimate stays ≥ the untried ones, so it never learns another arm pays more (~0.2 reward, best arm 0%).
4. **0.366** and **0.05** (0.99^{300} ≈ 0.049 < 0.05).

Reflection: explore **less**: little future is left to profit from new information.



3.5 Case Study: Robot Dogs

Why does this matter?

Four-legged robots are used for inspection, research and exploration. Walking is hard to program by hand: the robot is unstable, the ground varies, foot contact is brief and violent, and small errors cause falls. Over the last decade, RL policies have walked, run, recovered from falls and crossed rough ground on real quadrupeds (Hwangbo et al., 2019; Lee et al., 2020).

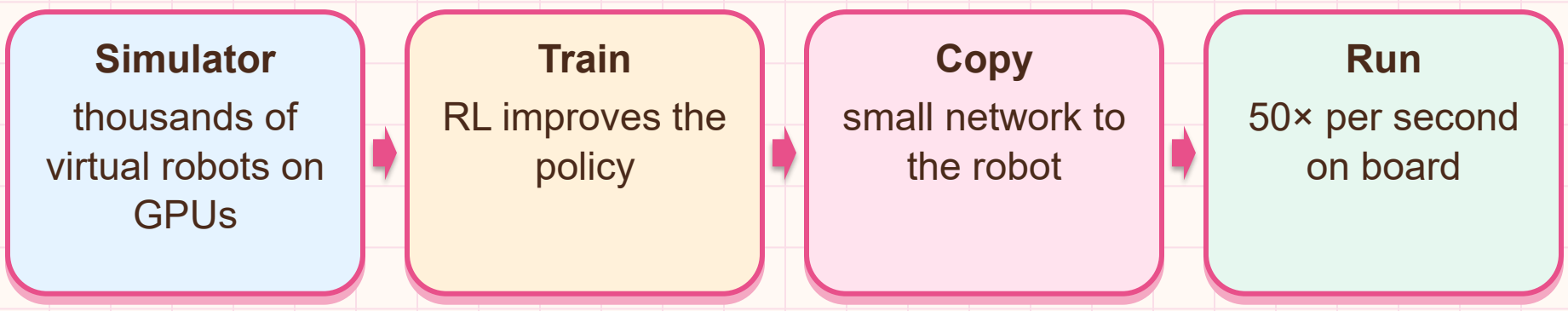


Figure 3.4 · The robot-dog learning pipeline.

👀 **How to read this:** the first two boxes happen in a data centre (Recipe 01's GPUs); the last two on the robot (3.2's tiny policy). Optionally the robot fine-tunes with real experience.

RL part	For the robot dog
Agent	The control policy (a small neural network)
Environment	The robot body, the ground, gravity: simulated or real
State	12 joint angles + 12 joint speeds + 3 tilt + 3 rotation speeds + previous 12 actions + 3 velocity commands = **45 numbers**
Action	A target angle for each of the **12 joints** (a low-level controller drives the motors)
Reward	A formula scoring each step (next page)

Table 3.7 · The robot dog as an RL problem.

👀 **How to read this:** each row maps one word from 3.2 onto the real robot. The state comes from sensors (joint encoders, an IMU): Recipe 04's topic.

3.5 You Design the Reward!

Explain

The reward combines three terms, each with a purpose:

- **Velocity tracking** exp(−(v_cmd − v)² ÷ 0.25): 1 when speed matches the command, falling as the error grows. **Dense**: a useful signal every step.
- **Energy penalty** −0.0001 × Σ torque²: don't waste power or strain motors.
- **Action-rate penalty** −0.01 × Σ (Δaction)²: avoid jerky, vibrating motion.

Scoring three behaviours

```
v_cmd = 1.0 m/s
Walking well:  v = 0.8, torque^2 = 400, d^2 = 2.0 -> 0.8521 - 0.04 -
0.02 = 0.7921
Walking slow:  v = 0.5, same effort                -> 0.3679 - 0.06
= 0.3079
Standing still, no effort                          -> exp(-4)
```

Explain

The reward ranks the behaviours exactly as we want, so maximising it pushes the robot towards walking well. Note the velocity **command** is part of the state: "go forward at 1 m/s", and **one** policy works out how.

Chef's Warning

"Just make the energy penalty huge to save battery!" **Not quite!** Then **standing still** becomes the best policy. Too small, and the gait becomes jerky and damages motors. Engineers tune the weights over many runs and always **watch** the robot.

3.5 Thousands of Virtual Dogs

How it works, step by step

Suppose walking needs **100 million** steps of experience (illustrative).

1. One real robot at 50 steps/s: 100,000,000 ÷ 50 = 2,000,000 s ≈ **23.1 days** (plus falls, resets and wear).
2. 4,096 simulated robots at 50 steps/s each on one GPU: 204,800 steps/s → ≈ 488 s ≈ **8.1 minutes**.

Explain

This is Recipe 01's parallelism: the same small computation on thousands of independent robots. Research systems report learning to walk in minutes on one GPU (Rudin et al., 2022) with GPU simulators such as Isaac Gym. Reality falls short of the ideal 4,096×: Amdahl's serial part and multi-GPU communication (Recipe 02).

Explain

The ==sim-to-real gap==: a simulator is never exactly right (slipperier floor, weaker motors, heavier body), so a policy trained only in simulation can fail on the real robot. Remedies:

- **Domain randomisation**: randomly vary friction, mass, motor strength, delays and sensor noise every run, so the real robot looks like just another variation (Tobin et al., 2017).
- **Better actuator models** learned from real motor data (Hwangbo et al., 2019).
- **Learning on real hardware** with safety measures and efficient algorithms (e.g. Smith et al., 2022: walking in about 20 minutes).

3.5 Thumbs Up or Down?

👍 Advantages

One policy walks, runs, turns, recovers

Robust to unexpected terrain

Change the reward and retrain

Tiny policy runs on board

Learns to recover from falls

👎 Limitations

Reward design needs expertise

Sim-to-real gap

Safety and wear on real hardware

Lots of GPU time to train

Hard to explain decisions (certification)

🔬 Real Tech

Classic model-based control is more predictable and still widely used; modern robots often **combine** both.

✅ Check

1. Compute the reward when the command is 1.0 m/s, the robot moves at 0.5 m/s, Σ torque² = 400, Σ (Δaction)² = 2.0.
2. Why can't the 50 Hz policy run on a cloud server over 5G? Use Recipe 02's numbers.
3. Give two reasons the real speed-up from 4,096 simulated robots is below 4,096×.
4. One benefit and one risk of domain randomisation.

⭐ Explained answers

1. exp(−1) ≈ 0.3679 − 0.06 = **0.3079**.
2. 50 Hz = **20 ms** per step; best remote path (5G + MEC) ≈ 36 ms, cloud 56–140 ms: too late. The ~46 KiB policy runs easily on the robot.
3. Any two: non-parallel parts of each step (Amdahl), memory-bandwidth limits, communication between GPUs, robots not finishing together.
4. Benefit: robust to simulator–reality differences. Risk: too much makes the task needlessly hard or over-cautious.

Kitchen Lab: GridWorld Live

Kitchen Lab

🎯 **Goal:** see Q-learning (TD learning) and exploration happen live.

🧺 **Preparation:** REINFORCEjs GridWorld TD demo by Andrej Karpathy: a static web page, no account or install (Chrome or Edge).

🔪 **Task:** open cs.stanford.edu/people/karpathy/reinforcejs/gridworld_td.html. Press **Toggle TD Learning** and watch the agent learn. Then change **epsilon** (exploration) to 0 and to 0.5, and change the learning rate **alpha**; press reset after each change.

Observe

How many episodes until the arrows point to the goal? What happens with epsilon = 0? With epsilon = 0.5?

💭 **Reflection:** which setting behaves like the greedy agent in Figure 3.3?

Mama's Feedback

Default settings: the agent wanders at first; after a short while (often under a minute of fast learning) the arrows on the main route point to the +1 goal and avoid the red −1 squares. The exact count varies because moves are random.

ε = 0: repeats the first route that worked; some squares are never tried and it can get stuck. **ε = 0.5:** values everywhere get learned, but the path is messy and hits red squares more often. **Reflect:** ε = 0 is the greedy bandit agent.

No internet? Fallback

Draw a 4×4 grid on paper, goal in a corner (+1), one pit (−1). Play the agent with a coin: heads = explore (random move), tails = exploit (best arrow). After each move update the square: value ← value + 0.5 × (reward + 0.9 × next value − value).

📺 Watch: Curated Videos

📺 Video 1 · for 3.2–3.3

Reinforcement Learning: Machine Learning Meets Control Theory · Steve Brunton · 26:00 · youtube.com/watch?v=0MNVhXEX9to

Why: a clear lecture on agent, environment, reward, MDPs and Q-learning (3.2–3.3).

Which part: 3:34 RL overview · 12:32 Markov decision process · 13:33 credit assignment · 21:50 Q-learning.

Do: at 21:50, compare his Q-update with the 0.2 → 0.325 worked example.

Watch out: university level; some equations go beyond this guide.

📺 Video 2 · for 3.2

Markov Decision Processes · Computerphile · 17:40 · youtube.com/watch?v=2iF9PRriA7w

Why: friendly whiteboard explanation of states, actions, rewards and policies.

Do: write the S, A, R and policy for "the pup fetching an ingredient".

Watch out: talk format, no code.

📺 Video 3 · for 3.3–3.5

AI Learns to Walk · AI Warehouse · 8:36 · youtube.com/watch?v=L_4BPjLBF4E

Why: a simulated creature goes from falling to walking: rewards and shaping in action (3.5). A fun one!

Do: note one moment where the agent "hacks" the reward and suggest a better reward.

Watch out: entertainment style; simulation only (no sim-to-real).

🎯 Final Round: Recipe 03

✅ Check

Task 1. A corridor has six squares (0–5), goal at 5, +1 for entering it, policy "always move right". V(0) with γ = 0.9? With γ = 0.5?

Task 2. A drone controller must run at 100 Hz. Time budget per step? Can it use a policy served from a 5G edge server with 36 ms delay?

Task 3. A policy network is 20 → 64 → 64 → 6. MACs per decision? Weight bytes in FP16?

⭐ Explained answers

Q1: V(4) = 1, then × γ per step back: V(0) = 0.9^4 ≈ **0.656**; with γ = 0.5: 0.5^4 = **0.0625**.

Q2: 100 Hz → **10 ms** per step; 36 ms is far too late: the policy must run on the drone.

Q3: 20 × 64 + 64 × 64 + 64 × 6 = 1,280 + 4,096 + 384 = **5,760 MACs**; × 2 bytes = **11,520 bytes** (≈ 11.25 KiB).

Stamp: Gold / Silver / Bronze, as before.

📌 Recipe Card 03

📌 Recap

- Behaviourist AI learns behaviour from **consequences**; RL is its maths.
- Everything is the loop: **state** → **action** → **reward** → **new state**, guided by a **policy**.
- **Return** and **value** carry delayed rewards back to earlier actions (credit assignment).
- **Q-learning** nudges estimates towards a target by a fraction α of the TD error.
- The reward defines the goal: design it carefully or the agent will **hack** it.
- Balance **exploration and exploitation**; explore in simulation when mistakes are costly.

📖 Key words (see Glossary)

agent · environment · state · action · reward · policy · episode · return · discount γ · value · Q-value · Bellman · TD error · Q-learning · shaping · reward hacking · ε-greedy · UCB · sim-to-real · domain randomisation

🎉 **Unlocked: Recipe Book page 3!**

🍳 **Recipe 03 · complete!**

RECIPE 04

Taste, Smell, See!

AI Sensors & Intelligent Systems

Mama's Problem

A good chef **sees** the colour of the onions, **hears** the sizzle, **smells** the burning and **feels** the dough. A self-driving car must do the same with cameras, lasers and sound. How does a machine sense the world, combine its senses, and act **in time**?

Today's route

4.1 Types of sensors: camera, LiDAR, ultrasonic (and radar)

4.2 Data acquisition and real-time processing

4.3 Sensor fusion and environmental perception

4.4 Integrating hardware with AI decision-making

4.5 Case study: Tesla's camera-only driving

The scene for this recipe

Running example: a car drives at 50 km/h at dusk, just after rain. A ball rolls out between two parked cars, and a child may follow. The system must notice, judge the distance, decide to brake and brake **in time**, in a fraction of a second.

4.1 What Is a Sensor?

Predict

How many sensors does your phone have? Guess a number, then list as many as you can (hint: how does it know it's tilted?).

Why does this matter?

An AI model can only be as good as the data it receives. For a robot or a car, that data comes from sensors, each with its own physics, strengths and blind spots. Choosing and combining them decides what the machine **can** know about the world.

Explain

A **sensor** converts a physical quantity (light, sound, distance, acceleration) into an electrical signal a computer can read.

Sensors are **passive** (they measure energy already there, like a camera using daylight) or **active** (they send energy out and measure what comes back, like LiDAR, ultrasonic and radar). Active sensors work in the dark; passive ones need light.

Every intelligent machine runs the same loop:

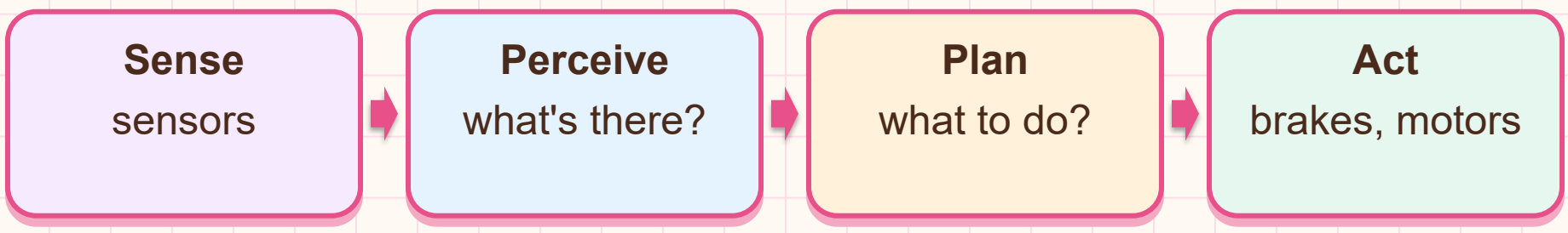


Figure 4.1 · The sense–perceive–plan–act loop.

How to read this: this is the agent–environment loop of Recipe 03, seen from the hardware side. Each arrow takes time.

Kitchen Analogy

A chef's eyes are **passive** (kitchen light). Knocking on a melon and listening is **active**: you make a sound and judge the echo.

Where the analogy breaks down

Chefs understand what they sense; a sensor only produces **numbers** that a network must interpret.

4.1 The Camera: Eyes Full of Pixels

Explain

A **camera** is **passive**. A lens focuses light onto an **image sensor**, a grid of light-sensitive **pixels**; a colour filter gives red, green and blue values. Each frame is millions of numbers.

Good at: very high resolution and colour: it reads signs, lane lines, traffic-light colours and brake lights. Cheap, small, no moving parts.

Bad at: it does **not** measure distance directly (a neural network must infer it). It needs light, so darkness, glare and fast brightness changes hurt it, as do rain, fog and a dirty lens.

How it works, step by step

Why range and resolution are linked. A camera has 1,920 pixels across and a 60° field of view (FOV). How wide does a 0.5 m child look?

1. Focal length in pixels: f = (1,920 ÷ 2) ÷ tan(30°) = 960 ÷ 0.577 ≈ **1,663 px**.
2. Width in the image = f × real width ÷ distance.
3. At 50 m: 1,663 × 0.5 ÷ 50 ≈ **16.6 px**. At 150 m: ≈ **5.5 px**.

Explain

Cause and effect: at 150 m the child is about five pixels wide: too few for a network to classify reliably. Seeing **further** needs more pixels or a narrower (telephoto) lens, which in turn sees less to the sides. That is why cars carry several cameras with different lenses.

4.1 LiDAR: A Tape Measure of Light

Explain

LiDAR (Light Detection and Ranging) is **active**. It fires very short laser pulses and times the echo: this is **time of flight**. Light travels at $c \approx 3 \times 10^8$ m/s, so:

distance = c × t ÷ 2 (÷ 2 because the pulse goes out **and** back)

Sweeping the laser across the scene builds a **point cloud**: many (x, y, z) points plus intensity, a 3-D sketch of the world.

Worked example

```
Pulse returns after 400 ns:
d = (3 x 10^8 x 4 x 10^-7) / 2 = 120 / 2 = 60 m
```

Good at

Direct, accurate distance (a few cm)

Works in darkness

3-D shape

Spots unusual objects a camera may not recognise

Weak at

No colour: can't read signs or lights

Rain, fog, snow, dust scatter it

Dark or shiny surfaces

Costlier (but falling)

Sparse far away

Real Tech

Data rate example: 64 beams, a 0.4° horizontal step and 10 sweeps per second → 360 ÷ 0.4 = 900 steps × 64 = 57,600 points per frame ≈ 0.92 MB, so about **9.2 MB/s**. LiDAR makes far **less** data than cameras.

4.1 Ultrasonic & Radar

Explain

An **ultrasonic sensor** sends a burst of sound above human hearing (about 40 kHz) and listens for the echo: like LiDAR, but with sound, about a million times slower. **distance = v × t ÷ 2**, v ≈ 343 m/s at 20 °C. The speed of sound depends on temperature: v ≈ 331.3 + 0.606 × T (°C).

Good at: very cheap, works in the dark, finds low obstacles: **parking**.
Bad at: short range (a few m), wide beam, can't tell what the object is, too slow for fast objects.

Worked example

```
Echo after 5.8 ms: 343 x 0.0058 / 2 = 0.99 m
Freezing day (0 C, v = 331.3): real 1.00 m -> echo 6.04 ms
Sensor assumes 343 -> reports 1.035 m  (3.5% error)
```

Explain

Radar emits radio waves (cars use 76–81 GHz) and measures the echo. Its superpower is **speed**: a moving object shifts the echo's frequency (the **Doppler effect**), so radar measures velocity **directly**, and keeps working in rain, fog and darkness. Weakness: coarse angular resolution, so it struggles to separate nearby objects or identify shapes.

$v = (\Delta f \times c) \div (2 \times f_0)$. Example: f_0 = 77 GHz, Δf = 10.27 kHz → v ≈ **20 m/s** (72 km/h).

Kitchen Analogy

Doppler is why an ambulance siren sounds higher coming towards you and lower driving away.

Where the analogy breaks down

You **hear** the pitch change; radar **calculates** it from tiny frequency shifts in billionths of a second.

4.1 Sensor Taste Test

	Camera	LiDAR	Ultrasonic	Radar
Principle	Light (passive)	Laser time of flight	Sound echo	Radio echo + Doppler
Measures	Colour image	3-D point cloud	Short distance	Range + speed
Typical range	Tens–250+ m	~100–250 m	~0.2–5 m	~5–250 m
Direct depth?	No (inferred)	Yes, accurate	Yes, short	Yes
Direct speed?	No	No	No	Yes
Darkness	Poor	Yes	Yes	Yes
Rain / fog	Degraded	Degraded	Mostly OK	Least affected
Reads signs?	Yes	No	No	No
Cost	Low	Medium–high	Very low	Low–medium

Table 4.1 · Automotive sensors compared (typical values).

👀 **How to read this:** read a column for one sensor's personality, or a row to find who wins in one condition. No column wins every row: that is why we fuse (4.3).

⚠️ **Chef's Warning**

"The table says LiDAR reaches 250 m, so mine does!" **Not quite!** Table numbers are **typical**. A specific product can be far better or worse. Always check the datasheet of the sensor you actually use.

Try It: Taste Test Challenge

Try It

🎯 **Goal:** calculate basic sensor quantities and choose the right sensor for a job.

🧺 **Preparation:** pages 4.1. $f = (W/2) \div \tan(FOV/2)$; width = f × size ÷ d; ToF $d = c \cdot t \div 2$; echo $d = v \cdot t \div 2$; Doppler $v = \Delta f \cdot c \div (2 \cdot f_0)$. A calculator.

🔪 **Task:** solve, then choose:

1. A 3,840-px camera with a 90° FOV: how many pixels wide is a 1.8 m car at 100 m?
2. A LiDAR pulse returns after 1.0 µs. Distance?
3. An ultrasonic echo returns after 11.7 ms. Distance?
4. A 77 GHz radar sees a 15.4 kHz shift. Speed?
5. Which sensor for: (a) reversing with 30 cm clearance (b) a speed-limit sign (c) a stalled car in dense fog (d) the 3-D shape of an odd object at night?

👀 **Observation:** write the formula first, then the numbers, then the unit.

☁️ **Reflection:** why did no sensor win every row of Table 4.1?

Mama's Feedback

1. $f = 1{,}920 \div \tan 45° = 1{,}920$ px → $1{,}920 \times 1.8 \div 100 \approx$ **34.6 px**. 2. **150 m**. 3. $343 \times 0.0117 \div 2 \approx$ **2.0 m**. 4. **30 m/s** (108 km/h).
5. (a) ultrasonic (b) camera (c) radar (careful: stationary clutter) (d) LiDAR.

Reflection: each sensor measures a different physical quantity and fails in different conditions; that is why we fuse them.

4.2 From Farm to Wok: Acquisition

Predict

One car camera at 36 frames per second: guess how many **megabytes per second** it makes. 1? 10? 100?

Why does this matter?

A car must turn photons into a braking decision in about a tenth of a second, continuously, and **can't skip a frame**. Every stage between the sensor and the decision costs time and moves data, and Recipe 01 taught us moving data is expensive.

How it works, step by step

The **acquisition chain**, step by step:

1. **Sensing element** turns light/sound into an analogue voltage.
2. **ADC** (analogue-to-digital converter) turns it into numbers.
3. **Pre-processing:** an **ISP** (image signal processor) fixes colour and noise, or a filter cleans the signal.
4. **Timestamp** and **synchronise**: every reading is tagged with when it was taken.
5. **Transfer** over a bus or automotive Ethernet to **memory** (DRAM/SRAM).
6. The **neural network** runs on a CPU/GPU/NPU.

Explain

Why timestamps matter: a car at 30 m/s moves **0.45 m** in 15 ms. If camera and LiDAR clocks are 15 ms apart, their views of the same object disagree by almost half a metre.

Kitchen Analogy

Farm to wok: picked, washed, date-labelled, trucked, stored, cooked. A delay anywhere means a late dinner.

4.2 How Much Data? Can We Keep Up?

Data rate

```
8 cameras, 1,920 x 1,080 px, 12-bit raw, 36 fps
1. frame      = 1,920 x 1,080 x 12 / 8 = 3,110,400 bytes
2. per camera = 3,110,400 x 36         ~ 112 MB/s
3. all 8      ~ 0.9 GB/s
```

Explain

One camera (112 MB/s) makes about **three times** the data of a 128-beam LiDAR (~37 MB/s). Cameras dominate the data volume, so (Recipe 01) systems keep raw images on the chip or compress early.

Compute needed

```
100 GFLOP per frame (illustrative), 8 cameras, 36 fps
sustained compute = 100 x 10^9 x 8 x 36 = 28.8 TFLOP/s
at 40% utilisation -> PEAK needed = 28.8 / 0.4 = 72 TFLOP/s
```

Explain

Recipe 01's 10 TFLOP/s accelerator is far too small. That is why self-driving computers carry **hundreds of TOPS**. Face ID was one image at batch 1; a car makes hundreds of inferences per second, every second. Recipe 01's lessons (utilisation, memory-bound vs compute-bound) decide whether the system works at all.

4.2 Every Millisecond Is Distance

Explain

A system is **real-time** when a **late** answer is a **wrong** answer. While the car "thinks", it keeps moving: **distance driven blind = speed × latency**. At 50 km/h (13.9 m/s), 100 ms is 1.4 m.

Stage	Time (ms)
Exposure and readout	10
ISP	5
Transfer to memory	2
Neural-network inference	**30**
Fusion and tracking	10
Planning	15
Actuation (brake build-up)	30
Total	**102**

Table 4.2 · An illustrative latency budget.

👀 **How to read this:** add the column: with a 100 ms target the system is already 2 ms over. Fix the biggest slices first: inference (Recipe 01's tricks) and actuation.

⚠ Chef's Warning

"A faster chip fixes latency." **Not quite!** Only the inference slice depends on the chip. Exposure, transfer and brake build-up don't, and an operation that falls back to the CPU (Recipe 01) can silently blow the whole budget.

Try It: Beat the Timer

Try It

🎯 **Goal:** calculate data rates, stopping distances, latency budgets and compute needs.

🧺 **Preparation:** pages 4.2. Frame bytes = w × h × bits ÷ 8. Stopping = v × latency + v² ÷ (2a). Compute = FLOP/frame × cameras × fps; peak = compute ÷ utilisation. Convert km/h ÷ 3.6.

🔪 **Task:** solve each:

1. Six cameras, 1,280 × 960, 10-bit, 30 fps. Data rate per camera and in total?
2. 72 km/h, braking 5 m/s², latency 150 ms. Total stopping distance?
3. In Table 4.2, inference grows from 30 to 60 ms. New total? Miss by how much? Two fixes?
4. 12 cameras at 30 fps, 60 GFLOP network. Sustained TFLOP/s? Peak at 30% utilisation?

👀 **Observation:** keep units visible at every step (m/s, s, bytes).

☁️ **Reflection:** which matters more for safety: throughput or latency? Why?

Mama's Feedback

1. 1,536,000 bytes/frame → **46.08 MB/s** per camera; **276.48 MB/s** total.
2. 20 m/s: 20 × 0.15 = 3.0 m + 20² ÷ 10 = 40 m → **43.0 m**.
3. **132 ms**, misses by **32 ms**. Fixes: INT8 quantisation, smaller/pruned network, lower resolution, fused operators, no CPU fall-back, pipelining.
4. **21.6 TFLOP/s** sustained; **72 TFLOP/s** peak.

Reflection: latency: every millisecond is distance driven blind.

4.3 Why Mix Senses? Fusion

Why does this matter?

No single sensor sees everything (Table 4.1). Fusion lets one sensor's strength cover another's weakness, and tells the planner **how uncertain** the picture is.

Explain

Sensor fusion combines measurements from several sensors into one estimate. In our dusk scene, the camera sees the ball's colour but struggles with low light and wet reflections; LiDAR or radar give the distance regardless of light. Together they beat either alone.

A popular shared space is a **bird's-eye view** (BEV): a top-down map around the vehicle. Putting every sensor's features into the same BEV grid makes them natural to combine.

Level	What is fused	Advantage	Disadvantage
Early (raw)	Raw pixels and points	Keeps all information	Needs precise calibration; heavy data
Mid-level (feature)	Features from each network, merged in BEV	Learns how to combine; strong accuracy	Joint training; harder to debug
Late (object)	Each sensor's object list, merged by a tracker	Simple, modular; drop a failed sensor	Information thrown away before fusion

Table 4.3 · Three levels of sensor fusion.

👀 **How to read this:** moving down the table, fusion happens later: simpler and more modular, but with less information left.

Kitchen Analogy

Early = blending raw ingredients into one batter. Mid = combining each cook's sauces. Late = each cook plates a dish and Mama picks the menu.

Where the analogy breaks down

Cooks can taste and argue; fusion follows fixed maths or a trained network, with no judgement of its own.

4.3 Trust the Surer Sensor

Explain

If a sensor's error has standard deviation σ, its **variance** is σ², and the best weight is **1 ÷ σ²** (**inverse-variance weighting**). A more precise sensor deserves more weight:

fused = Σ(w_i × z_i) ÷ Σw_i · fused σ = √(1 ÷ Σw_i)

Radar + camera

```
Radar:  50.0 m, sigma 0.5 -> weight 1 / 0.25 = 4
Camera: 52.0 m, sigma 2.0 -> weight 1 / 4    = 0.25
fused = (4 x 50.0 + 0.25 x 52.0) / 4.25 = 213 / 4.25 = 50.12 m
fused variance = 1 / 4.25 = 0.235 -> sigma = 0.49 m
```

Explain

The fused answer sits close to the precise radar, and its σ (0.49 m) is slightly **better** than the best sensor alone (0.5 m). Adding information never hurts, **if** the errors are independent.

Chef's Warning

"Fusion always helps!" **Not quite!** The formula assumes independent, unbiased, bell-shaped errors. If both sensors share a flaw (both blinded by fog, or a calibration error), the fused result is **less** trustworthy than the formula claims.

4.3 Fusion Over Time: Kalman Filter

Explain

Measurements keep arriving and objects move. The **Kalman filter** (Kalman, 1960) keeps a running estimate x **and** its uncertainty P (a variance), alternating two steps:

How it works, step by step

1. **Predict:** move the estimate forward with a motion model. Uncertainty **grows** by the process noise Q, because the model is imperfect: $P \leftarrow P + Q$.
2. **Update:** a measurement z arrives with noise variance R. The **Kalman gain** $K = P \div (P + R)$ decides how much to trust it. Then $x \leftarrow x + K(z - x)$ and $P \leftarrow (1 - K)P$.

Big P (unsure prediction) or small R (precise sensor) → K near 1 → trust the measurement.

Start	Predict	Radar	Camera
x = 50.0	Q = 1	z = 52, R = 1	z = 50.5, R = 4
P = 4.0	P = 5.0	K = 0.83	K = 0.17
		x = 51.67, P = 0.83	x = 51.47, P = 0.69

Figure 4.2 · One predict step and two updates (distance to the lead car).

👀 **How to read this:** follow x and P left to right: the precise radar pulls x strongly; the noisier camera only a little; and P falls with every update.

Real Tech

Real trackers estimate position **and** velocity (a constant-velocity model), but the idea is identical. Trackers also do **association**: deciding which new detection belongs to which tracked object.

4.3 Scoring Perception

Explain

How do we know a perception system is good? Three common scores:

- **IoU** (intersection over union) = overlap ÷ union of the predicted and true boxes.
- **Precision** = correct detections ÷ all detections ("of what I saw, how much was real?").
- **Recall** = correct detections ÷ all real objects ("of what was there, how much did I see?").

Worked example

```
Boxes 4 x 2 m (8 m^2) shifted 1 m: overlap 6, union 8 + 8 - 6 = 10
-> IoU 0.6
100 detections: 90 correct, 10 false alarms; 30 objects missed
precision = 90 / 100 = 0.90     recall = 90 / 120 = 0.75
```

Explain

Cause and effect: false alarms (low precision) cause **phantom braking**; misses (low recall) cause collisions. Tuning one usually worsens the other, so these scores are really a **safety-versus-comfort** trade-off (4.4).

Public datasets with labelled camera, LiDAR and radar data let methods be compared fairly: KITTI (2012), nuScenes (2020), Waymo Open Dataset (2020).

Explain

Depth from cameras gets worse with range. For a stereo pair, depth error ≈ $Z^2 \div (f \cdot B)$: it grows with the **square** of distance. At 50 m one pixel of error can mean metres of depth error, while LiDAR measures to centimetres.

Try It: Fusion Kitchen

Try It

🎯 **Goal:** fuse two measurements, run one Kalman update, and score a detector.

🧺 **Preparation:** pages 4.3 and a calculator. Optional role-play: two "sensors" estimate the distance to a wall by eye and by pacing; a third student fuses them.

🔪 **Task:** solve each:

1. Radar 40.0 m ($\sigma = 0.4$); camera 42.0 m ($\sigma = 1.6$). Fused estimate and σ?
2. Kalman: $x = 20.0$, $P = 2.0$, $Q = 0.5$. Predict, then update with $z = 21.0$, $R = 2.5$. New x and P?
3. 200 detections, 170 correct; 230 real objects. Precision? Recall?

👀 **Observation:** after each calculation, say which input the result moved closer to, and why.

☁️ **Reflection:** when would fusing two sensors make the estimate worse than one?

Mama's Feedback

1. Weights $1 \div 0.16 = 6.25$ and $1 \div 2.56 = 0.39 \rightarrow (250 + 16.4) \div 6.64 \approx$ **40.12 m**; $\sigma = \sqrt{(1 \div 6.64)} \approx$ **0.39 m**.
2. Predict $P = 2.5$. $K = 2.5 \div 5.0 = 0.5 \rightarrow x = 20 + 0.5 \times 1 =$ **20.5**; $P = 0.5 \times 2.5 =$ **1.25**.
3. Precision $170 \div 200 =$ **0.85**; recall $170 \div 230 \approx$ **0.74**.

Reflection: when errors are shared (both sensors fooled by the same fog or a shared calibration error) or one sensor is biased.

4.4 The Autonomy Stack

Why does this matter?

Perceiving the world is only half the job. The machine must predict what others will do, plan, and act, on real chips, under deadlines and uncertainty. This is where Recipes 01–03 meet.

Perception	→	Prediction	→	Planning	→	Control
what is where?		where will it go?		choose a path		steer, brake

Figure 4.3 · The autonomy stack.

How to read this: each box hands its output to the next; each adds latency.

Explain

A **modular** stack keeps each stage separate with defined interfaces: easy to test and to see which stage failed. An **end-to-end** system trains one network from sensors to trajectory (Bojarski et al., 2016 steered from camera images; Hu et al., 2023 trained one network with linked tasks): fewer hand-written rules, but harder to explain. Real systems are often hybrids.

Explain

A vehicle computer is **heterogeneous**: different chips for different work (Recipe 01). CPUs run planning logic and safety checks; GPUs/NPUs run the perception networks; small microcontrollers drive actuators. Batch-1 streaming perception is **memory-bound**, so on-chip memory and low precision matter.

Kitchen Analogy

Modular = a kitchen brigade (one chops, one fries, one plates). End-to-end = one super-chef who does it all by feel.

Where the analogy breaks down

In the brigade it's easy to see who burnt the rice; in a network, finding the "guilty" part is a research problem.

4.4 Can the Car Stop in Time?

How it works, step by step

Car at 50 km/h = 13.89 m/s, total latency 0.15 s, braking 7 m/s².

1. Blind distance: 13.89 × 0.15 ≈ **2.08 m**.
2. Braking distance: 13.89² ÷ (2 × 7) ≈ **13.78 m**.
3. Total needed ≈ **15.9 m**.
4. Child detected at **25 m**: stops with ~9 m to spare. ✓
5. Detected at only **15 m**: after 2.08 m, 12.92 m remain; impact speed = √(13.89² − 2 × 7 × 12.92) ≈ **3.5 m/s** (12.5 km/h). ✗

Detection distance vs stopping distance

Explain

This ties the recipe together: **detection range** (4.1), **latency** (4.2) and the **quality of the fused estimate** (4.3) decide whether a decision can be carried out at all. Even 3.5 m/s is dangerous for a child: early, reliable detection beats any amount of late braking.

4.4 Deciding Under Uncertainty

Explain

Perception outputs **probabilities**, not certainties. The planner must act anyway. One principled way: compare **expected costs**.

How it works, step by step

Cost of an unnecessary hard brake (false alarm) = 1; cost of failing to brake for a real pedestrian = 1,000. Let p = probability the object is a pedestrian.

1. Expected cost of braking = (1 − p) × 1.
2. Expected cost of not braking = p × 1,000.
3. Brake when (1 − p) < 1,000p, i.e. when **p > 1 ÷ 1,001 ≈ 0.1%**.

Explain

The threshold is tiny because the errors are so unequal. But braking at 0.1% confidence means braking often for shadows and plastic bags: **phantom braking**, which is irritating and can itself cause rear-end crashes. Real thresholds are tuned with data, simulation and road tests. A cautious planner also uses the fused uncertainty: with x = 51.47 m and σ ≈ 0.83 m it may plan for 51.47 − 2 × 0.83 ≈ **49.8 m**.

Explain

Safety standards: SAE J3016 defines the automation levels; assists needing constant supervision are **Level 2**. ISO 26262 covers **faults** (ASIL A–D, D strictest); **SOTIF** (ISO 21448) covers hazards when **nothing is broken** but a sensor hits its limits (glare).

4.4 Two Cooks Are Safer... If Different

Explain

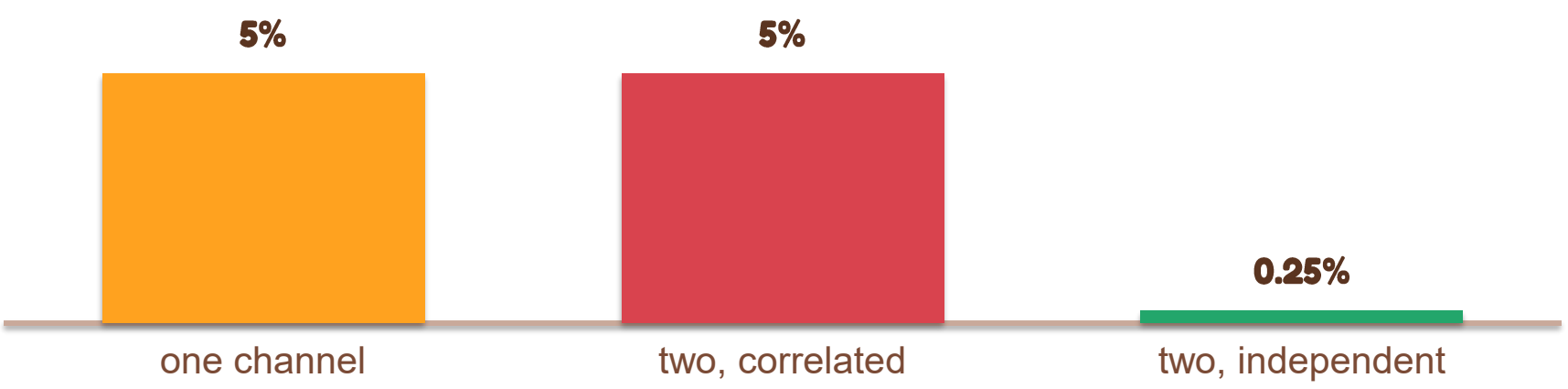
Safety-critical systems duplicate parts: two computers, two power supplies, and **diverse** sensors. **Fail-operational**: keeps working in a reduced mode. **Fail-safe**: stops safely. A **minimal-risk manoeuvre**, like pulling over, is the fallback.

How it works, step by step

Why redundancy needs independence. Each of two channels misses a hazard 5% of the time.

1. **Independent** failures: both miss 0.05 × 0.05 = **0.25%**: 20× better than one.
2. **Perfectly correlated** (fog blinds both): both miss **5%**: no better than one.

Real systems fall between. That is the argument for **different physics** (camera, radar, LiDAR) instead of copies of one sensor.



Real Tech

The counter-argument (4.5): one very capable channel plus a great network is simpler, cheaper, and avoids conflicting sensors. Which side is right is an **evidence** question.

4.5 Case Study: Tesla Vision

Why does this matter?

Tesla made the most debated sensor decision in the industry: drive with **cameras and a neural network alone**. The case shows every idea of this recipe at once, and it is a lesson in separating what a company **claims** from what independent evidence **shows**.

Read this first

Checked against public sources in early November 2026. Tesla's software, hardware and regulatory status change quickly; figures from news reports of company statements are marked **reported** or **company claim**.

When	What changed
2016	Eight cameras, a forward radar and twelve ultrasonic sensors
2019	Tesla's own FSD computer (HW3), described by Tesla as about 144 TOPS
2021–22	Radar removed from new cars (Tesla Vision)
Oct 2022	Ultrasonics dropped; a vision "occupancy network" replaces them
2023 →	HW4 computer and new cameras
Early 2024	FSD v12: end-to-end network replaces much hand-written code (reported)
2025–26	Robotaxi service launched in Austin (June 2025) and expanded (reported)

Table 4.4 · Timeline of Tesla's sensor and computing changes.

How to read this: read top to bottom: each step removes a sensor type or adds compute. Rows marked "reported" were not independently verified.

4.5 Pixels In, Steering Out

Explain

Sensors: eight external cameras give 360° (no radar, LiDAR or ultrasonic where Tesla Vision is fully deployed): three forward (wide, main, narrow; narrow quoted at ~250 m), two forward-side, two rear-side, one rear. Resolution reported from ~1.2 MP (HW3) to ~5 MP (HW4). **No directly measured depth anywhere: every distance is inferred.**

How it works, step by step

How the network sees (Tesla AI Days 2021–22):

1. **Multi-camera BEV fusion:** image **features** from all eight cameras fused into one 3-D top-down space: **mid-level fusion** (4.3).
2. **Occupancy networks:** which 3-D cells are occupied, even by unnamed objects (a fallen ladder): LiDAR-like geometry inferred from video.
3. **Video, not single images:** a memory of recent frames estimates speed and sees through brief occlusions.

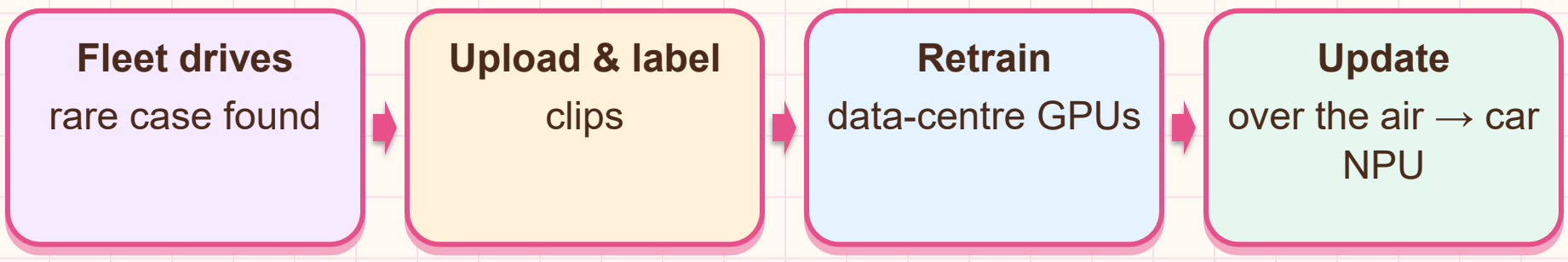


Figure 4.4 · The data engine.

👀 **How to read this:** a loop: mistakes become training data. It is Recipe 01's training-vs-inference split at the scale of millions of vehicles.

Real Tech

The chip (Talpes et al., 2020): each accelerator is a 96 × 96 MAC array, 8-bit, ~2 GHz → 96 × 96 × 2 × 2 × 10^9 ≈ **36.9 TOPS**; four of them ≈ 147.5 TOPS, close to Tesla's 144. But batch-1 streaming is memory-bound: Tesla said (Q1 2026, reported) HW3 has ~1/8 of HW4's memory bandwidth, and on the roofline that matters more than TOPS.

4.5 Why Only Cameras? Both Sides

Arguments for

Humans drive with vision alone

Cameras are cheap and high-resolution

One sensor type: no conflicting readings

Fleet data scales the network

Fewer parts, less wiring

Arguments against

No direct depth: estimates, not measurements

Darkness, glare, fog hurt every camera together (common cause)

One blocked camera removes information nothing else gives

Independent evidence is limited

Explain

Plot twist: Waymo does it differently. Waymo is also **reducing** sensors (from 29 cameras and 5 LiDARs in its previous generation) while **keeping** LiDAR and radar. The industry is not settling on "more is better" but on "enough **diverse** sensors, with better software".

Explain

You're the judge. Tesla claimed (company claim, April 2026) that supervised FSD has several times fewer collisions per mile than average drivers. Ask: **Who measured it? What exposure** (easier roads, good weather)? **Who is driving** (a supervising human)? **What counts** as a collision? **What do regulators find** (NHTSA investigations)?

Honest conclusion: camera-only driving is **possible to demonstrate** and **not yet proven equal** to sensor-rich designs in all conditions.

Chef's Warning

"That's a convincing argument, so it's true!" **Not quite!** A strong argument is not evidence. Only measurements, ideally independent ones, can show which side is right.

✅ Check: Case Study Questions

✅ Check

1. Using stereo depth, why does camera-only find long-range distance harder than LiDAR? How does Tesla compensate?
2. HW3 has 1/8 of HW4's memory bandwidth. Using the roofline, why could this limit performance even with similar TOPS?
3. Recompute TOPS for a 128 × 128 array at 1.5 GHz.
4. Removing ultrasonics: one benefit and one risk.
5. Classify FSD (Supervised) on the SAE scale. What does it mean for driver monitoring?

⭐ Explained answers

1. Depth error grows with Z^2 (≈ 8.3 m per pixel at 50 m); LiDAR measures to cm. Tesla uses learned depth, 8-camera BEV fusion, many frames and fleet data: **estimates**, not measurements.
2. Batch 1 = low intensity = memory-bound slope: performance = intensity × bandwidth, so 1/8 bandwidth ≈ **1/8 performance**.
3. $16{,}384 \times 2 \times 1.5 \times 10^9 \approx$ **49.2 TOPS**.
4. Benefit: cost, wiring, fewer parts. Risk: short-range geometry now inferred; one blocked camera removes information nothing else supplies.
5. **Level 2**: the human supervises at all times; driver monitoring is essential.

Kitchen Lab: Your Phone's Sensors

Kitchen Lab

🎯 **Goal:** feel real sensor data: noise, sampling rate, delay.

🧺 **Preparation:** install **phyphox** (RWTH Aachen University, free, no account, Android or iOS; version 1.2 used) from phyphox.org.

🔪 **Task:** open "Acceleration (without g)" and hold the phone still on a table for 10 s; then walk with it. Open "Sonar" (or "Acoustic Stopwatch") near a wall at 1 m and read the distance; clap twice for the stopwatch.

Observe

Holding still, is the acceleration exactly zero? How far off is the sonar at 1 m?

💭 **Reflection:** which ideas from 4.1–4.3 did you see (noise σ, temperature-dependent sound speed, sampling rate)?

Mama's Feedback

Phone still: acceleration jitters around 0 (about ±0.05 m/s²), never exactly zero: that jitter **is** sensor noise σ. **Sonar at 1 m:** usually within a few cm; more if the room temperature differs from what the app assumes (sound speed changes ≈ 0.6 m/s per °C).

No internet? Fallback

Two students 20 m apart: one claps, the other starts a stopwatch on **seeing** the clap and stops on **hearing** it. Repeat 5 times, average t, compute v = 20 ÷ t, compare with 343 m/s. **Expected:** the true delay is only ≈ 0.058 s, shorter than human reaction time, so single results scatter widely (often 150–600 m/s); the **average** of five is closer: fusion in action!

📺 Watch: Curated Videos

📺 Video 1 · for 4.1

How do self-driving cars "see"? · TED-Ed (Sajan Saini) · 5:25 · youtube.com/watch?v=PRg5RNU_JLk

Why: animated intro to cameras, radar and LiDAR (4.1). **Do:** fill a 3-column table: sensor · what it measures · one weakness. **Watch out:** beginner level; no fusion maths.

📺 Video 2 · for 4.1

How Does LiDAR Remote Sensing Work? · NEON Science · 7:42 · youtube.com/watch?v=EYbhNSUnIdU

Why: time of flight and point clouds explained by a science organisation. **Do:** use d = c × t ÷ 2 for one example shown. **Watch out:** airborne mapping LiDAR, not car LiDAR.

📺 Video 3 · for 4.3

Visually Explained: Kalman Filters · Visually Explained · 11:18 · youtube.com/watch?v=IFeCIbljreY

Which part: 4:30 Kalman filters · 5:37 prediction · 7:14 update. **Do:** at 7:14, match each symbol to x, P, K, z and R from Figure 4.2. **Watch out:** matrix notation near the end.

📺 Primary source · for 4.5

Tesla AI Day 2021 · Tesla · 3:03:20 · youtube.com/watch?v=j0z4FweCy4M

Which part: 48:44–1:13:12 Tesla Vision. **Do:** list three claims and mark which are "company claims". **Watch out:** very long; a company presentation, not independent; 2021 technology.

Final Round: Recipe 04

Check

Task 1. A car at 100 km/h has a total latency of 0.1 s and brakes at 6 m/s². Total stopping distance? Extra distance if latency becomes 0.3 s?

Task 2. Radar says 40.0 m (σ = 0.4); camera says 42.0 m (σ = 1.6). Fused estimate and σ?

Task 3. Pick a sensor set for a delivery robot on pavements at night and in rain. Justify each choice with one row of Table 4.1.

Explained answers

Q1: 27.78 m/s → braking 27.78² ÷ 12 ≈ 64.3 m; + 2.78 m → **67.1 m**. At 0.3 s → 72.6 m: **5.6 m more**.

Q2: ≈ **40.12 m**, σ ≈ **0.39 m** (same as the Try It).

Q3 (example): ultrasonic for close obstacles and kerbs (works in the dark, cheap); radar for moving people and bikes in rain (least affected); a camera with lights for signs and crossings (reads colour). LiDAR optional for 3-D shape at night.

Stamp: Gold / Silver / Bronze.

Recipe Card 04

Recap

- No sensor sees everything: choose which **failures** you can live with.
- Cameras = detail and colour; LiDAR = direct 3-D distance; ultrasonic = cheap close range; radar = speed in bad weather.
- Real-time: every millisecond of **latency** is distance driven blind.
- **Fusion** weights sensors by confidence and tracks uncertainty over time (Kalman).
- Safety needs **diverse**, independent channels and honest evidence, not just strong claims.

Key words (see Glossary)

sensor · active / passive · pixel · FOV · LiDAR · time of flight · point cloud · ultrasonic · radar · Doppler · ADC · ISP · latency budget · sensor fusion · BEV · Kalman filter · IoU · precision · recall · SOTIF · fail-safe

Unlocked: Recipe Book page 4!

Recipe 04 · complete!

RECIPE 05

The Grand Banquet!

AI System Integration & Optimisation

Mama's Problem

One chef can cook one dish. A **banquet** for thousands needs hundreds of cooks, recipes they all follow, a head chef who plans, stoves, gas bills… and a plan for when a burner breaks. Real AI is the same: never one chip, always a **system**.

Today's route

5.1 Hardware–software interaction: compilers, fusion, MFU

5.2 Distributed AI: data, tensor and pipeline parallelism; all-reduce

5.3 Optimisation and resource allocation: measure first, batching, sharing

5.4 Energy, cost and scalability: PUE, 6ND, failures, checkpoints

5.5 Case study: Google TPU infrastructure

The story for this recipe

Running example: a team trains and serves a **7-billion-parameter** chat model on a rented cluster of **256 chips**. They must train it on **1.4 trillion tokens**, keep the chips busy, and then answer thousands of requests per second at a cost the business can afford.

Recipe 05 · start

5.1 From Code to Silicon

Why does this matter?

Predict: same model, same chip: can it run **3× faster**? Guess before reading.

A chip's peak number is only a promise. Much of real-world AI speed comes from the **software layers** between your code and the transistors.

Figure 5.1 · The AI software stack.

How to read this: your code enters on the left and becomes chip instructions on the right; each box can win or waste work.

Explain

A **framework** builds a **computation graph**: nodes are operations, arrows are tensors. A **compiler** improves the graph before it runs. Chip and compiler designed together (TPUs and XLA) is **hardware–software co-design**.

Compiler job	What it does	Why it helps
Operator fusion	Merges operations into one pass	No off-chip round trips
Layout & tiling	Cuts tensors into on-chip tiles	Data stays in fast memory
Precision	Picks FP16, BF16 or INT8	Fewer bytes, more MACs
Placement	Chooses which unit runs what, when	Keeps units busy

Table 5.1 · Four jobs of an AI compiler.

How to read this: the last column always comes back to Recipe 01: move less data, keep the MACs busy.

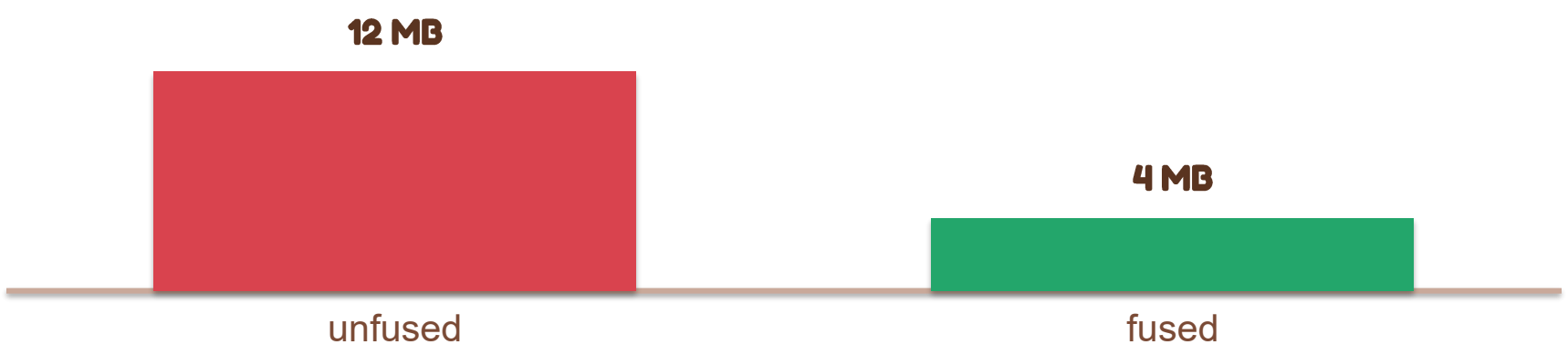
5.1 Fusion: One Trip to the Fridge

How it works, step by step

Why fusion saves time (illustrative numbers). A tensor has 1,000,000 FP16 values = **2 MB**. We apply three element-wise steps: scale, add bias, ReLU.

1. **Unfused:** each step reads the whole tensor and writes it back: 3 × (read + write) × 2 MB = **12 MB** of traffic.
2. **Fused:** read once, do all three steps while each value sits in a register, write once: **4 MB**.
3. These steps do almost no arithmetic per byte, so they are **memory-bound** (Recipe 01): 3× less traffic ≈ **3× less time**.

Memory traffic for three element-wise steps



Kitchen Analogy

Unfused = walking to the fridge three times for three ingredients. Fused = bringing everything back in one trip.

Where the analogy breaks down

A cook remembers what's next; a compiler can only fuse operations it **understands**. A custom operation acts as a wall.

Explain

Device switches. When an operation is unsupported, it **falls back** to the CPU. Data must be copied off the accelerator and back again. A 10-operation model with operation 5 unsupported splits into 1–4 (accelerator), 5 (CPU), 6–10 (accelerator): **two switches**, each with a copy that can cost more than the arithmetic saved.

5.1 MFU: Are the Cooks Really Cooking?

Explain

For training big models, the standard score is **MFU** (model FLOPs utilisation):

MFU = useful model FLOP/s ÷ peak FLOP/s of the hardware

It counts only the arithmetic the model **needs**, not wasted or repeated work. Well-tuned large training runs commonly report **30–60%**. The rest is lost waiting for memory, for other chips, and in gaps between operations.

Worked example

```
Peak 275 TFLOP/s, useful work 110 TFLOP/s
MFU = 110 / 275 = 40%
```

Explain

Cause and effect: low MFU means you are paying for **idle silicon**. In 5.4 you will see that raising MFU from 40% to 50% cuts training time, cost **and** energy by 20%, with no new hardware.

Chef's Warning

"The monitor says 100% busy, so we're efficient!" **Not quite!** A chip can look fully busy while mostly **copying data**. MFU measures **useful** work, which is why engineers prefer it.

Try It: Compiler Kitchen

Try It

🎯 **Goal:** calculate fusion savings, MFU and device switches.

🧺 **Preparation:** pages 5.1. Traffic = ops × (read + write) × tensor size; fused = 1 read + 1 write. MFU = useful ÷ peak. FP16 = 2 bytes.

🔪 **Task:** solve each:

1. A tensor of 4,000,000 FP16 values; four element-wise operations. Traffic unfused and fused? Percentage saved?
2. Peak 400 TFLOP/s; useful arithmetic 140 TFLOP/s. MFU?
3. 12 operations in a row; operations 3 and 8 are unsupported. How many device switches?
4. In 2–3 sentences: why must a compiler know the on-chip memory size when it tiles a big matrix?

👀 **Observation:** draw the 12 operations as boxes and colour the CPU ones before counting.

☁️ **Reflection:** in which question did the chip's peak speed **not** matter at all?

Mama's Feedback

1. 8 MB tensor: unfused 4 × 2 × 8 = **64 MB**; fused **16 MB**; saved **75%**.
2. 140 ÷ 400 = **35%**.
3. 1–2 acc → 3 CPU → 4–7 acc → 8 CPU → 9–12 acc = **4 switches**.
4. Tiles must fit on chip: too large spills to slow memory and repeats traffic; too small gives little reuse and extra overhead.

Reflection: Q1 and Q3: both are about **data movement**, not arithmetic speed.

5.2 Why Use Many Chips?

Why does this matter?

One chip is not enough for two reasons: either the model **doesn't fit** in its memory, or training would take **years**. Splitting the work fixes both, but every split creates communication, and communication is the price of going distributed.

Item	Bytes per parameter
Weights (BF16)	2
Gradients (BF16)	2
Master copy of weights (FP32)	4
Optimiser state: two moments (FP32)	8
Total	**16**

Table 5.2 · Training memory per parameter (mixed-precision Adam).

How to read this: a widely used rule of thumb; exact recipes vary, and activations come on top.

How it works, step by step

Reason 1: memory. Our 7B model, chips with 96 GB (illustrative):

1. Training state = $7 \times 10^9 \times 16$ bytes = **112 GB** > 96 GB: it doesn't fit, before activations.
2. **Sharded** over 256 chips: $112 \div 256 \approx$ **0.44 GB** per chip.

Reason 2: time. Training needs about **6 × N × D** FLOPs (N parameters, D tokens) = $6 \times 7 \times 10^9 \times 1.4 \times 10^{12} \approx 5.9 \times 10^{22}$. One chip at 100 TFLOP/s effective: $\approx 5.9 \times 10^8$ s ≈ **19 years**.

Kitchen Analogy

One cook can't fit a banquet on one stove (memory), and couldn't finish it before the guests grow old (time).

5.2 Three Ways to Split the Work

Strategy	What is split	Communication	Best when
Data parallel	The batch of examples; each chip has the whole model (or its shards)	Gradients summed once per step (all-reduce)	The model fits; you want more throughput
Tensor parallel	The matrices inside each layer	Partial results inside **every** layer; needs very fast links	One layer is too big
Pipeline parallel	Groups of layers (stages)	Activations passed stage to stage	The model is deep; links between stages are slower

Table 5.3 · Three parallelism strategies.

How to read this: read the Communication column: it explains why the strategy chosen depends on the network.

Figure 5.2 · Pipeline parallelism: each chip is one station on the line.

How to read this: a batch flows left to right like a dish passing from the chopping station to the wok to the plating station.

Explain

Real large-model training combines all three (**3D parallelism**): tensor parallelism inside a group of tightly linked chips, pipeline across groups, data parallelism across replicas. **Rule of thumb:** put the most communication-hungry strategy (tensor) on the **fastest** links. This is Recipe 02 in action: the network decides what is practical.

Kitchen Analogy

Data = many identical kitchens, each cooking part of the orders. Tensor = two cooks sharing one huge pot. Pipeline = an assembly line.

Where the analogy breaks down

Kitchens don't have to agree after every dish; data-parallel chips **must** sync their gradients every single step.

5.2 Keeping Chips in Sync: All-Reduce

Explain

In data parallelism each chip computes gradients on its own slice of the batch. Before the weights update, all gradients must be **summed and shared**: an **all-reduce**. In a **ring all-reduce** the chips sit in a ring and pass chunks around. For S bytes of gradients on N chips, each chip sends and receives about:

traffic per chip = 2 × (N − 1) ÷ N × S

For large N this approaches **2 × S**, almost independent of N: that is why data parallelism scales well in principle.

Worked example (illustrative)

```
Gradients: 7e9 x 2 bytes (BF16)              = 14 GB
Traffic:   2 x 255/256 x 14 GB               = 27.9 GB
Comms:     27.9 GB / 100 GB/s                = 0.28 s
Compute:   6 x 7e9 x 4e6 tokens / (256 x 200T x 40%) = 8.2 s
Efficiency: 8.2 / (8.2 + 0.28)               = 96.7%
```

Explain

The overhead is small because each step has lots of compute (a 4-million-token batch). With a smaller batch per chip, compute shrinks but the all-reduce stays the same size, so overhead **grows**. Real systems also **overlap** communication with the backward pass to hide it.

⚠ Chef's Warning

"Double the chips, halve the time!" **Not quite!** Every chip added means more syncing, and serial parts never speed up (Amdahl, Recipe 01). This example is also simplified: real 7B training would shard the state, which changes the pattern.

5.2 Did the Extra Cooks Pay Off?

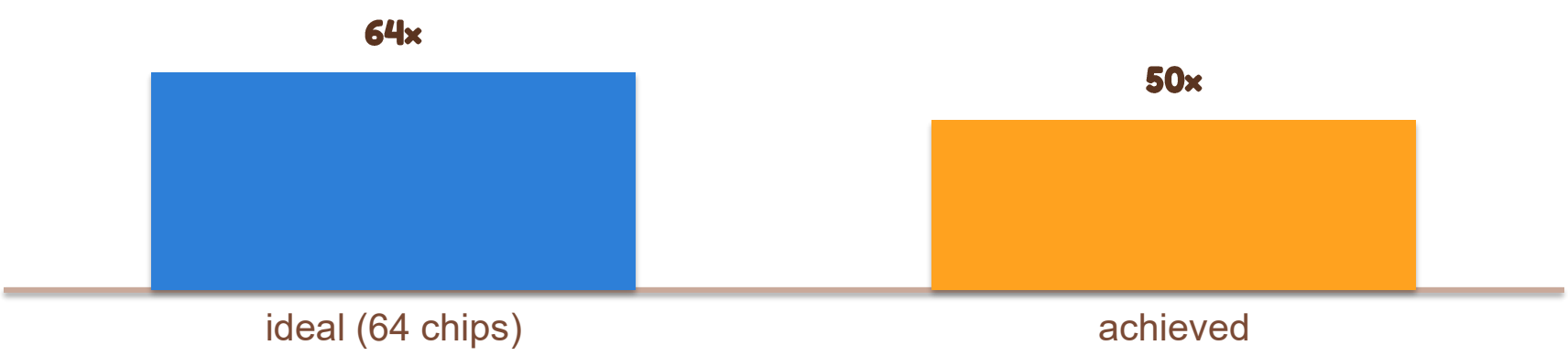
Explain

Measure how well adding chips pays off:

speed-up = T(1 chip) ÷ T(N chips) · scaling efficiency = speed-up ÷ N

Worked example

```
1 chip: 100 h        64 chips: 2.0 h
speed-up = 100 / 2.0 = 50
efficiency = 50 / 64 = 78%
```



Explain

The missing 22% went to communication, idle units and serial work. Two ways to measure:

- **Strong scaling**: fixed problem, more chips. Efficiency falls as each chip gets less work.
- **Weak scaling**: the problem grows with the chips. Easier to keep efficient, and what large training runs usually rely on.

Try It: Split the Banquet

Try It

🎯 **Goal:** size a model's memory, time an all-reduce, score scaling, and pick a strategy.

🧺 **Preparation:** pages 5.2. 16 bytes per parameter; traffic = 2(N − 1)/N × S; time = traffic ÷ bandwidth; efficiency = speed-up ÷ N.

🔪 **Task:** solve each:

1. A 13B model, chips with 80 GB. Training-state size? Minimum chips to hold it?
2. A 1B model, BF16 gradients, 8 chips at 50 GB/s each. Traffic per chip and time for one ring all-reduce?
3. 120 h on 1 chip; 4.5 h on 32 chips. Speed-up and efficiency?
4. Best strategy: (a) model fits, want more data per second (b) one matrix too big (c) very deep model in stages?

👀 **Observation:** write units on every line (GB, GB/s, s).

☁️ **Reflection:** why does tensor parallelism need the fastest links of all?

Mama's Feedback

1. 13 × 10⁹ × 16 = **208 GB**; 208 ÷ 80 = 2.6 → at least **3 chips** (more with activations).
2. S = 2 GB; traffic = 2 × 7/8 × 2 = **3.5 GB**; 3.5 ÷ 50 = **0.07 s**.
3. Speed-up ≈ **26.7**; efficiency ≈ **83.3%**.
4. (a) data (b) tensor (c) pipeline.

Reflection: it exchanges partial results inside **every** layer: many small messages, so latency and bandwidth both bite.

5.3 Measure First, Then Fix

Why does this matter?

Beginners **guess** what is slow. Experts **measure**. Optimising the wrong thing wastes days, and because chips are expensive, every idle minute is money burned.

1 · Profile where does time go? → **2 · Locate** the one bottleneck → **3 · Change** one thing only → **4 · Re-measure** keep it if better

Figure 5.3 · The optimisation loop.

👀 **How to read this:** go round again and again: removing one bottleneck simply exposes the next one.

Symptom	Likely bottleneck	Typical fix
Low MFU, units idle between ops	Many small ops; launch overhead	Fusion; bigger batches; compile the graph
Low intensity on the roofline	Memory bandwidth	Lower precision; reuse on chip
Step time grows with chips	Communication	Overlap comms; better placement
Chips wait for data	Input pipeline	Prefetching; more loaders; faster storage
Out-of-memory errors	Memory capacity	Sharding; recomputation; lower precision

Table 5.4 · Matching symptoms to bottlenecks.

👀 **How to read this:** start in the Symptom column with what your profiler shows, then read across. Never start from the fix.

Explain

Training tricks: **mixed precision** (BF16 maths, FP32 master copy), overlapping comms, bigger batches, and **activation recomputation**: recompute intermediates in the backward pass instead of storing them. Step = 1 forward + 2 backward = 3 parts → 4 parts: **33% more compute** (8.2 s → **10.9 s**) to save memory.

5.3 Serving: The Speed Limit of a Chat

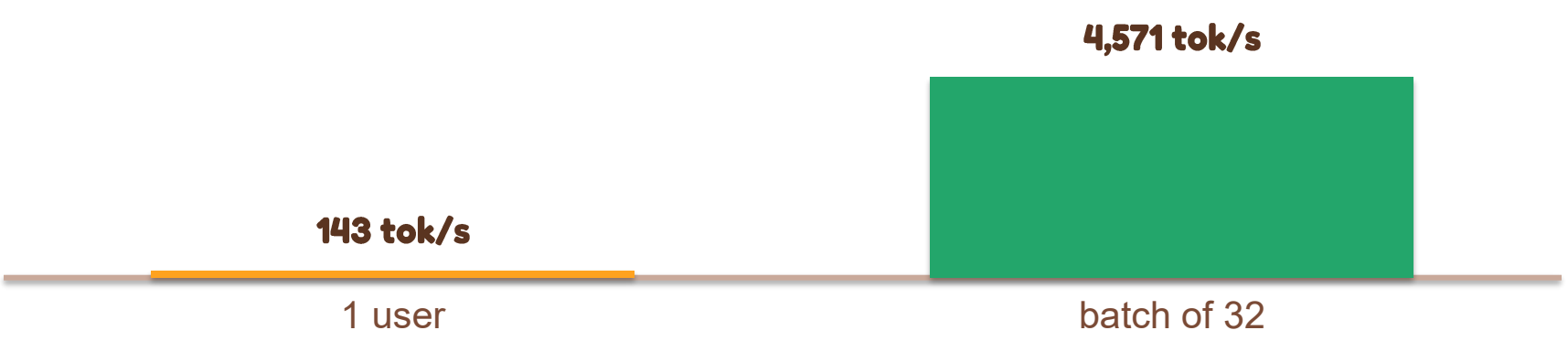
Explain

A language model writes **one token at a time**. For every new token the chip reads **all** the weights from memory once, but does little arithmetic with them: the **memory-bound** case of Recipe 01 (about 1 FLOP per byte at batch 1).

How it works, step by step

Worked example (illustrative): 7B model in INT8 = 7 GB; chip bandwidth 1 TB/s.

1. Read the weights once: 7 GB ÷ 1,000 GB/s = **7 ms** per token.
2. One user: 1 ÷ 0.007 ≈ **143 tokens/s**, however many MACs the chip has.
3. Batch 32 users: weights read once for all → ≈ **4,571 tokens/s** in total; each user still sees ≈ 143.



Explain

The catch: waiting. Requests must wait to be grouped. At 100 requests/s, a full batch of 32 makes the first request wait ≈ **0.31 s**. Servers use a **timeout** (e.g. run after 50 ms, ≈ 5 requests) or **continuous batching**, where new requests join a running batch between tokens. This is the **throughput–latency trade-off**.

Chef's Warning

Upper bounds: they ignore the **KV cache** (stored attention state that grows with the conversation). Still, batching raises throughput almost free until compute-bound.

5.3 Sharing the Kitchen

Explain

A cluster is shared by many teams. A **scheduler** decides who gets which chips, and when.

Idea	Meaning
Quota	Each team has a guaranteed share of chips
Priority & preemption	Customer-facing serving can pause lower-priority experiments
Gang scheduling	A training job starts only when **all** its chips are free at once
Topology-aware placement	Chips that talk a lot are placed close on fast links
Bin packing	Fit jobs of different sizes with as few gaps as possible
Autoscaling	Add serving replicas when load rises; release them when it falls

Table 5.5 · Key ideas in cluster resource allocation.

How to read this: every row has the same goal: fewer idle chips.

How it works, step by step

Why utilisation is a money issue. A chip costs **$2 per hour** whether busy or not.

1. At 35% utilisation, each **useful** hour costs 2 ÷ 0.35 ≈ **$5.71**.
2. At 60%: 2 ÷ 0.60 ≈ **$3.33**.
3. Saving ≈ **42%**, without buying any hardware.

Kitchen Analogy

A restaurant pays rent for every table, full or empty. A good host seats parties to fill the gaps.

Where the analogy breaks down

Diners can share a big table; a training job needs **all** its chips at once and in the right shape.

Try It: Head Chef's Profiler

Try It

🎯 **Goal:** apply recomputation, the serving speed limit, the bottleneck table and the utilisation cost.

🧺 **Preparation:** pages 5.3. Recomputation = × 4 ÷ 3. Tokens/s ≈ bandwidth ÷ model bytes. Cost per useful hour = price ÷ utilisation.

🔪 **Task:** solve each:

1. A step takes 20 s. With full recomputation?
2. 13B model in INT8, 2 TB/s memory bandwidth. Memory-bound limit for one user?
3. MFU is 15% and the profiler shows chips idle while the next batch is read from storage. Which row of Table 5.4? What do you try first?
4. A chip costs $3/hour. Cost per useful hour at 25% and at 50% utilisation?

👀 **Observation:** for Q3, say the symptom out loud before choosing a fix.

☁️ **Reflection:** in Q3, why would buying more chips make things worse?

Mama's Feedback

1. 20 × 4 ÷ 3 ≈ **26.7 s**.
2. 13 GB ÷ 2,000 GB/s = **6.5 ms** → ≈ **153.8 tokens/s**.
3. "Chips wait for data" (input pipeline): try prefetching, more loader workers or faster storage.
4. **$12** and **$6** per useful hour.

Reflection: more chips would wait for the **same** slow storage: more idle silicon, more cost.

5.4 The Gas Bill: Energy and PUE

Why does this matter?

Three ceilings limit how big an AI system can usefully grow: the **electricity** it needs, the **money** it costs, and the **failures** it suffers. They are linked: improving one often pushes pressure onto another.

Explain

Power is the rate of using energy (watts). **Energy** is power × time (kWh). A 400 W chip for 10 hours uses 4 kWh.

Chips aren't the only load: cooling, power conversion and lighting add more. **PUE** (power usage effectiveness) = total facility energy ÷ computing equipment energy. 1.0 would mean zero overhead; older facilities often reach 1.5 or worse; the most efficient large operators report about 1.1.

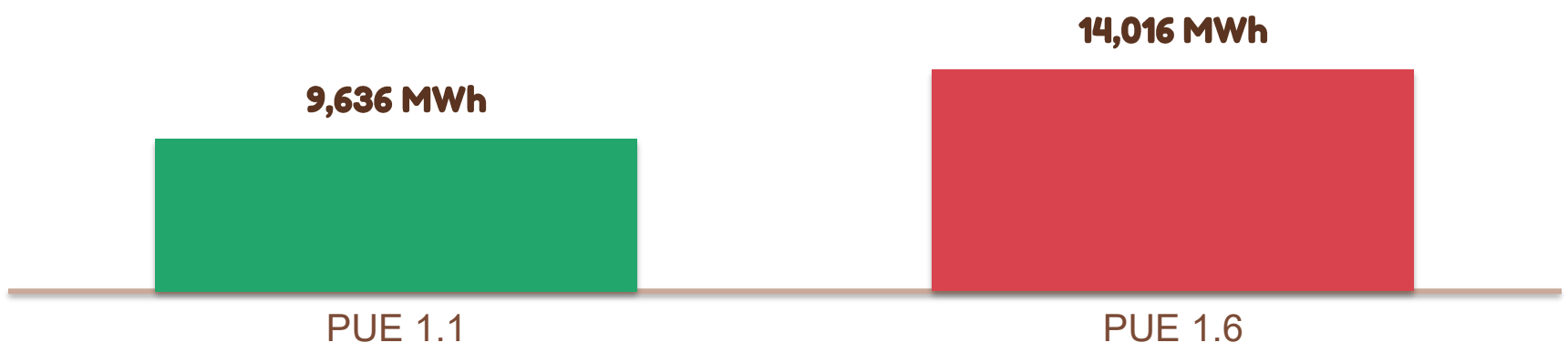
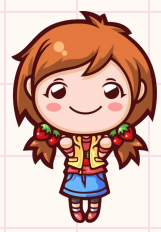


Figure 5.4 · Same computing, different buildings.

How to read this: the gap is 4,380 MWh per year: at an illustrative $80 per MWh, about $350,400, for exactly the same work.

Explain

From energy to carbon: emissions = energy × carbon intensity of the grid. A 89.8 MWh job is about 35.9 t CO_2e at 0.4 kg/kWh but about 4.5 t at 0.05 kg/kWh: **where and when** a job runs can matter as much as how efficiently it runs. And since moving data costs ~170× a multiply (Recipe 01), every data-saving trick is also an **energy** trick.



5.4 The Full Bill for Our 7B Model

Explain

A widely used estimate for dense transformers is **training FLOPs ≈ 6 × N × D** (Kaplan et al., 2020; Hoffmann et al., 2022): 2 FLOPs per MAC × 3 (forward + about twice for backward). Hoffmann et al. found about **20 tokens per parameter** a good balance; 7B with 1.4T tokens is exactly that.

The bill

```
Illustrative: 256 chips, 200 TFLOP/s peak, 40% MFU, 400 W, $2/chip-h, PUE
1.1
FLOPs   = 6 x 7e9 x 1.4e12                 = 5.88e22
Rate    = 256 x 200e12 x 0.40              = 2.048e16 FLOP/s
Time    = 5.88e22 / 2.048e16 = 2.87e6 s ~ 33.2 days (797.5 h)
Cost    = 256 x 797.5 h x $2               ~ $408,333
Energy  = 256 x 0.4 kW x 797.5 h x 1.1     ~ 89.8 MWh
```

Explain

Cause and effect: raising MFU from 40% to 50% cuts time, cost **and** energy by 20% each. That is why 5.1–5.3 matter.

Explain

Serving is the recurring bill. At 5,000 tokens/s a chip makes 5,000 × 3,600 = 18 million tokens per hour; at $2/hour that is ≈ **$0.11 per million tokens**. **TCO** (total cost of ownership) adds purchase price, electricity, cooling, networks, buildings, maintenance and staff.

Chef's Warning

"$408k trains our model." **Not quite!** That is **one successful run**. Real projects also pay for failed runs, experiments and tuning (often several times the final run), data preparation and staff.

5.4 When Breakdowns Become Routine

Explain

One chip may run for years without failing. Thousands of chips fail **every few hours**, because failure rates add up. If each chip has a **MTBF** (mean time between failures) of M hours and the job stops when **any** chip fails, the system MTBF ≈ **M ÷ N**.

How it works, step by step

4,096-chip job, chip MTBF 5 years = 43,800 h (illustrative):

1. System MTBF ≈ 43,800 ÷ 4,096 ≈ **10.7 hours**: dozens of interruptions a month.
2. Defence: **checkpointing**: save the full training state regularly.
3. Young (1974): **best interval ≈ √(2 × C × MTBF)**, C = time to write one checkpoint.
4. C = 2 min (0.0333 h): √(2 × 0.0333 × 10.7) ≈ 0.84 h ≈ **51 minutes**, overhead ≈ **3.9%**.

Kitchen Analogy

Saving a long video game: save too rarely and a crash loses an hour; save every minute and you barely play.

Where the analogy breaks down

A game has one player. A training checkpoint must capture the state of **thousands** of chips at one consistent moment.

Lever	Helps	May hurt
More chips	Time to train	Cost, power, failures, comms
Lower precision	Speed, energy, memory	Accuracy (needs care)
Larger batch	Utilisation, throughput	Serving latency; learning quality
Frequent checkpoints	Resilience	Time and storage

Table 5.6 · Levers and their side effects.

How to read this: every lever has a "may hurt" column: there is no free lunch, only trade-offs.

Try It: Run the Restaurant

Try It

🎯 **Goal:** compute facility energy, training time, system MTBF, checkpoint interval and serving cost.

🧺 **Preparation:** pages 5.4. Energy = load × PUE × hours. Time = 6ND ÷ (chips × peak × MFU). System MTBF = M ÷ N. Interval = √(2 × C × MTBF). Cost/M tokens = $/h ÷ (tokens/s × 3,600 ÷ 10^6).

🔪 **Task:** solve each:

1. A 2 MW load all year: energy at PUE 1.2 and 1.5? Extra used?
2. 3B parameters, 60B tokens, 64 chips at 200 TFLOP/s and 40% MFU. FLOPs and time in hours?
3. 1,024 chips, chip MTBF 4 years. System MTBF?
4. A checkpoint takes 3 min. Best interval (use Q3)?
5. A $3/hour chip serves 6,000 tokens/s. Cost per million tokens?

👀 **Observation:** convert years to hours (× 8,760) and minutes to hours (÷ 60) first.

☁️ **Reflection:** which ceiling (energy, cost, failures) surprised you most, and why?

Mama's Feedback

1. 21,024 vs 26,280 MWh → **5,256 MWh** extra.
2. **1.08×10^{21}** FLOPs; rate 5.12×10^{15} → **≈ 58.6 h**.
3. 35,040 ÷ 1,024 ≈ **34.2 h**.
4. √(2 × 0.05 × 34.2) ≈ 1.85 h ≈ **111 min** (overhead ≈ 2.7%).
5. 21.6 M tokens/h → ≈ **$0.14** per million.

5.5 Case Study: Google TPU Pods

Why does this matter?

Google's TPU infrastructure is this whole recipe in one real system: a chip built around MACs and on-chip memory (Recipe 01), joined by a purpose-built network (5.2), programmed through a compiler (5.1), shared by a scheduler (5.3), and designed around energy, cost and failure (5.4).

Read this first

TPU generations and figures change quickly. This section teaches the **design ideas**, using the TPU v4 numbers from Jouppi et al. (2023). Check Google Cloud's TPU documentation before quoting any current specification.

Level	What it is (TPU v4)	Why it exists
Chip	Matrix units, vector unit, on-chip memory, HBM, ICI ports	Executes MACs efficiently
Tray	4 chips + a host CPU	Accelerators need a host
Cube	64 chips as 4 × 4 × 4	Unit of wiring and deployment
Pod	64 cubes = **4,096 chips**	Largest tightly coupled unit
Slice	The part of a pod one job gets	Many jobs share a pod
Multi-slice	Slices linked by the data-centre network	Training beyond one pod

Table 5.7 · Levels of a TPU system (v4 example).

How to read this: each row is built from the row above it, like dishes → tables → a banquet hall.

Explain

History in one line: the first TPU (2015) was inference-only, 8-bit, 92 TOPS; from 2017 TPUs added training (bfloat16, HBM, pods); v4 brought 4,096-chip pods; Ironwood (2025) targets inference at scale. Each step moved attention from the **chip** to the **system**.

5.5 A Doughnut-Shaped Network

Explain

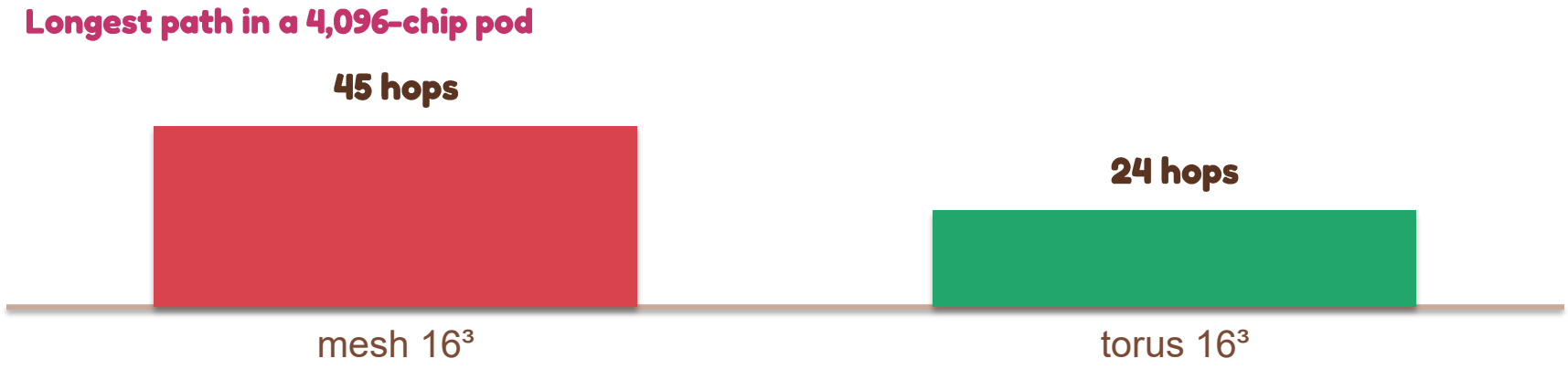
TPU chips connect **directly** to their neighbours over dedicated **ICI** (inter-chip interconnect) links in a 3-D grid: six neighbours each (up, down, left, right, front, back). In a **torus** the links **wrap around**: the chip at one edge links to the chip at the opposite edge.

Why it suits AI: ring all-reduces become neighbour-to-neighbour hops on short, fast cables; there's no central switch to become a bottleneck; and wraparound shortens the longest path.

How it works, step by step

How much does wraparound help? The **diameter** is the longest shortest path, in hops.

1. **Mesh** (no wraparound): 3 × (side − 1).
2. **Torus**: 3 × (side ÷ 2).
3. 4 × 4 × 4 cube: mesh 9, torus **6** hops. 16 × 16 × 16 pod: mesh 45, torus **24** hops.



Explain

Optical circuit switches (**OCS**). Links **between cubes** pass through switches that steer light with tiny mirrors, so software can re-patch connections without touching a cable. Benefits: **availability** (route around a failed cube), **flexible slice shapes**, and **incremental deployment** (start before every cube arrives).

5.5 Software and the Fleet

Explain

XLA compiles the graph for the TPU (fusion, layouts). **JAX, TensorFlow and PyTorch/XLA** let programmers write models normally. With **sharding annotations** the programmer just says how each tensor is split; the compiler inserts the all-reduces automatically. **Multi-slice training** joins slices over the slower data-centre network.

How it works, step by step

Why communicate hierarchically? (illustrative) Two slices of 256 chips; 14 GB of gradients; the slow network gives each chip 12.5 GB/s.

1. **Naive:** every chip sends all 14 GB across: 14 ÷ 12.5 ≈ **1.12 s**.
2. **Hierarchical:** first a **reduce-scatter** inside each slice (fast ICI) leaves each chip holding 14 ÷ 256 ≈ 0.055 GB. Only that crosses: ≈ **4.4 ms**.
3. Slow-network time shrinks **256×**, the chips per slice.

Explain

Principle: do as much reduction as possible on the **fastest** links; send only the minimum across the **slowest**. It's Recipe 01's memory hierarchy, applied to networks.

Running the fleet: the scheduler gives slices by quota and priority, places chips as neighbours and gang-schedules jobs; OCS makes any healthy cubes usable. Health monitoring, rerouting and frequent checkpoints turn a failure into minutes lost, not days. Google reports a fleet-wide PUE close to 1.1, and recent TPUs use **liquid cooling**.

Kitchen Analogy

Each table totals its own bill first; only one waiter per table walks the total to the cashier.

Where the analogy breaks down

Waiters add simple numbers; chips sum huge gradient arrays, perfectly in sync, thousands of times a day.

5.5 Should Your Team Use TPUs?

👍 Strengths

High performance per watt and per dollar on large matrix work

Chip, network, compiler and scheduler tuned together

Torus scales training to thousands of chips

Optical switching raises availability

Generations for training or for serving

👎 Limitations

Less flexible than a GPU for unusual operations

Tied to XLA / JAX / PyTorch-XLA; porting from CUDA takes effort

Mostly through Google's cloud: one-provider dependence

Extra complexity (OCS)

Peak numbers ≠ real performance

🔍 Explain

Judge with your own numbers. Measure MFU and cost per useful token on **your** workload, and include porting effort. A chip faster on paper may not be cheaper in practice.

Conclusion: AI performance is a property of **systems**, not chips: compiler, network, scheduler, cooling and budget all decide whether the peak number is ever reached.

✅ Check

1. Why is a 3-D torus a good match for all-reduce?
2. Diameter of an 8 × 8 × 8 mesh and torus? Shortening in %?
3. Why does hierarchical communication cut slow-network time by exactly the chips per slice?
4. A start-up's PyTorch model uses a custom GPU kernel: two things to check before moving to TPUs.

⭐ Explained answers

1. Six direct neighbours: ring collectives use short fast links; no central switch; wraparound keeps paths short. 2. Mesh **21**, torus **12** → ≈ **43%** shorter. 3. After the reduce-scatter each of the 256 chips holds 1/256 of the sum; only that crosses. 4. A compiler-supported equivalent of the kernel; porting effort; measured MFU and cost per token; provider dependence.

🌐 Kitchen Lab: Time a Real Accelerator

🌐 Kitchen Lab

🎯 **Goal:** see compilation, warm-up and scaling on a real accelerator.

🧺 **Preparation:** colab.research.google.com (Google account) → New notebook → Runtime → Change runtime type → **TPU** (or T4 GPU if TPUs are busy). JAX is preinstalled; check with print(jax.devices()).

🔪 **Task:** run the code below, then change 4096 to 1024 and 8192.

💻 Code

```
import jax, time; import jax.numpy as jnp
x = jnp.ones((4096, 4096))
f = jax.jit(lambda a: (a @ a).sum())
f(x).block_until_ready(); t = time.time()
f(x).block_until_ready(); print(time.time() - t)
```

🔍 Observe

Why is the **first** call slower? How does time grow when the size doubles (FLOPs grow 8×)? Compute TFLOP/s = $2 \times n^3 \div$ time and compare with the chip's peak: what MFU do you get?

💭 **Reflection:** which page of this recipe explains the gap?

⭐ Mama's Feedback

The first call takes about 0.1–1 s because **XLA compiles** the function; later calls take milliseconds. n = 4,096 is ≈ 137 GFLOP. Doubling n gives close to 8× the time only for large sizes; at 1,024 fixed overhead dominates. Time 3 runs: one is noisy. MFU well below 100% is normal (5.1).

📴 No internet? Fallback

"Running the Bill" on paper: with the numbers from 5.4 (256 chips), recompute days, chip-hours, $ and MWh for MFU = 30%, 50% and 60%, and plot cost against MFU. **Expected:** cost falls as 1 ÷ MFU: ≈ $544k at 30%, $327k at 50%, $272k at 60%.

Watch: Curated Videos

Video 1 · for 5.2

A friendly introduction to distributed training · TensorFlow (ML Tech Talks) · 24:20 · youtube.com/watch?v=S1tN9a4Proc

Which part: 0:37 why distribute · 1:49 data vs model parallelism · 6:05 synchronous data parallelism · 18:20 asynchronous. **Do:** at 6:05, connect the ring all-reduce to 2(N − 1)/N × S. **Watch out:** TensorFlow-focused; no cost or energy.

Video 2 · for 5.5

Scaling AI with Google Cloud's TPUs · Google Cloud Tech · 5:54 · youtube.com/watch?v=8zBaa20mvoc

Which part: 0:26 TPUs · 1:11 HBM · 2:18 ICI and resiliency · 3:44 multislice · 4:40 frameworks. **Do:** at 3:44, explain why multislice needs hierarchical communication. **Watch out:** vendor video; claims not independently tested.

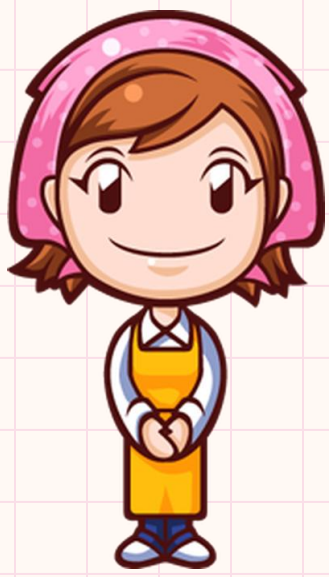
Video 3 · for 5.4

Inside a Google Cloud TPU Data Center · Google Cloud · 2:42 · youtube.com/watch?v=FsxthdQ_sL4

Why: see real TPU racks and liquid cooling (5.4). **Do:** name two parts of the building that exist only to remove heat or supply power. **Watch out:** short marketing tour; no PUE or cost numbers.

No internet? Fallback

No video access? Each task can be done from the matching page and figure of this recipe: Figure 5.2 and the all-reduce example (Video 1), the torus page (Video 2), and Figure 5.4 (Video 3).



🎯 Final Round: Recipe 05

✅ Check

Task 1. A 20B model, 16 bytes per parameter, 80 GB chips. State size? Minimum chips? Memory per chip if sharded over 512?

Task 2. Training needs 1.2×10^{22} FLOPs on 128 chips at 300 TFLOP/s peak and 45% MFU, $2.50/chip-hour. Time in days? Cost?

Task 3. Diameter of a 6 × 6 × 6 mesh and torus? In one sentence: why do fewer hops matter for an all-reduce?

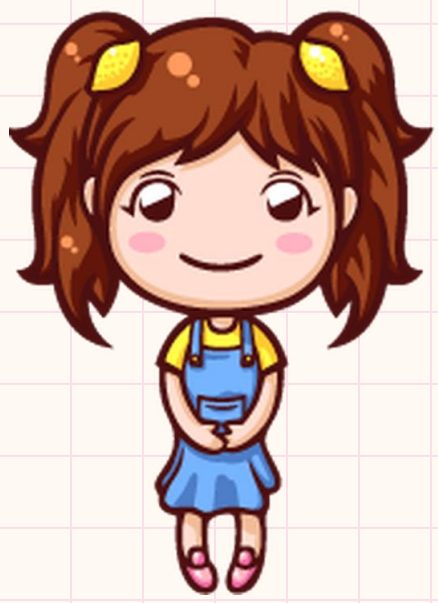
⭐ Explained answers

Q1: 320 GB → **4 chips** minimum (more with activations); sharded 512 ways ≈ **0.63 GB** per chip.

Q2: rate = 1.728×10^{16} FLOP/s → ≈ 694,444 s ≈ **8.0 days** (192.9 h); 128 × 192.9 ≈ 24,691 chip-hours → ≈ **$61,728**.

Q3: mesh 3 × 5 = **15**; torus 3 × 3 = **9** hops. Every hop adds latency to every message, and collectives send many messages, so shorter paths finish the all-reduce sooner.

Stamp: Gold / Silver / Bronze.



Recipe Card 05

Recap

- Software layers decide how well a chip is used: fuse, tile, choose precision, schedule.
- Split work when the model doesn't fit or would take years: data, tensor, pipeline.
- Every split costs communication: measure it with scaling efficiency.
- Measure first; fix one bottleneck at a time. Serving is memory-bound; batching trades latency for throughput.
- Energy, cost and failures are the three ceilings; checkpoint wisely.
- AI performance is a property of **systems**, not chips.

Key words (see Glossary)

computation graph · fusion · XLA · MFU · sharding · data / tensor / pipeline parallelism · all-reduce · scaling efficiency · recomputation · KV cache · continuous batching · gang scheduling · PUE · 6ND · TCO · MTBF · checkpoint · torus · OCS · pod · slice

Unlocked: the final Recipe Book page!

Recipe 05 · complete!

BONUS KITCHEN ARCADE

PLAY!

Pup's Pantry Maze

Fixed rules vs search vs reinforcement learning

🐶 Pup's Problem

Mama needs a carrot from the pantry! Three "chefs" try to guide me through the maze: one follows a **fixed rule**, one **searches** a map, and one lets me **learn from rewards**. Play the game on a screen, then come back for the paper puzzles!

🌐 Play on a screen

🎮 **The interactive game is separate from this PDF.** Scan the QR code or open https://claude.ai/artifact/KU7JXbM21K5xvFqT2TUSow.

💾 **Offline:** open **Pup_Pantry_Maze_offline.html** (in our repository / shared folder) in any browser. No install, no account, no internet.

👩‍🍳 Inside: 5 missions, a scoreboard, a kitchen editor and a quiz. About 30–40 minutes.

Scan to play

📄 What's on these pages

📄 **In this PDF (paper activities):** trace the Rule Chef and the Map Chef by hand on a printed maze, then solve the **Kitchen Crossword** of key terms. Answers are on the answer-key page.

🎯 **You'll be able to** say whether a program uses a fixed rule, search or reinforcement learning, and explain why only one of them improves with experience.

Three Chefs, Three Kinds of AI

Why does this matter?

People call any game character “AI”. But a rule-follower, a map-searcher and a reward-learner need different information, fail differently, and only one **improves with experience**.

	Rule Chef	Map Chef	Learner Pup
Technique	Fixed rule: right-paw wall following	Search: breadth-first search (BFS)	Reinforcement learning: tabular Q-learning
Needs the map?	No	**Yes**, all of it, before moving	No
Memory	Only which way it faces	Every square it checked	A Q-table: one value per square and move
Learns?	No: same mistake forever	No: re-plans from scratch	**Yes**, from rewards over many runs
Guarantee	Solves mazes whose walls all connect	Fewest steps, if a path exists	Moves towards the highest total reward
Weak spot	Walls that form an “island”	Only counts steps; needs a full map	Needs practice; learns exactly what the reward says

Table B.1 · The three chefs in the game.

How to read this: read a row to compare one property; read a column for one chef's personality. Only the last column has “Yes” for learning.

Kitchen Analogy

Rule Chef = a cook who always turns right at every corner. Map Chef = a cook with the floor plan. Learner Pup = a puppy rewarded with treats until it knows the way.

Where the analogy breaks down

A real puppy remembers feelings and places; the Learner Pup only stores numbers in a table, one per square and move.

Chef's Warning

"It found a perfect route, so it must be learning!" **Not quite!** Map Chef learns nothing: it searches a map it was given. Ask: **does its behaviour change from experience?**

Paper Maze: Island Trap

Try It · Trace the chefs

🎯 **Goal:** run two of the chefs by hand, with no computer.

🧺 **Preparation:** a pencil. 🧺 = start (facing east, →); carrot = goal.

🔪 **Task:** 1. **Rule Chef:** from 🧺, keep your right hand on the wall: at each square try right, else straight, else left, else turn back. Mark each square with a dot. Stop when you reach the carrot or repeat a move.

2. **Map Chef:** write 0 in 🧺, 1 in every open neighbour, 2 in theirs… until a number reaches the carrot. Then trace back down the numbers to draw the shortest path.

👀 **Observation:** how many moves before Rule Chef repeats itself? What number lands on the carrot?

☁️ **Reflection:** Map Chef “knew” the map; Rule Chef did not. Which one would improve if you tried again tomorrow?

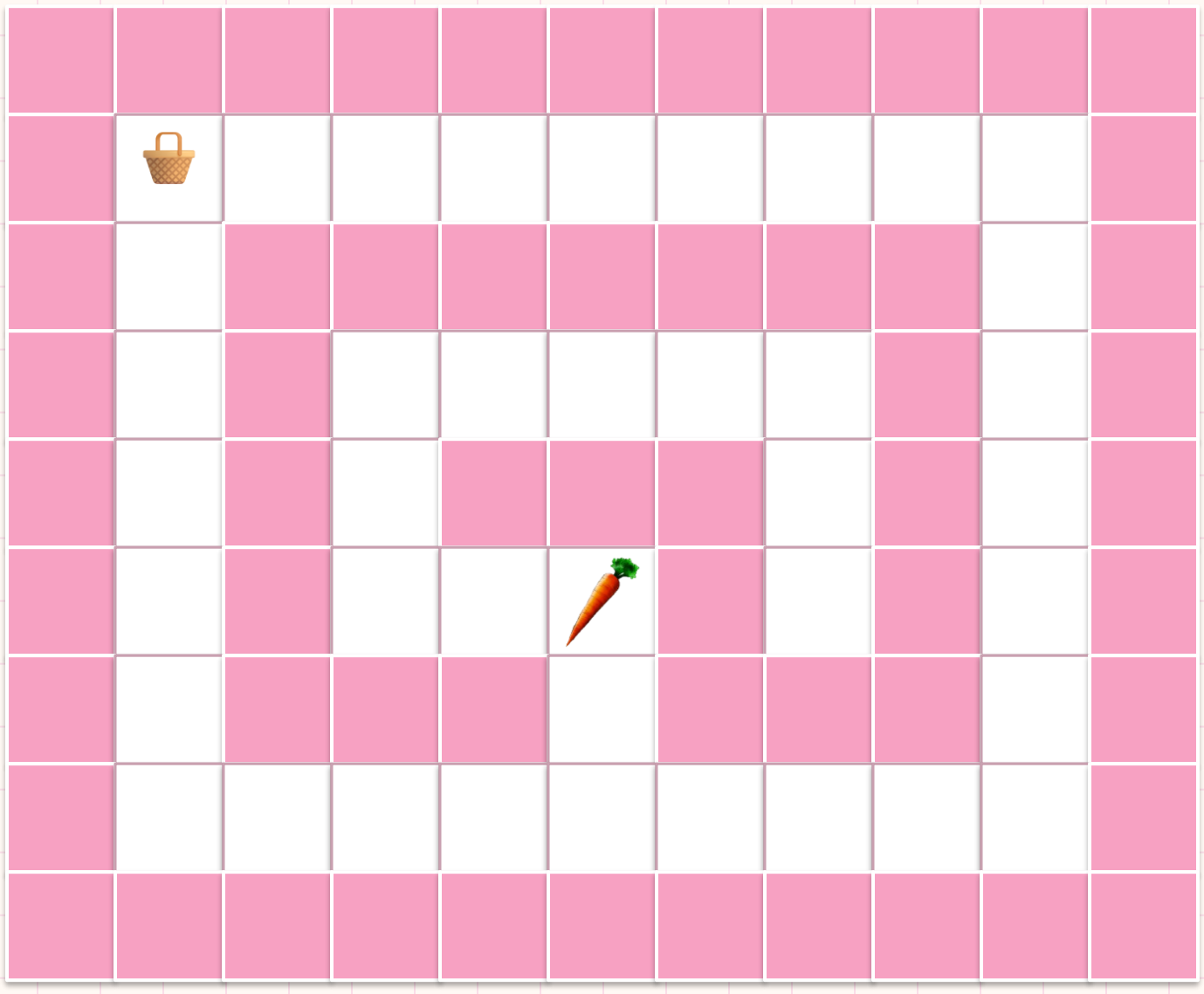
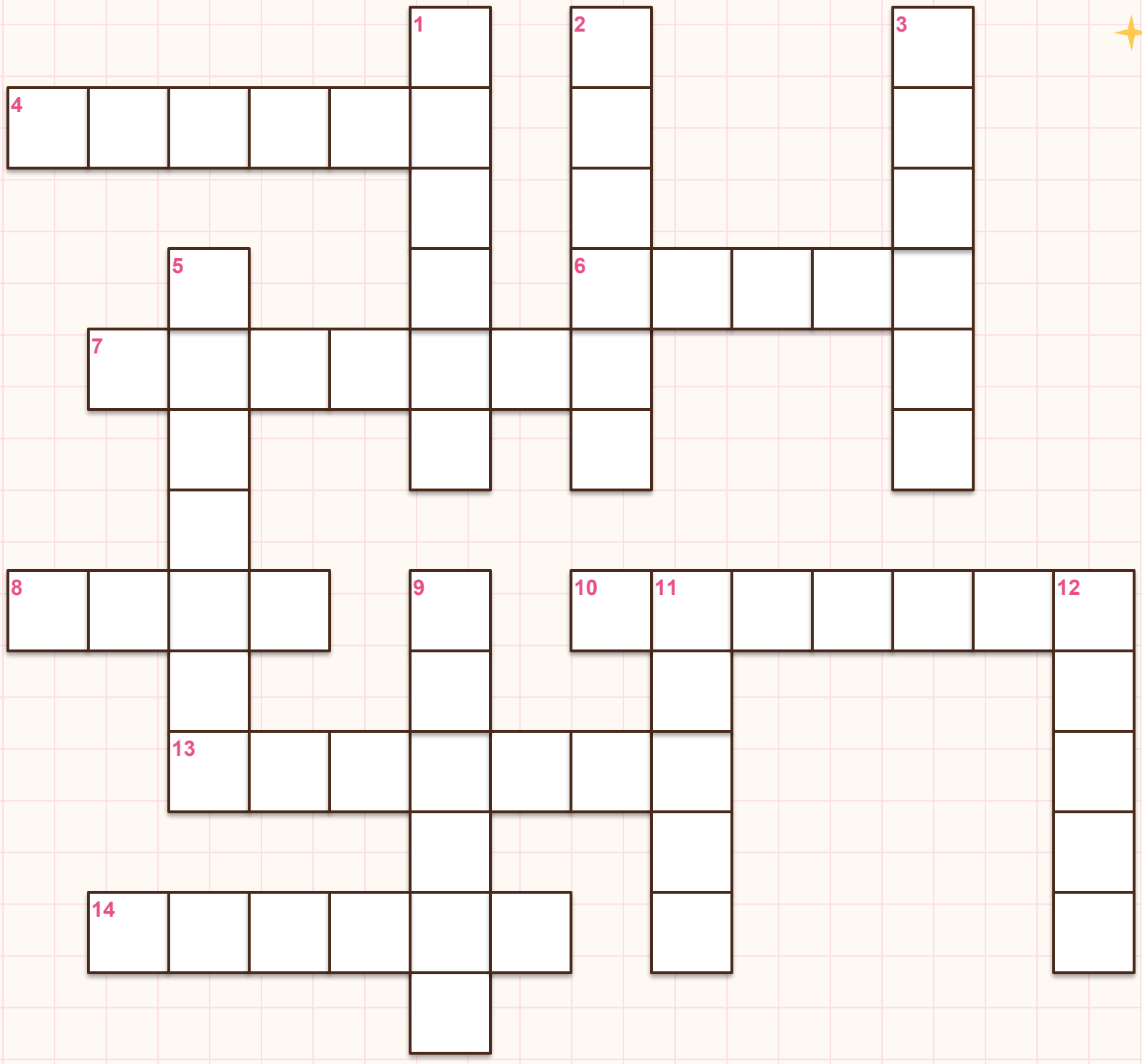


Figure B.1 · Island Trap (the same kitchen as in the game).

👀 **How to read this:** pink squares are walls. The carrot's walls are not joined to the outer wall: that is the “island”.

Kitchen Crossword



Across

4. The ingredient Mama needs from the pantry. (6)

6. The learning rate: how big each Q-update is. (5)

7. Greek letter for the chance of a random, exploring move. (7)

8. What Rule Chef gets stuck in on Island Trap. (4)

10. Reward ___: farming cookies instead of fetching the carrot. (7)

13. Try a new move instead of the best-known one. (7)

14. What Map Chef does over the whole map before taking a step. (6)

Down

1. The Learner Pup's memory: one value for every square and move (one word). (6)

2. Walls not joined to the outer wall, which defeat wall-following. (6)

3. The number that tells the pup how good its last move was. (6)

5. One practice run, from the basket until the carrot or the time limit. (7)

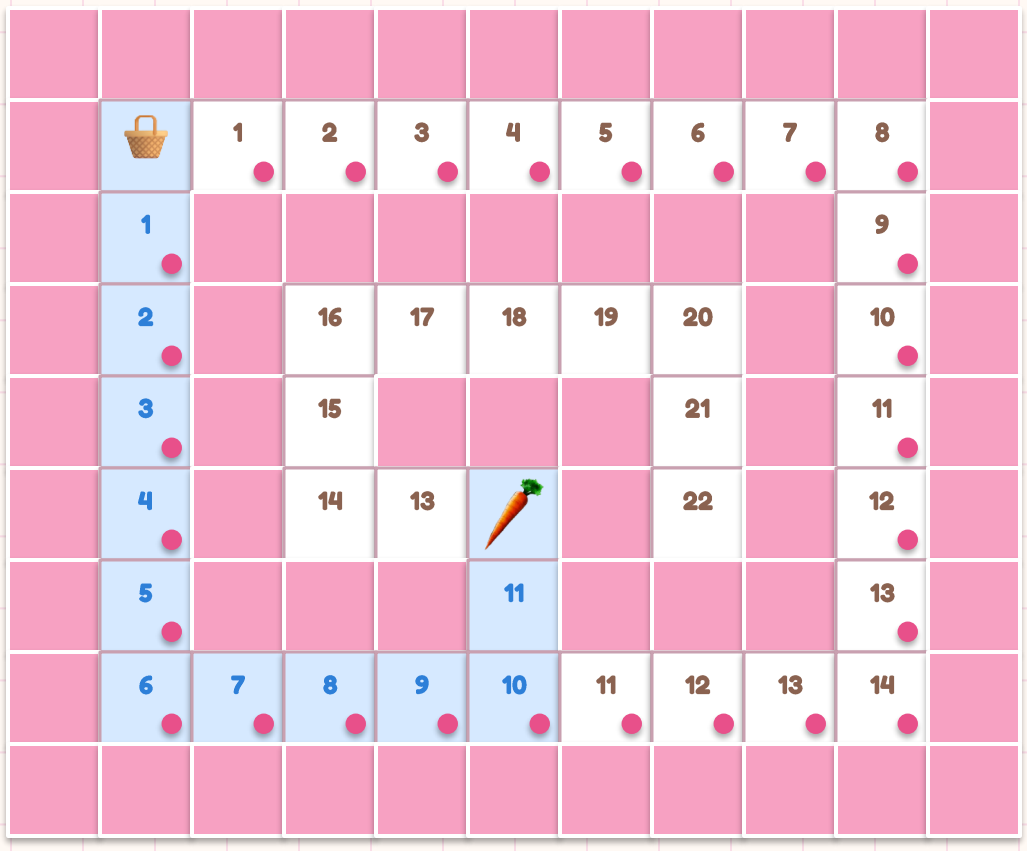
9. The agent's rule for choosing a move in each square. (6)

11. The learner and decision-maker in reinforcement learning (the pup!). (5)

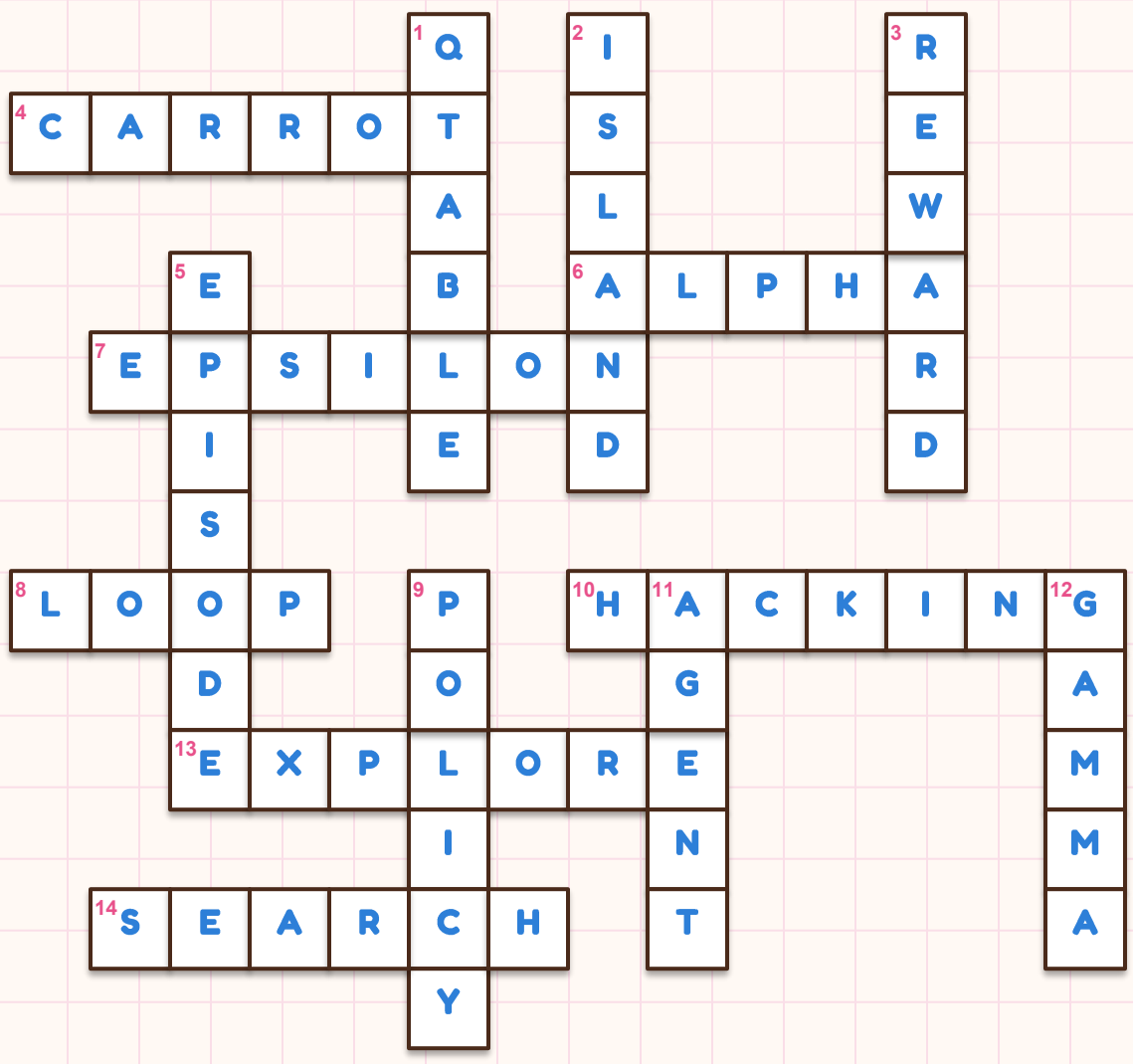
12. Greek letter that sets how much future rewards count. (5)

Answer Key

Paper maze. Pink dots = Rule Chef: it runs round the outer corridor and repeats a move after **29 moves**, never touching the island. Blue squares = Map Chef's shortest path: the carrot gets the number **12**. Numbers show each square's distance from the basket.



Crossword.



Mama's Feedback

Reflection: neither Rule Chef nor Map Chef would do better tomorrow: a fixed rule and a search algorithm don't learn. Only the Learner Pup improves with practice (try Missions 3–4 in the game).

How We Designed & Tested It

Explain

Purpose. Students often call any game AI “learning”. The arcade puts a fixed rule, a search algorithm and Q-learning side by side on the **same mazes**, so the differences are visible, measurable and labelled accurately.

Two formats. The game (online or offline file) gives instant feedback: a speech bubble explains each event, every mission has Mama's feedback, and the quiz explains wrong answers. These PDF pages work on paper, with an answer key.

Kitchen	🐾 Rule Chef	🗺️ Map Chef	🧠 Learner Pup (100 runs)
First Shift	Reached, 29 steps	17 steps	17 steps · reward +3
Island Trap	Loops after 29 steps	12 steps	12 steps · reward +8
Hot Stove (−10 / −1)	Reached, 12 steps	8 steps over 2 🔥	12 steps / 8 steps
Cookie Jar	Reached, 26 steps	16 steps	Farms cookies (reward hacking)

Table B.2 · Results measured in a separate Python re-implementation; Learner results were identical in 10 of 10 random seeds and matched the game.

Real Tech

Accessibility: one offline file; keyboard focus; screen-reader messages; symbols as well as colour; phone layout; reduced-motion support. The paper pages need only a pencil.

Team contribution (fill in before submitting)

Who built, tested and reviewed which part: [TEAM: add names and roles].

Classmate try-out: [TEAM: add how many testers, what confused them, what you changed].

AI use: built with help from an AI assistant (Claude); the team chose the design, checked every number against Recipe 03 and the test script, and is responsible for the content.

DESSERT

Sweet Extras!

Mama says

Every good meal ends with dessert! Here you'll find every key word from the five recipes, the sources behind our numbers, and who made the pictures.

What's on the dessert tray

Glossary A–Z: every **key term** with a one-line meaning and the recipe where it is taught.

References: the main sources of each recipe (full lists in Main/Chapter-01 to 05 of our repository).

Image credits & notes: where the artwork comes from, and how this guide was made.

Graduation: your final stamp!

How to use it

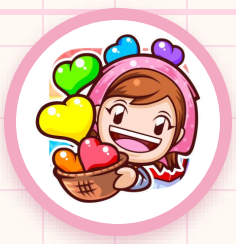
Use the glossary as a **self-test**: cover the meaning column, read a term, explain it out loud, then uncover and check. Any term you can't explain? Jump back to the recipe in brackets.

📖 Glossary A–C

Term	Meaning	Recipe
Accelerator	A chip specialised for one kind of work, such as AI maths (GPU, TPU, NPU).	01
Action	A choice the agent makes at each step.	03
Active sensor	Sends energy out and measures the echo (LiDAR, radar, ultrasonic).	04
ADC	Analogue-to-digital converter: turns a sensor voltage into numbers.	04
Agent	The learner and decision-maker in reinforcement learning.	03
All-reduce	Sums values (e.g. gradients) across all chips and gives every chip the result.	05
Amdahl's law	The serial part of a job limits the speed-up from adding parallel workers.	01
Arithmetic intensity	FLOPs done per byte moved from memory; decides memory- vs compute-bound.	01
ASIL	Automotive Safety Integrity Level (A–D, D strictest) in ISO 26262.	04
Bandwidth	How much data a link or memory can move per second.	01 · 02
Batch	A group of inputs processed together to reuse loaded weights.	01 · 05
BEV (bird's-eye view)	A top-down map around a vehicle where sensor features are fused.	04
Breadth-first search (BFS)	Search that checks squares in rings of equal distance; finds fewest steps.	Bonus
Checkpointing	Regularly saving the full training state so a job can restart after a failure.	05
Cloud computing	Using remote data-centre computers over a network.	02

Glossary page 1 of 6 · the Recipe column says where the term is taught.

👀 **How to read this:** cover the Meaning column and explain each term from memory.



📖 Glossary C–F

Term	Meaning	Recipe
Compiler (AI)	Software that turns a model graph into efficient chip instructions (e.g. XLA).	05
Computation graph	A model drawn as operations (nodes) joined by tensors (arrows).	05
Continuous batching	New requests join a running batch between generated tokens.	05
CPU	General-purpose processor: a few powerful cores, great at branching logic.	01
Data parallelism	Each chip trains on a different part of the batch with a copy of the model.	05
Discount factor (γ)	How much future rewards count compared with immediate ones.	03
Doppler effect	A moving object shifts an echo's frequency; radar uses it to measure speed.	04
DRAM / HBM	Main memory off the compute units; HBM is stacked, very high-bandwidth DRAM.	01
Edge computing	Running computation near the user or device instead of in a distant cloud.	02
ε-greedy	Explore with a random action with probability ε, otherwise exploit the best one.	03
End-to-end driving	One network maps sensor input straight to a trajectory.	04
Environment	Everything outside the agent that responds to its actions.	03
Episode	One run of an RL task from start to finish.	03
Exploration / exploitation	Trying new actions to learn vs using the best known action.	03
Fail-safe / fail-operational	Stops safely / keeps working in a reduced mode after a fault.	04

Glossary page 2 of 6 · the Recipe column says where the term is taught.

👀 **How to read this:** cover the Meaning column and explain each term from memory.

📖 Glossary F–M

Term	Meaning	Recipe
FLOP / TOPS	One floating-point operation / trillion operations per second.	01
Gang scheduling	Starting a job only when all of its chips are free at once.	05
GPU	Thousands of simple cores doing the same maths in parallel.	01
ICI	Inter-chip interconnect: direct high-speed links between TPU chips.	05
Inference	Using a trained model to make predictions.	01
IoU	Intersection over union: overlap ÷ union of a predicted and a true box.	04
IP address	The number that identifies a device on a network so packets can reach it.	02
ISP (image)	Image signal processor: cleans and colours raw camera data.	04
Jitter	Variation in latency from one packet to the next.	02
Kalman filter	Tracks an estimate and its uncertainty with predict and update steps.	04
KV cache	Stored attention state for earlier tokens; grows with conversation length.	05
Latency	The delay between a request (or event) and the response.	02 · 04
LiDAR	Measures distance with laser pulses: d = c × t ÷ 2; builds point clouds.	04
MAC	Multiply-accumulate: a × b + c, the basic step of neural-network maths.	01
Memory-bound / compute-bound	Limited by data movement / by arithmetic speed.	01

Glossary page 3 of 6 · the Recipe column says where the term is taught.

👀 **How to read this:** cover the Meaning column and explain each term from memory.

📖 Glossary M–Q

Term	Meaning	Recipe
MFU	Model FLOPs utilisation: useful model FLOP/s ÷ hardware peak.	05
MTBF	Mean time between failures; system MTBF ≈ chip MTBF ÷ number of chips.	05
NPU	Neural processing unit: a low-power AI accelerator in phones and laptops.	01
OCS	Optical circuit switch: re-patches optical links under software control.	05
Operator fusion	Merging several operations into one pass to avoid memory traffic.	05
Packet	A small chunk of data with an address header, sent across a network.	02
Passive sensor	Measures energy already present (a camera uses existing light).	04
Pipeline parallelism	Different chips hold different groups of layers (stages).	05
Point cloud	A set of 3-D points (x, y, z) measured by LiDAR.	04
Policy	The agent's rule for choosing an action in each state.	03
Precision / recall	Share of detections that are real / share of real objects detected.	04
Protocol	Agreed rules for how devices format and exchange messages.	02
PUE	Power usage effectiveness: facility energy ÷ computing energy.	05
Q-learning	Learns a value Q(s, a) for each state–action pair from rewards.	03
Q-table	The Learner Pup's memory: one value for every square and move.	03 · Bonus

Glossary page 4 of 6 · the Recipe column says where the term is taught.

👀 **How to read this:** cover the Meaning column and explain each term from memory.

📖 Glossary Q–S

Term	Meaning	Recipe
Quantisation	Storing numbers with fewer bits (e.g. INT8) to save memory and energy.	01
Radar	Radio-wave sensor that measures range and speed; robust in bad weather.	04
Real-time system	A system where a late answer is a wrong answer.	04
Reinforcement learning	Learning by trial and error from rewards.	03
Return	The total (discounted) reward collected from a point onwards.	03
Reward	A number from the environment saying how good the last step was.	03
Reward hacking	The agent gets a high reward in a way the designer didn't intend.	03
Roofline model	Chart of attainable speed vs arithmetic intensity: memory slope and compute roof.	01
Round-trip time (RTT)	Time for a message to reach the other side and a reply to come back.	02
Scaling efficiency	Speed-up ÷ number of chips.	05
Sensor fusion	Combining several sensors into one estimate with its uncertainty.	04
Sharding	Splitting a tensor or training state across chips.	05
Sim-to-real	Training in simulation, then transferring the policy to a real robot.	03
SOTIF	ISO 21448: hazards when nothing is broken but a sensor reaches its limits.	04
SRAM	Small, very fast on-chip memory.	01

Glossary page 5 of 6 · the Recipe column says where the term is taught.

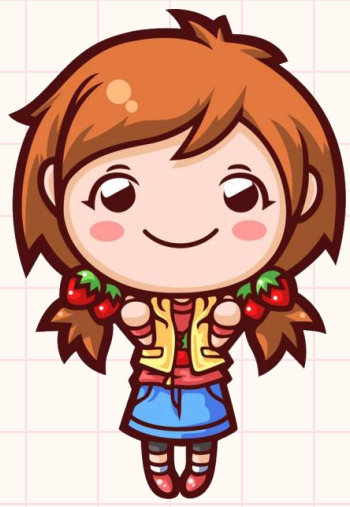
👀 **How to read this:** cover the Meaning column and explain each term from memory.

📖 Glossary S–5

Term	Meaning	Recipe
State	The agent's description of the current situation.	03
Systolic array	A grid of MAC units that pass data to neighbours each cycle (TPU).	01
TCP / UDP	Reliable, ordered transport / fast transport without delivery guarantees.	02
Tensor parallelism	Splitting the matrices inside each layer across chips.	05
Throughput	How much work is completed per second.	01 · 05
Time of flight	Measuring distance from how long an echo takes to return.	04
Torus	A grid network whose edges wrap around to the opposite side.	05
TPU	Google's tensor processing unit, built around a systolic matrix unit.	01 · 05
Training	Adjusting a model's weights from data.	01
Ultrasonic sensor	Sound-echo sensor for short ranges (parking): d = v × t ÷ 2.	04
Utilisation	The share of a chip's peak that is actually used.	01 · 05
Wall following	A fixed rule: keep one hand on the wall; no map, no learning.	Bonus
XLA	The compiler that maps models onto TPUs (and other devices).	05
5G	Fifth-generation mobile network: high bandwidth, low latency, many devices.	02

Glossary page 6 of 6 · the Recipe column says where the term is taught.

👀 **How to read this:** cover the Meaning column and explain each term from memory.



References

APA style, shortened. Full entries are in the References section of each chapter (Main/Chapter-01 to 05) of our repository github.com/Bvmhe/CS-AI-Group.

Recipe 01 · AI Hardware

- Amdahl, G. M. (1967). Validity of the single processor approach to achieving large scale computing capabilities. AFIPS Conference Proceedings.
- Apple Inc. (2017). Apple introduces iPhone X [press release].
- Apple Inc. (2024). Apple Platform Security guide (Face ID and Secure Enclave sections).
- Apple Inc. (2024). Core ML documentation.
- Google. (2025). Ironwood TPU announcement.
- Hennessy, J. L., & Patterson, D. A. (2019). A new golden age for computer architecture. Communications of the ACM, 62(2), 48–60.
- Horowitz, M. (2014). Computing's energy problem (and what we can do about it). IEEE ISSCC.
- Jouppi, N. P., et al. (2017). In-datacenter performance analysis of a Tensor Processing Unit. ISCA 2017.
- NVIDIA. (n.d.). GPU and Tensor Core documentation.
- Qualcomm. (2024). Hexagon NPU documentation.
- Sze, V., et al. (2017). Efficient processing of deep neural networks: A tutorial and survey. Proceedings of the IEEE.
- Williams, S., Waterman, A., & Patterson, D. (2009). Roofline: an insightful visual performance model for multicore architectures. Communications of the ACM, 52(4), 65–76.

References (1/2)

Recipe 02 · Networks & AI Infrastructure

- 3GPP. (n.d.). TS 23.501: System architecture for the 5G System (5GS).
- Al-Fares, M., Loukissas, A., & Vahdat, A. (2008). A scalable, commodity data center network architecture. ACM SIGCOMM 2008.
- Cerf, V. G., & Kahn, R. E. (1974). A protocol for packet network intercommunication. IEEE Transactions on Communications, 22(5), 637–648.
- Dean, J., & Barroso, L. A. (2013). The tail at scale. Communications of the ACM, 56(2), 74–80.
- Deering, S., & Hinden, R. (2017). IPv6 Specification (RFC 8200). IETF.
- Eddy, W. (2022). Transmission Control Protocol (RFC 9293). IETF.
- ETSI. (2014). Mobile-Edge Computing: Introductory Technical White Paper.
- ITU-R. (2015). Recommendation M.2083-0: IMT Vision (IMT-2020).
- ITU-R. (2017). Report M.2410-0: Minimum technical performance requirements for IMT-2020.
- Iyengar, J., & Thomson, M. (2021). QUIC (RFC 9000). IETF.
- Jacobson, V. (1988). Congestion avoidance and control. ACM SIGCOMM 1988.
- Kurose, J. F., & Ross, K. W. (2020). Computer Networking: A Top-Down Approach (8th ed.). Pearson.
- Mathis, M., et al. (1997). The macroscopic behavior of the TCP congestion avoidance algorithm. ACM SIGCOMM Computer Communication Review.
- McMahan, H. B., et al. (2017). Communication-efficient learning of deep networks from decentralized data. AISTATS 2017.
- Mell, P., & Grance, T. (2011). The NIST Definition of Cloud Computing (SP 800-145).
- NVIDIA. (n.d.). NVLink and NVSwitch documentation.

References (2/2)

Recipe 02 · Networks & AI Infrastructure

- O-RAN Alliance. (n.d.). O-RAN architecture description and RIC specifications.
- Patarasuk, P., & Yuan, X. (2009). Bandwidth optimal all-reduce algorithms for clusters of workstations. JPDC, 69(2), 117–124.
- Postel, J. (1980). User Datagram Protocol (RFC 768); (1981). Internet Protocol (RFC 791). IETF.
- Rescorla, E. (2018). TLS Protocol Version 1.3 (RFC 8446). IETF.
- Satyanarayanan, M. (2017). The emergence of edge computing. Computer, 50(1), 30–39.
- Shi, W., et al. (2016). Edge computing: Vision and challenges. IEEE Internet of Things Journal, 3(5), 637–646.

References (1/2)

Recipe 03 · Behaviourist AI & RL

- Amodei, D., et al. (2016). Concrete problems in AI safety. arXiv.
- Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. Machine Learning, 47, 235–256.
- Brooks, R. A. (1986). A robust layered control system for a mobile robot. IEEE J. Robotics and Automation; (1991). Intelligence without representation. Artificial Intelligence, 47, 139–159.
- Chomsky, N. (1959). A review of B. F. Skinner's Verbal Behavior. Language, 35(1), 26–58.
- Clark, J., & Amodei, D. (2016). Faulty reward functions in the wild [blog post]. OpenAI.
- Haarnoja, T., et al. (2019). Learning to walk via deep reinforcement learning. RSS 2019.
- Hwangbo, J., et al. (2019). Learning agile and dynamic motor skills for legged robots. Science Robotics.
- Kumar, A., Fu, Z., Pathak, D., & Malik, J. (2021). RMA: Rapid motor adaptation for legged robots. RSS.
- Lee, J., et al. (2020). Learning quadrupedal locomotion over challenging terrain. Science Robotics, 5(47).
- Makoviychuk, V., et al. (2021). Isaac Gym: High performance GPU-based physics simulation for robot learning. NeurIPS Datasets and Benchmarks.
- Mnih, V., et al. (2015). Human-level control through deep reinforcement learning. Nature, 518, 529–533.
- Ng, A. Y., Harada, D., & Russell, S. (1999). Policy invariance under reward transformations. ICML 1999.
- Pavlov, I. P. (1927). Conditioned Reflexes. Oxford University Press.
- Rudin, N., et al. (2022). Learning to walk in minutes using massively parallel deep reinforcement learning. PMLR (CoRL).
- Skinner, B. F. (1938). The Behavior of Organisms. Appleton-Century.
- Smith, L., Kostrikov, I., & Levine, S. (2022). A walk in the park: Learning to walk in 20 minutes with model-free RL. arXiv.

References (2/2)

Recipe 03 · Behaviourist AI & RL

- Sutton, R. S. (1988). Learning to predict by the methods of temporal differences. Machine Learning, 3, 9–44.
- Sutton, R. S., & Barto, A. G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press.
- Tan, J., et al. (2018). Sim-to-real: Learning agile locomotion for quadruped robots. RSS 2018.
- Thorndike, E. L. (1911). Animal Intelligence: Experimental Studies. Macmillan.
- Tobin, J., et al. (2017). Domain randomization for transferring deep neural networks from simulation to the real world. IROS 2017.
- Watkins, C. J. C. H., & Dayan, P. (1992). Q-learning. Machine Learning, 8, 279–292.
- Watson, J. B. (1913). Psychology as the behaviorist views it. Psychological Review, 20(2), 158–177.

References (1/2)

Recipe 04 · AI Sensors

- Bojarski, M., et al. (2016). End to end learning for self-driving cars. arXiv:1604.07316.
- Caesar, H., et al. (2020). nuScenes: A multimodal dataset for autonomous driving. CVPR 2020.
- Feng, D., et al. (2021). Deep multi-modal object detection and semantic segmentation for autonomous driving. IEEE Transactions on Intelligent Transportation Systems.
- Geiger, A., Lenz, P., & Urtasun, R. (2012). Are we ready for autonomous driving? The KITTI vision benchmark suite. CVPR 2012.
- Hall, D. L., & Llinas, J. (1997). An introduction to multisensor data fusion. Proceedings of the IEEE, 85(1), 6–23.
- Hu, Y., et al. (2023). Planning-oriented autonomous driving. CVPR 2023.
- ISO. (2018). ISO 26262: Road vehicles: Functional safety. ISO. (2022). ISO 21448: Safety of the intended functionality.
- Kalman, R. E. (1960). A new approach to linear filtering and prediction problems. Journal of Basic Engineering, 82(1), 35–45.
- Li, Z., et al. (2022). BEVFormer. ECCV 2022. Liu, Z., et al. (2023). BEVFusion. ICRA 2023.
- Philion, J., & Fidler, S. (2020). Lift, Splat, Shoot. ECCV 2020.
- Rosique, F., et al. (2019). A systematic review of perception system and simulators for autonomous vehicles research. Sensors.
- SAE International. (2021). J3016: Taxonomy and definitions for terms related to driving automation.
- Sun, P., et al. (2020). Scalability in perception for autonomous driving: Waymo Open Dataset. CVPR 2020.
- Talpes, E., et al. (2020). Compute solution for Tesla's Full Self-Driving computer. IEEE Micro, 40(2), 25–35.
- Thrun, S., Burgard, W., & Fox, D. (2005). Probabilistic Robotics. MIT Press.
- Wang, Y., et al. (2019). Pseudo-LiDAR from visual depth estimation. CVPR 2019.

References (2/2)

Recipe 04 · AI Sensors

- Winner, H., et al. (Eds.). (2016). Handbook of Driver Assistance Systems. Springer.
- Tesla. (2019). Autonomy Day; (2021, 2022). Tesla AI Day [presentations]. Karpathy, A. (2021). CVPR Workshop on Autonomous Driving keynote.
- Tesla Support. (2021). Transitioning to Tesla Vision; (2022). Replacing Ultrasonic Sensors with Tesla Vision.
- TechCrunch; Electrek. (2022, October). Reports on Tesla removing ultrasonic sensors.
- Waymo. (2024, August 19). Meet the 6th-generation Waymo Driver. Electrek. (2026, February 12). Waymo begins fully autonomous ops with 6th-gen Driver.
- Teslarati. (2026). Coverage of Tesla's Q1 2026 earnings call (HW3 memory bandwidth). Not a Tesla App. (2026). Coverage of AI5 chip timing.
- Elluswamy, A. (2026, April 27). Post on X on camera-only FSD, as reported by Basenor. Company claim; not independently verified.
- NHTSA. (n.d.). Investigations and recalls. nhtsa.gov.

References

Recipe 05 · Integration & Optimisation

- Barroso, L. A., Hölzle, U., & Ranganathan, P. (2018). The Datacenter as a Computer (3rd ed.). Morgan & Claypool.
- Google. (n.d.). Data centers: efficiency (PUE reporting). Google Cloud. (n.d.). Cloud TPU documentation.
- Hoffmann, J., et al. (2022). Training compute-optimal large language models. NeurIPS 2022.
- Jouppi, N. P., et al. (2023). TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. ISCA 2023.
- Kaplan, J., et al. (2020). Scaling laws for neural language models. arXiv:2001.08361.
- Kwon, W., et al. (2023). Efficient memory management for LLM serving with PagedAttention. SOSP 2023.
- Narayanan, D., et al. (2021). Efficient large-scale language model training on GPU clusters using Megatron-LM. SC21.
- Patterson, D., et al. (2021). Carbon emissions and large neural network training. arXiv:2104.10350.
- Rajbhandari, S., et al. (2020). ZeRO: Memory optimizations toward training trillion parameter models. SC20.
- Xu, Y., et al. (2021). GSPMD: General and scalable parallelization for ML computation graphs. arXiv:2105.04663.
- Young, J. W. (1974). A first order approximation to the optimum checkpoint interval. Communications of the ACM, 17(9), 530–531.
- Also cited in Recipe 05: Amdahl (1967), Hennessy & Patterson (2019), Horowitz (2014), Jouppi et al. (2017), Patarasuk & Yuan (2009), Williams et al. (2009): see above.

Video List

Recipe	Video · creator · length	YouTube ID
01	But what is a neural network? · 3Blue1Brown · 18:40	aircAruvnKk
01	How do Graphics Cards Work? · Branch Education · 28:29	h9Z4oGN89MU
01	A very short intro to the Roofline model · NHR@FAU · 18:36	IrkNZG8MJ64
02	How does the INTERNET work? · Lesics · 9:00	x3c1ih2NJEg
02	What is bandwidth? How is it different than latency? · internet-class · 3:24	QLJCqY-7Vrw
02	Inside a Google data center · Google Workspace · 5:28	XZmGGAbHqa0
03	Reinforcement Learning: ML Meets Control Theory · Steve Brunton · 26:00	0MNVhXEX9to
03	Markov Decision Processes · Computerphile · 17:40	2iF9PRriA7w
03	AI Learns to Walk · AI Warehouse · 8:36	L_4BPjLBF4E
04	How do self-driving cars "see"? · TED-Ed · 5:25	PRg5RNU_JLk
04	How Does LiDAR Remote Sensing Work? · NEON Science · 7:42	EYbhNSUnIdU
04	Kalman Filters · Visually Explained · 11:18	IFeCIbljreY
04	Tesla AI Day 2021 · Tesla · 3:03:20	j0z4FweCy4M
05	A friendly introduction to distributed training · TensorFlow · 24:20	S1tN9a4Proc
05	Scaling AI with Google Cloud's TPUs · Google Cloud Tech · 5:54	8zBaa20mvoc
05	Inside a Google Cloud TPU Data Center · Google Cloud · 2:42	FsxthdQ_sL4

All links have the form youtube.com/watch?v=<ID>. IDs, titles and creators were checked by the team on 4 November 2026.

How to read this: each video page in the recipes says which part to watch, what to do, and its limitations.

Image Credits & Notes

Artwork

Characters, dishes, ingredients and stickers: **Cooking Mama** series © Office Create Corp.; published on Nintendo platforms by Majesco Entertainment / Taito / 505 Games and others. Sprites were cut from the team's collected game images by removing backgrounds and cropping.

Used only for a **non-commercial university class assignment**, for illustration and commentary. **Not for sale, not for public distribution**, and no ownership of the artwork is claimed. If this guide is ever shared beyond the class, the artwork must be replaced with original or licensed images.

Fonts & layout

Fonts: Fredoka (Google Fonts, SIL Open Font License) for headings; Arial for body text; Segoe UI Emoji (Microsoft) for icons.

Layout, frames, gingham pattern, cards, diagrams and charts: made by the team. Every diagram is a native, editable shape: nothing important is a flat picture.

Who made this

Content: written from the team's five shared Markdown chapters (Main/Chapter-01 to 05), reviewed by every member. **Videos:** selected and timestamped by the Video Lead. **Accuracy check** of all worked examples and answers: Quality Lead and Content Leads.

AI use: we used an AI assistant (Claude) to help turn our chapters into page layouts, draft analogies, and check arithmetic. Every number was re-checked against our chapters' worked examples; all references come from our chapters.

A note on numbers

Tesla, Waymo and TPU figures change quickly; some are **reported** or **company claims** rather than independently verified. Numbers marked **illustrative** are chosen for teaching. Check current sources before quoting.

Credits

You did it, Head Chef!

Graduation

You now know **which cook does the maths** (chips), **how the orders travel** (networks), **how the pup learns** (rewards), **how the chef senses** (sensors and fusion), and **how to run the whole banquet** (systems).

Next: try the Test Bank and see how many **Gold** stamps you collect!

Hot Wok, Smart Chips: We're Cooking!

Hardware, Networks & Behaviourist AI · Stage 2 Study Guide · November 2026

github.com/Bvmhe/CS-AI-Group

The end!